

EEBus Technical Specification

Smart Home IP

Version 1.1.0

Cologne, 2025-09-22

EEBus Initiative e.V.

Deutz-Mülheimer-Straße 183
51063 Cologne
GERMANY

Rue d'Arlon 25
1050 Brussels
BELGIUM

Phone: +49 221 / 715 938 - 0
Fax: +49 221 / 715 938 - 99

info@eebus.org
www.eebus.org

District court: Cologne
VR 17275

Terms of use for publications of EEBus Initiative e.V.**General information**

The specifications, particulars, documents, publications and other information provided by the EEBus Initiative e.V. are solely for general informational purposes. Particularly specifications that have not been submitted to national or international standardisation organisations by EEBus Initiative e.V. (such as DKE/DIN-VDE or IEC/CENELEC/ETSI) are versions that have not yet undergone complete testing and can therefore only be considered as preliminary information. Even versions that have already been published via standardisation organisations can contain errors and will undergo further improvements and updates in future.

Liability

EEBus Initiative e.V. does not assume liability or provide a guarantee for the accuracy, completeness or up-to-date status of any specifications, data, documents, publications or other information provided and particularly for the functionality of any developments based on the above.

Copyright, rights of use and exploitation

The specifications provided are protected by copyright. Parts of the specifications have been submitted to national or international standardisation organisations by EEBus Initiative e.V., such as DKE/DIN-VDE or IEC/CENELEC/ETSI, etc. Furthermore, all rights to use and/or exploit the specifications belong to the EEBus Initiative e.V., Deutz-Mülheimer-Straße 183, 51063 Cologne, Germany and can be used in accordance with the following regulations:

The use of the specifications for informational purposes is allowed. It is therefore permitted to use information evident from the contents of the specifications. In particular, the user is permitted to offer products, developments and/or services based on the specifications.

Any respective use relating to standardisation measures by the user or third parties is prohibited. In fact, the specifications may only be used by EEBus Initiative e.V. for standardisation purposes. The same applies to their use within the framework of alliances and/or cooperations that pursue the aim of determining uniform standards.

Any use not in accordance with the purpose intended by EEBus Initiative e.V. is also prohibited.

Furthermore, it is prohibited to edit, change or falsify the content of the specifications. The dissemination of the specifications in a changed, revised or falsified form is also prohibited. The same applies to the publication of extracts if they distort the literal meaning of the specifications as a whole.

It is prohibited to pass on the specifications to third parties without reference to these rights of use and exploitation.

It is also prohibited to pass on the specifications to third parties without informing them of the authorship or source.

Without the prior consent of EEBus Initiative e.V., all forms of use and exploitation not explicitly stated above are prohibited.

1	Table of contents	
2	Table of contents.....	3
3	List of figures	4
4	List of tables	4
5	Introduction.....	6
6	1 Scope	7
7	1.1 Key features of SHIP 1.1.....	7
8	2 Normative References.....	8
9	3 Terms and Definitions	10
10	4 Architecture Overview	14
11	4.1 General Considerations On Closing Communication Channels	15
12	4.2 SHIP Node Parameters.....	15
13	5 Registration	17
14	5.1 Successful Registration	18
15	5.2 Registration details and recommendations (informative).....	19
16	6 Reconnection.....	20
17	6.1 Reconnection details in case of changed key material (informative).....	20
18	7 Discovery	22
19	7.1 Service Instance	22
20	7.2 Service Name	22
21	7.3 Multicast DNS Name	22
22	7.4 Recommendations for re-discovery.....	25
23	8 TCP.....	26
24	8.1 Limited Connection Capabilities	26
25	8.2 Online Detection	26
26	8.3 TCP Connection Establishment	27
27	8.4 Retransmission Timeout	27
28	9 TLS	28
29	9.1 Cipher Suites	28
30	9.2 Maximum Fragment Length.....	29
31	9.3 TLS Compression	29
32	9.4 Server Name Indication	29
33	9.5 Renegotiation.....	29
34	9.6 Session Resumption	29
35	9.7 TLS extension for ECC.....	30
36	9.8 TLS Probing.....	31
37	10 WebSocket	32
38	10.1 TLS Dependencies	32
39	10.2 Opening Handshake	32
40	10.3 Data Framing.....	32
41	10.4 Connection Keepalive	33
42	11 Message Representation Using JSON Text Format	34
43	11.1 Introduction	34
44	11.2 Definitions.....	34

45	11.3	Examples For Each Type.....	34
46	11.4	XML to JSON Transformation.....	35
47	11.5	JSON to XML Transformation.....	42
48	12	Key Management	43
49	12.1	Certificates	43
50	12.2	SHIP Node Specific Public Key.....	49
51	12.3	Verification Procedure	52
52	12.4	Symmetric Key	60
53	12.5	SHIP Node PIN	60
54	12.6	SHIP Commissioning Tool.....	61
55	12.7	QR Code	64
56	13	SHIP Data Exchange.....	68
57	13.1	Introduction	68
58	13.2	Terms and Definitions.....	68
59	13.3	Protocol Architecture / Hierarchy.....	70
60	13.4	SHIP Message Exchange.....	72
61	14	Well-known protocolId.....	122
62			

63 List of figures

64	Figure 1: Physical Connections in the Overall System.....	14
65	Figure 2: SHIP Stack Overview.....	14
66	Figure 3: Full TLS 1.2 Handshake with mutual authentication.....	28
67	Figure 4: Quick TLS Handshake with Session Resumption	30
68	Figure 5: Easy Mutual Authentication with QR codes and Smart Phone	60
69	Figure 6: QR Code Model 2, "low" ECC level, 0.33mm/Module, with SKI and PIN	67
70	Figure 7: QR Code Model 2, "low" ECC level, 0.33mm/module, with all values.....	67
71	Figure 8: QR Code Model 2, "low" ECC level, 0.33mm/module, with brainpoolP256r1 SKI and	
72	brainpoolP384r1 SKI	67
73	Figure 9: Protocol Architecture and Hierarchy	71
74	Figure 10: CMI Message Sequence Example.....	76
75	Figure 11: Connection State "Hello" Sequence Example Without Prolongation Request: "A"	
76	and "B" already trust each other; "B" is slower/delayed.....	84
77	Figure 12: Connection State "Hello" Sequence Example With Prolongation Request.	85
78	Figure 13: Connection State "Protocol Handshake" Message Sequence Example	92
79	Figure 14: Connection State "PIN verification" Message Sequence Example (Begin)	103
80		

81 List of tables

82	Table 1: SHIP Parameters Default Values.....	16
83	Table 2: Mandatory Parameters in the TXT Record	23
84	Table 3: Optional Parameters in the TXT Record	24
85	Table 4: Mapping from the XSD Types to JSON Types.	35

86	Table 5: Transformation of a simple type.	36
87	Table 6: Mapping from the XSD compositors to JSON Types.....	36
88	Table 7: Examples for XML and JSON representations.	40
89	Table 8: Example transformation of several combined XSD item types.	41
90	Table 9: Example for JSON to XML transformation.....	42
91	Table 10: Trust Levels	59
92	Table 11: MessageType Values	73
93	Table 12: Structure of SmeHelloValue of SME "hello" Message.....	78
94	Table 13: Structure of SmeProtocolHandshakeValue of SME "Protocol Handshake" Message.	87
95	Table 14: Structure of SmeProtocolHandshakeErrorValue of SME "Protocol Handshake Error"	
96	Message.....	87
97	Table 15: Values of Sub-element "error" of messageProtocolHandshakeError.	91
98	Table 16: Structure of SmeConnectionPinStateValue of SME "Pin state" message.	93
99	Table 17: Structure of SmeConnectionPinInputValue of SME "Pin input" message.....	94
100	Table 18: Structure of SmeConnectionPinErrorValue of SME "Pin error" message.	95
101	Table 19: Values of Sub-element "error" of connectionPinError.....	102
102	Table 20: Structure of MessageValue of "data" Message.....	105
103	Table 21: Structure of SmeConnectionAccessMethodsRequestValue of SME "Access methods	
104	request" message.	108
105	Table 22: Structure of SmeConnectionAccessMethodsValue of SME "Access methods"	
106	message.....	109
107	Table 23: Structure of SmeConnectionCommissioningRequestValue of SME "commissioning	
108	request" message.....	111
109	Table 24: Structure of SmeConnectionCommissioningResponseValue of SME "commissioning	
110	response" message.....	111
111	Table 25: Structure of SmeConnectionKeyMaterialRequestValue of SME "key material	
112	request" message.....	112
113	Table 26: Structure of SmeConnectionKeyMaterialValue of SME "key material" message.	113
114	Table 27: Structure of SmeConnectionKeyMaterialResponseValue of SME "key material	
115	response" message.....	114
116	Table 28: Structure of SmeConnectionKeyMaterialDeleteValue of SME "key material delete"	
117	message.....	115
118	Table 29: Structure of SmeConnectionKeyMaterialDeleteResponseValue of SME "key material	
119	delete response" message.	116
120	Table 30: Structure of SmeConnectionKeyMaterialStateValue of SME "key material state"	
121	message.....	117
122	Table 31: Structure of SmeConnectionKeyMaterialStateResponseValue of SME "key material	
123	state response" message	118
124	Table 32: Structure of SmeConnectionKeyMaterialStateRequestValue of SME "key material	
125	state request" message.....	119
126	Table 33: Structure of SmeCloseValue of SME "close" Message.	120
127		

128

Introduction

Over the past decades, different home automation technologies have been created, which connect devices using digital communication technologies. Most of these solutions bring along an infrastructure of their own, like dedicated home automation wires. These approaches are acceptable for commercial and industrial buildings, but they are too complex for private homes, especially if retrofitting into already existing infrastructure is necessary. For these cases, wireless technologies have been introduced to make installation easier.

In the meantime, even private homes have been expanded with IP (Internet Protocol) based installations by home or flat owners. IP based devices fitting different consumer needs have become increasingly popular over the past years. This means that most likely, a communication infrastructure is already available in private households. Additionally, there are a lot of potential extensions to other domains than just home automation, since smart phones, PCs, cloud communication, etc. continuously broaden the horizon of possible applications.

However, there is a need for a secure standardized TCP/IP protocol based on requirements for the next generation network within the Internet of Things (IoT). Things, in the IoT, can refer to a wide variety of devices and will bring a lot of additional possibilities, e.g. within home automation, Smart Grid, Smart Home or Ambient Assisted Living (AAL).

This specification describes an IP based approach for plug and play home automation, which can easily be extended to additional domains. The solution is called SHIP (Smart Home IP), with the communicating devices being called SHIP nodes.

1 Scope

This document describes an IP (Internet Protocol) based approach for machine-to-machine communication.

It describes all relevant mechanisms between the Network Layer (layer 3) and Application Layer (layer 7) based on the seven-layer ISO-OSI model.

The goal is to obtain a secure TCP/IP-based solution that allows interoperable connectivity between different implementers and vendors.

Communication security is in line with the Smart Meter Gateway HAN (Home Area Network) interface as described by the Federal Office for Information Security Germany (BSI) in TR-03109 Version 1.0, while also providing scalability, a high degree of usability, and efficient mechanisms for simple devices.

Scalability is an important design principle for SHIP, as a wide variety of devices should be addressed within the SHIP-protocol. Simple devices with limited connection capabilities (worst case assumption: only one simultaneous connection) or no or simple user interfaces (e.g. push button) shall be enabled, as well as gateway or cloud solutions with enhanced capabilities.

To provide a future-proof solution, this specification also defines different mechanisms for downward compatible extensibility.

1.1 Key features of SHIP 1.1

In essence, the following features and respective messages and procedures have been introduced in SHIP version 1.1:

- Report of certificate changes between SHIP nodes (section 12.1.3 and partly 13.4.7).
- Management of "foreign" certificates via SHIP commissioning tool (sections 12.3.1.6, 12.6, and partly 13.4.7).
- Support of additional cryptographic curves (sections 9.7, 12.2.1, 12.7 and some more).
- Discovery and ranking of additional TLS ECC curves/public keys (sections 9.8, 9.7, 12.3).

2 Normative References

The following documents, in whole or in part, are normatively referenced in this document and are indispensable for its application. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

IETF RFC 768: 1981, User Datagram Protocol

IETF RFC 793: 1981, Transmission Control Protocol

IETF RFC 1035: 1987, Domain Names

IETF RFC 2104: 1997, HMAC, Keyed-Hashing for Message Authentication

IETF RFC 2119: 1997, Key words for use in RFCs to indicate requirement levels

IETF RFC 3279: 2002, Algorithms and Identifiers for the Internet X.509 Public Key Infrastructure certificate and Certificate Revocation List (CRL) Profile

IETF RFC 3280: 2002, Internet X.509 Public Key Infrastructure Certificate Revocation List (CRL) Profile

IETF RFC 4055: 2005, The Additional Algorithms and Identifiers for RSA Cryptography for use in the Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile

IETF RFC 4627: 2006, The application/JSON Media Type for JavaScript Object Notation (JSON)

IETF RFC 5077: 2008, Transport Layer Security (TLS) Session Resumption without Server-Side State

IETF RFC 5234: 2008, Augmented BNF for Syntax Specifications: ABNF

IETF RFC 5246: 2008, The Transport Layer Security (TLS) Protocol Version 1.2

IETF RFC 5280: 2008, Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile

IETF RFC 5289: 2008, TLS Elliptic Curve Cipher Suites with SHA-256/384 and AES Galois Counter Mode (GCM)

IETF RFC 5480: 2009, Elliptic Curve Cryptography Subject Public Key Information

IETF RFC 5758: 2010, Internet X.509 Public Key Infrastructure: Additional Algorithms and Identifiers for DSA and ECDSA

IETF RFC 6066: 2011, Transport Layer Security (TLS) Extensions

IETF RFC 6090: 2011, Fundamental Elliptic Curve Cryptography Algorithms

IETF RFC 6298: 2011, Computing TCP's Retransmission Timer

IETF RFC 6455: 2011, The WebSocket Protocol

IETF RFC 6762: 2013, Multicast DNS

- 205 IETF RFC 6763: 2013, DNS-Based Service Discovery
- 206 IETF RFC 7027: 2013, Elliptic Curve Cryptography (ECC) Brainpool Curves for Transport Layer Security
207 (TLS)
- 208 IETF RFC 7320: 2014, URI Design and Ownership
- 209 ISO/IEC 18004:2015: Information technology — Automatic identification and data capture
210 techniques — QR Code bar code symbology specification
- 211 IANA TLS Parameters: Internet Assigned Numbers Authority, "Transport Layer Security (TLS)
212 Parameters", <http://www.iana.org/assignments/tls-parameters>.
- 213

3 Terms and Definitions

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in IETF RFC 2119.

0.., 0..256, 1.. etc.

Number of instances or value range. This notation is used in tables that explain the content of a message. For a single element that can take an integer value, a notation like "a..b" expresses the permitted value range of the element (at least a, at maximum b). For list elements (elements that can occur multiple times) it expresses the number of occurrences of the element (at least a, at maximum b). If "b" is not given (i.e. just "a.." like in "1.."), then there is no explicit upper bound.

CA

Certificate Authority or Certification Authority. A CA can provide a digital signature for certificates. Other SHIP nodes can check this digital signature with the certificate from the CA itself, the "CA-certificate".

Commissioning Tool

In the scope of SHIP, a commissioning tool may be used to establish the trust between different devices in the smart home installation, e.g. distribute trustworthy credentials from some SHIP nodes to other SHIP nodes. E.g. a smart phone, a web server or a dedicated device can embody the role of a commissioning tool. So far, the SHIP specification does not specify a commissioning tool. An interoperable protocol for commissioning can be used on the layer above SHIP. A manufacturer may also use their own solutions.

DNS

Domain Name System, see IETF RFC 1035.

DNS host name

Fully qualified domain name used within DNS as host name to get the IP address of the corresponding internet host.

DNS-SD

Domain Name System – Service discovery, see IETF RFC 6763.

EUI

Extended Unique Identifier, see <http://standards.ieee.org/develop/regauth/tut/eui64.pdf>.

Factory Default

A factory default SHALL allow the user to reset the SHIP node to the as-new condition. This means that all data that has been provided and stored by the SHIP node during its operation time SHALL be deleted.

Foreign key material, foreign SKI, foreign public key

For a SHIP node "A", key material is "foreign" if it is not the "own key material" of SHIP node "A".

- 251 **IANA**
252 Internet Assigned Numbers Authority.
- 253 **IP**
254 Internet Protocol.
- 255 **LAN**
256 Local Area Network.
- 257 **MAC**
258 Media Access Control.
- 259 **mDNS, multicast DNS host name**
260 Fully qualified domain name used within mDNS as host name to get the IP address of the
261 corresponding local SHIP node.
- 262 **Numerical representation**
263 0x introduces the hexadecimal representation of an unsigned value. For example, 0xab represents a
264 decimal value of 171.
- 265 **Own certificate**
266 A certificate that authenticates a SHIP node "A" is called an "own certificate" of SHIP node "A".
- 267 **Own key material**
268 Key material that belongs to an "own certificate".
- 269 **PIN**
270 Personal Identification Number. This specification makes use of a PIN as secret for SHIP specific
271 verification procedures.
- 272 **PKI**
273 Public Key Infrastructure.
- 274 **Push Button**
275 The term Push Button is used to describe a simple trigger mechanism. A Push Button event does not
276 necessarily mean that a real physical button has to be used to trigger this event. A Push Button event
277 may also be generated by other means, e.g. via a smart phone application or a web-interface (secure
278 connection to SHIP node required). A Push Button shall provide a simple mechanism for a user to
279 bring the device to a certain state or start a certain process.
- 280 **QR Code**
281 The term "QR Code" is a registered trademark of DENSO WAVE INCORPORATED. "QR Code" is the
282 short form for "Quick Response Code" and used for efficient encoding of data into a small graphic.
- 283 Among others, the international standard ISO/IEC 18004:2015 specifies the encoding of QR code
284 symbols.
- 285 **RFC**
286 Request for comments.

287 SHIP

288 Abbreviation of "Smart Home IP". This term is used throughout this document to refer to the
289 described communication protocol.

290 SHIP Commissioning

291 The term SHIP commissioning denotes the distribution of trustworthy SKIs from one SHIP node to
292 another. The distribution of the trustworthy SKIs is handled by a so-called SHIP commissioning tool.

293 SHIP Commissioning Tool

294 A SHIP commissioning tool is used to distribute trustworthy SKIs from certain SHIP nodes to other
295 SHIP nodes. It allows a user to handle the trust relationships in the whole SHIP installation (if each
296 node in the installation supports commissioning) over one simple user interface.

297 SHIP ID

298 Each SHIP node has a globally unique SHIP ID. The SHIP ID is used to uniquely identify a SHIP node,
299 e.g. in its service discovery. This ID is present in the mDNS/DNS-SD local service discovery, SME "key
300 material" messages and SME "Access methods" messages (see also section 7.4).

301 SHIP Client

302 The SHIP client role shall be assigned to the SHIP node that also embodies the TCP client role for a
303 specific peer-to-peer connection.

304 SHIP Node

305 A SHIP node is a logical device which communicates via the described SHIP protocol and can be
306 integrated into a web server or physical device.

307 Note: One physical device may have more than one logical SHIP node. In this case, each SHIP node
308 MUST use distinct ports (e.g. a physical device provides 3 open ports with 3 different SHIP services).

309 SHIP Server

310 The SHIP server role shall be assigned to the SHIP node that also embodies the TCP server role for a
311 specific peer-to-peer connection.

312 SKI

313 Each SHIP node has a specific public key. The Subject Key Identifier (SKI) is derived from this public
314 key and is used as a cryptographically backed identification and authentication criterion.

315 SPINE

316 Smart Premises Interoperable Neutral message Exchange: Technical Specification of EEBus Initiative
317 e.V.

318 Trusted SHIP Node

319 A trusted SHIP node is a term which is only applicable from a specific SHIP node point of view.

320 If SHIP node A has a communication partner and a trusted relationship to SHIP node B, SHIP node B is
321 called a trusted SHIP node from SHIP node A's point of view. A trusted relationship can be
322 established in different ways, as described in section 12.2.3.

323 UCS

- 324 Universal Character Set.
- 325 **UTF**
- 326 UCS Transformation Format. A computing industry standard for the consistent encoding,
- 327 representation, and handling of text expressed in most of the world's writing systems.
- 328 **WAN**
- 329 Wide Area Network.
- 330 **Web server based SHIP node**
- 331 A SHIP node that is hosted by a web server.
- 332 **WiFi**
- 333 IP networks based on the IEEE802.11 set of standards, used for wireless IP communication.

4 Architecture Overview

Smart Home IP (SHIP) describes an IP-based approach for interoperable connectivity of smart home appliances, which covers local SHIP nodes in the smart home as well as web server based SHIP nodes and remote SHIP nodes.

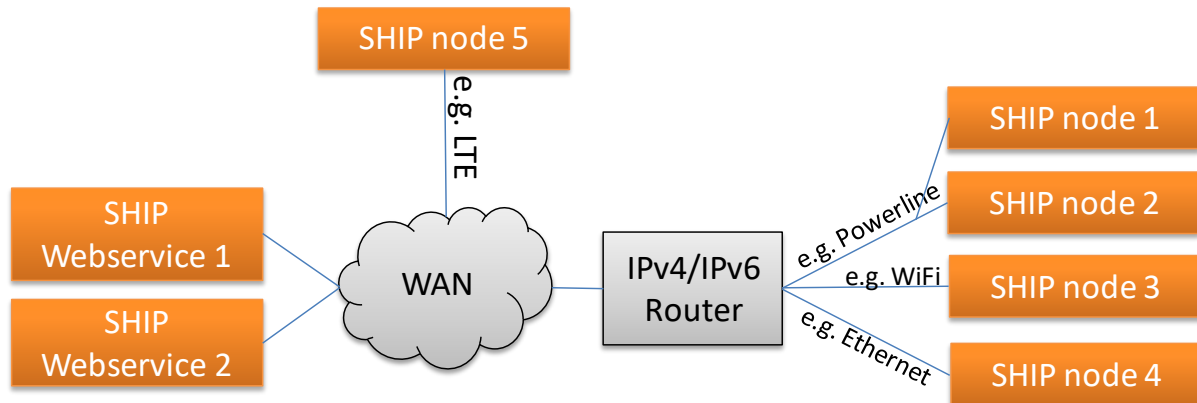


Figure 1: Physical Connections in the Overall System

SHIP nodes MAY base on different physical layer approaches, e.g. WiFi or powerline technologies. An IP router can be used to connect different physical networks and provide access to the internet, but this is out of the scope of the SHIP specification.

On the IP Layer, IPv4 as well as IPv6 are permitted. IP addresses can be preconfigured, assigned via DNS-server, with SLAAC, or by any other appropriate means.

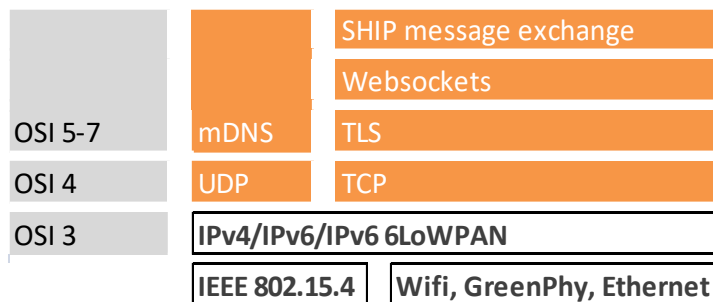


Figure 2: SHIP Stack Overview

A SHIP node SHALL support mDNS/DNS-SD for local device/service discovery. The SHIP protocol is based on TCP, TLS and WebSocket.

Note: Computationally limited SHIP nodes MAY only support a limited number of connections. For further information see section 8.1.

Note: A SHIP node MUST always provide a server port. Only a SHIP node that supports only one simultaneous active connection MAY close the server port in order to establish a client connection.

In SHIP, it is not important which SHIP node takes over the server or client role. If two SHIP nodes try to connect to each other virtually simultaneously, double connections are prevented by the mechanism described in section 12.2.3.

SHIP specific messages conveyed over the established WebSocket connection shall be encoded using JSON, like described in chapter 11.

As the SHIP specification is defined by members of the EEBus Initiative e.V., it can be used to transport EEBus-specific payload and provide an EEBus IP-Backbone, but also other protocols can be used above SHIP. To provide a clean solution, SHIP is defined without dependencies to the EEBus data model.

4.1 General Considerations On Closing Communication Channels

In general, different manufacturers will favour different strategies on opening and closing communication channels. Some manufacturers will choose to leave communication channels open as long as possible, while others will choose to close communication channels as soon as possible. In practice, limitations of the number (or duration) of supported parallel connections might force connections to be closed at least temporarily. Thus, a termination process is defined in order to distinguish sudden interrupts or failures from (typically) temporary disconnections. Details are explained in section 13.4.8.

4.2 SHIP Node Parameters

Throughout this specification, different parameters are defined to provide an exact description of the SHIP behaviour where needed. The different parameters are summarized in the following list. Please go to the corresponding section for a detailed description.

Description	Section	Default value range	Default value recommendation
Initial TCP retransmission count	8.3		2
Initial TCP retransmission timeout	8.4		1s
Maximum TCP retransmission timeout	8.4		120s
MTU (Maximum Transmission Unit)	8	1500 bytes	
Maximum fragment length	9.2	512 bytes	
Connection Keepalive "ping" min interval	10.4	50s	
Connection Keepalive "pong" timeout	10.4	10s	
SKI length	12.2	20 bytes (40 digit hexadecimal string)	
PIN length	12.5	8-16 digit hexadecimal string	
Maximum "auto accept" time window	12.3.1.1	≤ 120s	60s

Description	Section	Default value range	Default value recommendation
"User trust level" necessary for general SHIP communication	12.3.2	≥ 8	
"User trust level" or "second factor trust level" necessary for SHIP commissioning	12.3.2	≥ 32	
CmiTimeout	13.4	10-30s	30s
Wait-For-Ready-Timer initial	13.4.4.1	60-240s	120s
Wait-For-Ready-Timer prolongation	13.4.4.1	60-240s	120s
PIN entry penalty after the 3rd invalid attempt	13.4.4.3	10-15s	15s
PIN entry penalty after the 6th invalid attempt	13.4.4.3	60-90s	90s
Certificate update transition stage	12.1.3.2	≥ 1 month	25% of certificate lifetime (but not less than 1 month)
Certificate update connection retry waiting period	12.1.3.2	≥ 5 minutes	
keyMaterialState timeout (waiting for keyMaterialStateResponse, retransmission)	12.1.3.2	≥ 30 s	30s
keyMaterialStateResponse: timely sending	12.1.3.2	≤ 20 s	

374 Table 1: SHIP Parameters Default Values

5 Registration

The registration of a SHIP node begins with a public key verification mode (as described in section 12.3.1).

Note: Details on how to find appropriate SHIP nodes with mDNS are explained in chapter 7.

Registration with secure verification:

After public key verification (user verification, manufacturer specific commissioning, SHIP commissioning or user input) a SHIP node SHALL search for SHIP nodes with trusted SKI values and connect to them. To search for SHIP nodes with trusted SKI of other ECC curves than secp256r1, a SHIP node SHALL also support TLS probing, as described in section 9.8. If the connection establishment with a SHIP node does not complete successfully (see section 5.1 for successful registration), the SHIP node SHALL cyclically retry to connect to the SHIP node with the corresponding SKI.

If the other local SHIP node aborts the SME "hello", as described in section 13.4.4.1, a SHIP node SHOULD wait until the register flag of the other SHIP node is set to "true" before starting a new connection attempt. If the register flag of the other SHIP node is set to "true", a SHIP node that already has the trusted SKI from the other SHIP node SHALL retry to connect to the other SHIP node again.

Registration with auto accept:

If the "auto accept" mode is active, as described in section 12.3.1.1, a SHIP node SHALL set its own register flag for the service discovery to true. Additionally, the SHIP node SHALL start a service discovery for other SHIP nodes that have set the register flag to true.

Note: If both SHIP nodes use the "auto accept" mode to connect to each other, this is called mutual "auto accept". In the case of mutual "auto accept", no side verifies any SKI and therefore a so-called "man-in-the-middle" attack cannot be excluded. In the "man-in-the-middle" case, a third device (the "man in the middle", a potentially harmful device) is able to secretly read and even manipulate the communication between two SHIP nodes. Therefore, it is STRONGLY RECOMMENDED to support at least one of the other verification modes to avoid mutual "auto accept" and potential "man-in-the-middle" attacks!

If a SHIP node discovers more than one other SHIP node with a "register" flag set to true, it SHALL pick one SHIP node by any means appropriate (e.g., it could interpret additional information contained in the service discovery).

If the "auto accept" mode is inactive, a SHIP node SHALL set its own "register" flag for the service discovery to false and SHALL stop searching for other SHIP nodes that also have set the "register" flag to "true".

Note: Only "auto accept" affects the register flag. The other verification modes ("user verify", "user input" and "commissioning") SHALL have no effect on the register flag.

412 The registration between a SHIP node and a web server based SHIP node SHOULD be established in
413 the following order:

- 414 1. Verify public key of the web server-based SHIP node, as described in section 12.3.
- 415 2. Retrieve IP address and port number from DNS (only if URL/ DNS host name is used; if IP
416 address is used, this step can be skipped)
- 417 3. Connect to IP address and port number
- 418 4. Check if the received public key matches with a trusted SKI.
- 419 5. SHIP message exchange (SME) Connection Mode Initialization
- 420 6. SHIP message exchange (SME) Connection Data Preparation

421 The registration with another local SHIP node SHOULD be established in the following order:

- 422 1. Verify public key of the communication partner as described in section 12.3. If "auto accept"
423 is active, set register flag in service discovery to "true"; otherwise, it must be set to "false".
- 424 2. If "auto accept" is used and the other SHIP node has set the register flag in service discovery
425 to true, it is strongly recommended to switch to another verification mode to prevent mutual
426 "auto accept". If "user verify", "user input" or "commissioning" is used, search for SHIP nodes
427 with corresponding SKI values in the service discovery.
- 428 3. Connect to IP address and port number retrieved via service discovery or accept incoming
429 connection.
- 430 4. Check if the received public key matches with a trusted SKI.
- 431 5. SHIP message exchange (SME) Connection Mode Initialization
- 432 6. SHIP message exchange (SME) Connection Data Preparation

433 With the SME "hello" message, a SHIP node can confirm the trustworthiness of the communication
434 partner as described in section 13.4.4.1. If a SHIP node trusts the communication partner, the SHIP
435 node SHALL store the credentials of the communication partner (SKI, trust level, etc. - see section
436 12.2.2).

437 Note: If a SHIP node only supports one simultaneous active connection, it MAY close the server port
438 during the registration phase in order to be able to establish a client connection. In this case, the
439 constrained SHIP node SHALL wait a time of X milliseconds before it closes the server port and tries
440 to establish a connection to another SHIP node that has the register flag in service discovery set to
441 "true". X SHOULD be a random value between 0-30000 milliseconds.

442 5.1 Successful Registration

443 When both sides have confirmed trustworthiness of each other with an SME "hello" message, the
444 registration is successfully completed. Every new connection between these two devices MUST now

be viewed as reconnection and not a registration until one of both SHIP nodes purposely aborts the SME "hello" handshake.

5.2 Registration details and recommendations (informative)

This specification permits two approaches for a SHIP node "A" to decide at which time a foreign SKI/public key needs to be trusted.

- **Pre-trust:** Trust before data exchange

This means the foreign SKI/public key of a SHIP node "B" is trusted by SHIP node "A" before any SME message exchange (see chapter 13) can take place. This means a TLS handshake between nodes "A" and "B" can only be completed successfully if SHIP node "A" already trusts the foreign SKI/public key of SHIP node "B".

- **Post-trust:** Trust within SME connection state "Hello"

Section 13.4.4 specifies a possibility to trust the foreign SKI/public key of a SHIP node "B" right after the completion of the TLS handshake, but before the so-called "SME connection data exchange" can take place. This means a SHIP node "A" may just temporarily accept the SKI/public key of SHIP node "B", present the SKI to SHIP node A's user for verification, and finally trust the foreign SKI if and only if the user accepts this SKI. Only the acceptance enables execution of further data exchange. A refusal by the user closes the connection. A post-trust approach requires careful implementation beyond the TLS handshake in order to not accidentally introduce security weaknesses.

The verification modes described in section 12.3.1 do not mandate whether pre- or post-trust is to be applied. Just the exchange of key material via SHIP commissioning tool (see section 12.6) can always be seen as a pre-trust approach.

Additional considerations on the pre-trust approach:

It can be expected that vendors prefer the pre-trust approach to harden their implementation for security reasons. However, since SHIP version 1.1 further cryptographic curves are supported as well as new messages to report new or update to renewed cryptographic key material (including changes of the so-called SKI of a public key of a certificate) used for SHIP connections. This means that devices with pre-trust approach need to consider certain aspects on re-discovery and reconnection carefully with regards to changes of key material, as otherwise unexpected connection problems may arise. Among others, section 6.1 will give some recommendations if a once established registration and trust with another SHIP node is to be kept even in case of key material changes.

Section 7.3.2 describes how key material for a secp256r1 curve of a SHIP node can be found. Section 9.8 describes how key material for curves other than secp256r1 can be found on a given IP address.

6 Reconnection

If two SHIP nodes have successfully established a connection before, both nodes can reconnect to each other at any time. It does not matter if the register flag is "true" or "false" during reconnection – the register flag is only important for the registration process when connecting to new SHIP nodes (see chapter 5).

If the public key still matches the previously (during registration) provided, verified and trusted public key, the SHIP node SHALL be accepted again without any delay.

A reconnection between a SHIP node and a web server-based SHIP node is established in the following order:

1. Retrieve IP address and port number from DNS (only if URL / DNS host name is used; if IP address is used, this step can be skipped)
2. Connect to IP address and port number
3. Check if the public key of the communication partner is still the same as during registration
4. SHIP message exchange (SME)

A reconnection with a local SHIP node is typically established in the following order:

1. Connect to IP address and port number retrieved via service discovery or accept incoming connection
2. Check if the public key of the communication partner is trusted and still the same as during registration
3. SHIP message exchange (SME)

In the reconnection scenario, both SHIP nodes should already trust each other, so no user interaction is necessary. With the SME "hello" message, a SHIP node can directly confirm the trustworthiness of the communication partner as described in section 13.4.4.1 and continue with SME protocol handshake, optional PIN verification, and data exchange as described in section 13.4.

6.1 Reconnection details in case of changed key material (informative)

This section summarizes some data to support reconnections even if key material changes.

- SHIP ID
 - o See section 7.4 for recommendations on the storage of a communication partner's so-called "SHIP ID". This recommendation holds for both SHIP nodes of a communication, regardless of who initiated a connection.
- Cryptographic key material
 - o See section 12.2.2 about storage of key material of a foreign SHIP node to enable the next reconnection.
Note: There can be more than one key material set per SHIP ID.
 - o See section 12.1.3: Providing changes of the own key material as well as getting changes of a foreign node's key material are required processes if a reconnection

515 with a foreign node has to be feasible even in case that one of the SHIP nodes
516 changes its key material. In particular adding a formerly unsupported curve as well as
517 dropping the support of a formerly supported curve needs to be considered with
518 care.

519

7 Discovery

A discovery mechanism is used to find available SHIP nodes and their services in the local network without knowing their multicast DNS host names or IP addresses. To allow discovery of other SHIP nodes in the local network, mDNS/DNS-SD SHALL be used.

DNS-SD records SHOULD have a TTL of 2 minutes.

mDNS/DNS-SD provides methods for local service discovery, resource discovery, and multicast DNS host name to IP address resolution. Detailed information on mDNS can be found in RFC 6762; information on DNS-SD can be found in reference RFC 6763.

A SHIP node that uses mDNS SHALL offer a service named "ship".

7.1 Service Instance

The SHIP node SHALL assign an <Instance> label of up to 63 bytes in UTF-8 format for each DNS SRV/TXT record pair that it advertises. In accordance with RFC 6763 and in order to avoid name conflicts, this label SHALL use user-friendly and meaningful names, for example the device type, brand and model. Using a hypothetical company "ExampleCompany", an example <Instance> of a product with a SHIP node could be "Dishwasher ExampleCompany EEB01M3EU".

Should a name conflict still occur, a node SHALL assign itself a new name until the conflicts are resolved. A conflict SHOULD be resolved by appending a decimal integer in parentheses to the <Instance> (for example, "Name (2)" for the first conflict, "Name (3)" for the second conflict, etc.).

7.2 Service Name

The service name used with DNS-SD SHALL be "ship".

The <Service> portion of a service instance name consists of the service name preceded by an underscore '_' and followed by a period '.' plus a second DNS label specified by SHIP as "_tcp".

Thus, a valid service instance name example would be:

"Dishwasher ExampleCompany EEB01M3EU._ship._tcp.local."

where "Dishwasher ExampleCompany EEB01M3EU" is the <Instance> portion (described in previous section), "ship" is the service name, "tcp" is the transport protocol, and "local" is the <Domain> portion.

7.3 Multicast DNS Name

A local SHIP node SHALL assign a unique multicast DNS host name of up to 63 bytes. In order to avoid name conflicts, names SHOULD use the unique ID as specified in the TXT record.

Thus, a complete multicast DNS host name example would be:

"EXAMPLEBRAND-EEB01M3EU-001122334455.local."

7.3.1 Default Records

DNS-SD defines several records by default. This information MUST NOT be included in other records.

554 The A record includes the IPv4 address and the AAAA record includes the IPv6 address of the node.
 555 The SRV record SHALL include the service name, multicast DNS host name and port.

556 Note: A SHIP node MAY freely choose its port for the SHIP TCP server, but MUST state it correctly in
 557 the SRV record.

558 7.3.2 TXT Record

559 This sub-section specifies the format of the TXT record to be used in conjunction with DNS-SD. A SHIP
 560 node SHALL use a single TXT record format. The TXT record SHALL NOT exceed 400 bytes in length.
 561 The following table contains additional service parameters that SHALL be included in the TXT record.

Key	Value	Example	Runtime Behavior	Required
txtvers	Version number	txtvers=1	Static	Mandatory
id	Identifier	id=EXAMPLEBRAND-EEB01M3EU-001122334455	Static	Mandatory
path	String with wss path	path=/ship/	Static	Mandatory
ski	40 digit hexadecimal string representing the 160 bit secp256r1 SKI value	ski=1234AAAAFFFF1111CCCC3333EEEEDDDD99992222	Dynamic (*3)	Mandatory
register	Boolean	register=false	Dynamic	Mandatory
ecc	Boolean	ecc=true	Dynamic (*1)	Mandatory for SHIP version ≥ 1.1 (*2)

562 Table 2: Mandatory Parameters in the TXT Record

563 (*1): The value is "usually static". It would only change if a SHIP node suddenly changes the support
 564 of additional ECC curves (see below) accordingly.

565 (*2): See detailed description of `ecc` below.

566 (*3): The value is "usually static" unless the certificate of the secp256r1 curve is updated.

567 The TXT record can include other optional key-value pairs as long as the TXT record does not exceed
 568 400 bytes in length. The following optional keys are defined by this specification:

Key	Value	Example	Runtime Behaviour	Required
brand	String with brand	brand=ExampleBrand	Static	Optional
type	String with device type	type=Dishwasher	Static	Optional
model	String with model	model=EEB01M3EU	Static	Optional

Table 3: Optional Parameters in the TXT Record

`txtvers` SHALL be the first key in the TXT record. For this specification, the value of the `txtvers` key SHALL be 1. If it is found in a response to be other than 1, the TXT record SHALL be ignored. The `txtvers` key SHALL be present with a non-empty value. Clients SHALL silently discard TXT records with `txtvers` keys that are not present or have a different value than 1.

Unknown key-value pairs in a response SHALL be ignored.

The `id`, `ski`, `brand`, `type` and `model` values SHALL be in UTF-8 format.

The value of the `id` key contains a globally unique ID of the SHIP node and has a maximum length of 63 bytes. The first part of the unique ID SHOULD be an abbreviation of the manufacturer name. Behind the abbreviation, the manufacturer defines a unique identifier. The `id` value (SHIP ID) shall be unique. Note: The presence of two SHIP nodes with identical `id` values in a local network is not considered a regular setup within this specification as it may disrupt regular SHIP communications.

The maximum length of the `brand`, `type` and `model` values will be 32 byte of UTF-8 data each.

The maximum length of the `path` value will be 32 bytes of UTF-8 data. The minimum length is 1, where the `path` key contains the value `"/`.

The value of the `ski` key allows other SHIP nodes to directly identify a SHIP node by its SKI for a `secp256r1` curve. This is very helpful for other SHIP nodes that were provided with one or more trustworthy `secp256r1` SKI values from other SHIP nodes via "manufacturer specific commissioning", "user verification", "SHIP commissioning" or "user input". Otherwise, trial-and-error TLS handshakes with all nodes would be necessary to find the nodes with the fitting `secp256r1` public key / SKI. Also, SHIP nodes that support "user verify" do not need to gather `secp256r1` SKIs from local SHIP nodes over a TLS handshake, but can gather the `secp256r1` SKIs simply via service discovery.

A SHIP node that supports a SHIP version ≥ 1.1 SHALL encode its `secp256r1` SKI value in the `ski` field as uppercase hexadecimal string.

SKI example value: `0x1234AABBFFFF1111CCCC3333EEEEDDDD99992222`

SKI example encoding in the `ski` field: `"1234AABBFFFF1111CCCC3333EEEEDDDD99992222"`

However, a SHIP node that supports a SHIP version ≥ 1.1 SHALL also evaluate `ski` values properly that are encoded as lowercase or mixed case hexadecimal string.

The `register` key is used to indicate whether auto accept is active in the SHIP node.

The `ecc` key SHALL be set to "true" if optional ECC curves are supported beside `secp256r1`.

The `ecc` key SHALL be set to "false" if only `secp256r1` is supported as ECC curve.

As SHIP versions < 1.1 do not support the `ecc` key, SHIP nodes with versions ≥ 1.1 SHALL NOT expect that this key is always present.

7.4 Recommendations for re-discovery

The TXT record key "id" is also called the "SHIP ID". Further sources for a node's SHIP ID are SME "key material" messages (field "keyMaterial.entry.id", see section 13.4.7.5) and SME "Access methods" Messages (field "accessMethods.id", see section 13.4.6).

Once a foreign SHIP node "B" has been trusted it is recommended to store the SHIP ID persistently for a subsequent re-discovery.

Rationale:

Since SHIP version 1.1, SKI values of curves other than secp256r1 might be trusted and used for SHIP communication. These other SKI values cannot be discovered via the TXT record (just via TLS probing on a given IP address, see section 9.8). Additionally, key updates are more likely to occur, even for secp256r1 curves. This means that a re-discovery of a SHIP node based upon an SKI value is less efficient than a re-discovery using the constant "id" value.

8 TCP

TCP SHALL be used for communication. A communication over UDP, apart from mDNS for service discovery, is not specified in this SHIP version, but might be added later for multicast and group communication scenarios.

The MTU size SHALL NOT exceed 1500 bytes.

For a local server, the port SHALL be set according to the DNS-SD SRV record, as described in section 7.3.1.

8.1 Limited Connection Capabilities

A SHIP node MUST support a minimum of "1" simultaneously active connection.

If a local SHIP node is limited to "1" simultaneously active connection, it SHALL provide a listening TCP server and SHALL only close the TCP server port if it wants to establish a connection to another SHIP node as a TCP client. After using a TCP client connection, it SHALL close the TCP client connection as soon as possible and start listening on the TCP server port again.

If a SHIP node supports more than "1" simultaneously active connection, it SHALL always reserve one connection for the TCP server port. This means that when the SHIP node is limited to "x" simultaneously active connections, it SHALL only use a maximum of "x-1" connections for TCP client connections.

If a SHIP node supports more than "1" simultaneously active connection, it SHALL always reserve one connection for TCP client connections. This means that when the SHIP node is limited to "x" simultaneously active connection, it SHALL only use a maximum of "x-1" connections for TCP server connections.

In general, a SHIP node MAY close the TCP server port when it has reached its connection limit. In this case, the SHIP node SHALL reopen the TCP server port as soon as possible. If a SHIP node has not reached its connection limit, it SHALL always have an open TCP server port.

8.2 Online Detection

Before a local SHIP node can try to establish a connection over TCP to another local SHIP node, the other SHIP node SHOULD be detected as "online".

If the TTL of the mDNS service announcement of a local SHIP node is not valid, this SHIP node SHOULD be interpreted as "offline". If the mDNS service announcement of the corresponding local SHIP node is updated and the TTL is valid, the SHIP node SHALL be interpreted as "online" again.

In addition, a local SHIP node MAY send ICMP echo requests (Pings) to another local SHIP node to check whether the other side is "online" or "offline".

Note: In certain environments or devices, ICMP echo requests/replies MAY be blocked. If a local SHIP node is unable to receive an ICMP echo reply, but mDNS service announcements are received from the other local SHIP node, the SHIP node SHALL consider the other SHIP node as subject to ICMP blocking. In this case, the local SHIP node SHALL NOT use the ICMP echo requests as an indicator for

651 the "offline" state and SHOULD only rely on the TTL of the mDNS service announcement as "offline"
652 indicator.

653 Note: The "offline" detection is especially important for local SHIP nodes with limited connection
654 capabilities. Trying to reach a SHIP node B that is "offline" can cost SHIP node A 10 seconds of
655 connection time, see 8.3. This means that other SHIP nodes may not be able to reach this SHIP node
656 A for about 10 seconds while it is trying to establish a connection with SHIP node B that is offline.

657 **8.3 TCP Connection Establishment**

658 As described in RFC 793, an initial SYN packet is sent from the client to the server to initiate a TCP
659 connection. When a server accepts the incoming connection, it responds with an acknowledgment of
660 the SYN packet (SYN ACK). When a SHIP server receives a SYN packet for a closed port, it SHALL
661 respond with a reset (RST) packet as described in section 3.4 / page 36 of RFC 793. Furthermore, the
662 RST packet SHOULD NOT be blocked or filtered out, e.g. by a firewall, on the SHIP node.

663 The usage of the RST packet allows SHIP clients to very quickly detect whether the server port of the
664 other SHIP node is closed. In that case, the connecting SHIP node can immediately abort the
665 connection attempt. This also reduces the usage time of TCP connections, which can be of high
666 importance for constrained devices, as TCP connections may be a limited resource, as described in
667 section 8.1.

668 As the SYN packet as well as the RST packet may get lost, the initial SYN packet SHOULD be
669 retransmitted twice if no response (e.g. an ACK or RST) is received. If the recommended timeouts
670 from section 8.4 are used, this results in a maximum connection establishment duration of:

671 $1 + 3 + 6 = 10$ seconds

672 **8.4 Retransmission Timeout**

673 A SHIP node SHOULD maintain a retransmission timer as defined by RFC 6298. For that, round-trip
674 times (RTT) of transmitted packets are measured. The RTT means the time from sending a packet to
675 receiving the corresponding acknowledge (ACK). Retransmitted packets MUST NOT be used for
676 measuring, because in that case an ACK cannot be uniquely assigned to a sent packet anymore.

677 From the measured values, a retransmission timeout (RTO) is calculated, which is used for
678 subsequent transmissions of packets. RFC 6298 recommends a minimum RTO value of 1 second.
679 Nonetheless, it also points out that this is a conservative value and it will most likely make sense to
680 reduce it in the future. Thus, a SHIP node MAY reduce the minimum RTO value.

681 For the initial SYN packet, the value of the RTO SHOULD be initialized with 1 second as also
682 recommended by RFC 6298. RFC 6298 appendix explains why this has been reduced from the
683 historical value of 3 seconds. A SHIP node MAY choose to increase this value for lossy networks.

684 After the first retransmission of a packet, RFC 6298 demands to set the RTO to a minimum of the
685 historical value of 3 seconds. Then, an exponential back-off is applied, which doubles the RTO with
686 every retransmission. A SHIP node SHOULD apply a maximum RTO value of 120 seconds.

9 TLS

TLS 1.2 is MANDATORY. Apart from the maximum fragment length, see 9.2, TLS MUST be used as specified in RFC 5246.

Note (informative): The use of TLS 1.3 might be subject of future versions of the SHIP specification, but is not considered further in this version.

SHIP nodes MUST use mutual authentication during the TLS 1.2 Handshake, hence the public key / certificate MUST be verified on the client and server side, as described in section 12.3.

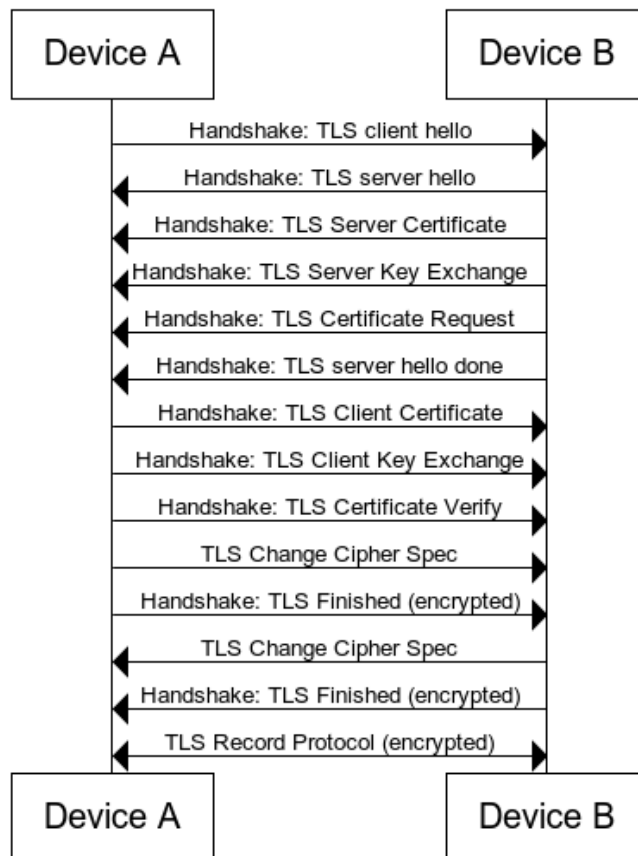


Figure 3: Full TLS 1.2 Handshake with mutual authentication

9.1 Cipher Suites

The TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256 cipher suite, as specified in RFC 5289, MUST be supported.

OPTIONAL cipher suites are:

- TLS_ECDHE_ECDSA_WITH_AES_128_CCM_8
- TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256

Hence, ECDSA is used for authentication and ECDHE for key exchange with perfect forward secrecy. The algorithms are based on ECC because the computational costs are lower than for e.g. RSA with similar security.

705 Only the following curves are supported in SHIP:

- 706 1. brainpoolP384r1 MAY be supported
- 707 2. brainpoolP256r1 MAY be supported
- 708 3. secp256r1 SHALL be supported

709 **9.2 Maximum Fragment Length**

710 Maximum Fragment Length Negotiation Extension, as specified in RFC 6066, SHOULD be supported.
711 If used, Maximum Fragment Length Negotiation Extension SHALL only support a length of 1024
712 bytes. This keeps the required buffer size for embedded devices low.

713 Some TLS implementations currently do not support Maximum Fragment Length Negotiation
714 Extension. Therefore, a SHIP node SHALL ensure that the fragment length (TLSPlaintext.length) of
715 outgoing packets does not exceed 1024 bytes, even if Fragment Length Negotiation Extension is not
716 supported.

717 **9.3 TLS Compression**

718 TLS Compression MUST NOT be used.

719 **9.4 Server Name Indication**

720 As specified by WebSocket in RFC 6455, the server name indication (SNI) extension MUST be
721 supported. However, local SHIP nodes SHALL ignore the information provided by the SNI extension.
722 Web server-based SHIP nodes MAY evaluate the SNI extension if they have a fixed DNS host name.

723 For local connections, the server name SHALL be equal to the mDNS host name of the local server.
724 For web server connections, the server name SHALL be the DNS hostname of the webserver.

725 Note: As described in RFC6066, the server name for SNI is represented as a byte string using ASCII
726 encoding without a trailing dot. This means that even if the server name in mDNS might have a
727 trailing dot, this trailing dot should not be used for SNI. However, some web browsers seem to use
728 the trailing dot for SNI in the client hello due to an incorrect implementation. Therefore, a SHIP
729 server implementation SHOULD ignore the last trailing dot if it is mistakenly inserted by the client.

730 **9.5 Renegotiation**

731 As the usage of TLS Renegotiation is not defined within SHIP, a SHIP node SHALL NOT support TLS
732 renegotiation and refuse TLS renegotiation requests in general.

733 **9.6 Session Resumption**

734 A full TLS 1.2 handshake introduces large computational costs and additional round trips. From a user
735 perspective, these computational costs can lead to delays in reaction time > 1 second for constrained
736 devices. To allow fast reconnections over TLS without the need for a full TLS handshake, session
737 resumption SHOULD be supported. This means that the session state holding the master secret and a
738 session id SHOULD be stored and reused during reconnections.

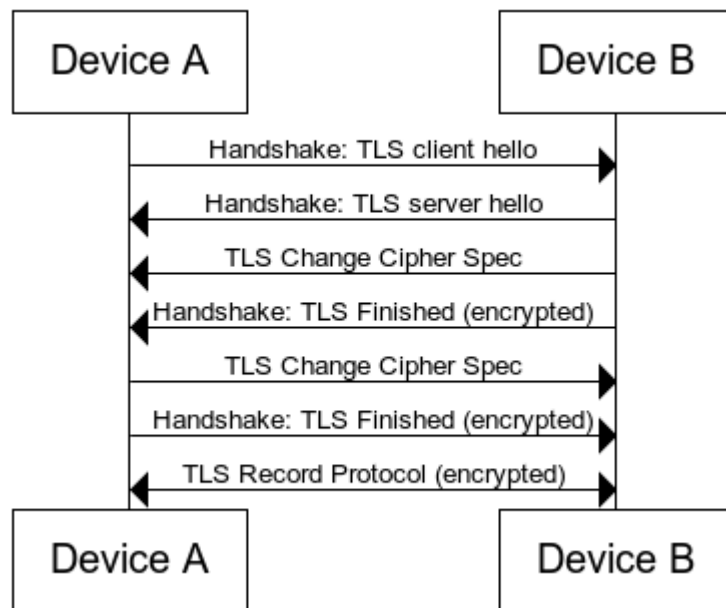


Figure 4: Quick TLS Handshake with Session Resumption

A SHIP node MAY discard a session state, e.g. if the connection has low requirements regarding the latency and reaction time, if the connection was not used for a certain amount of time, or if there is no more space for storage left and a new connection is established that is rated with a higher priority.

Note that discarding the session state always forces a full TLS handshake when the connection is re-established.

9.7 TLS extension for ECC

ECC TLS extensions SHALL be supported as described in RFC 8422. According to RFC 8422 ECC curves are ordered according to the client's preferences (favorite choice first).

Not ranking multiple ECC curves in a defined fashion would cause multiple issues:

- Based on whether node A is the client or the server in the connection, different resulting ECC curves might be used in a connection between a node A and B
- A user might need to verify multiple SKIs for the same SHIP node, which will be cumbersome
- If the ranking of multiple ECC curves might change at any point in time, the used certificates vary over time and might have to be re-verified
- To find out which SKIs are currently relevant between node A and B, A needs to connect to B and B needs to connect to A. This introduces further authentication issues.

Therefore, it is important that servers and clients apply the same curve rankings. The curve ranking does not imply a judgment about the level of security of different curves, but is a technical necessity as explained above. Only secp256r1 is mandatory.

For interoperability and the reasons given above, SHIP nodes SHALL rank ECC Curves as follows:

- brainpoolP384r1 (optional)

- 763 2. brainpoolP256r1 (optional)
764 3. secp256r1 (mandatory)

765 Note: The defined ECC curve ranking is not only used in the TLS extension for ECC, but shall be
766 applied throughout the specification. Among others, during public key verification mode, it must be
767 evaluated which curve would be used in TLS connections with a selected peer according to the
768 priority rules. Then, the peer's SKI of that curve will be presented for verification.

769 **9.8 TLS Probing**

770 In the mDNS TXT record, only the secp256r1 SKI of a SHIP node can be discovered.

771 If SHIP nodes connecting to each other both support additional ECC curves to secp256r1, only the SKI
772 of the curve chosen by the TLS ECC extension is relevant to establish a TLS connection.

773 SHIP nodes that support additional ECC curves to secp256r1 SHALL support TLS probing to discover
774 which curve and SKI is used by the other SHIP node in the TLS connection. This is an important step
775 to find the correct SKI, e.g. for user verification.

776 For TLS probing a SHIP node SHALL establish a connection to another SHIP node to get the key
777 material contained in the TLS Server Certificate message and to derive the SKI that is used by the
778 other SHIP node in the TLS connection.

779 In case of user verification this SKI SHALL be provided to the user. The ranking defined in section 9.7
780 ensures that the same SKI will be used independent of whether a SHIP node has the client or server
781 role in a TLS connection.

782 After the TLS Server Certificate message was received (as shown in Figure 3), a SHIP node acting as
783 TLS client SHALL stop the TLS probing connection, if the derived SKI is not trusted yet, for the
784 following reason: The other SHIP node might currently be in "auto accept" mode (see section 12.3.1
785 on public key verification modes). Without the aforementioned abortion of the TLS probing
786 connection, the other SHIP node would trust the TLS client's SKI used in the current TLS (probing)
787 connection. But TLS probing is not a public key verification mode and must also not interfere with
788 public key verification modes.

789 Note: A SHIP node may well be in auto accept mode while executing TLS probing. However, these
790 processes SHALL be treated independent from each other. This means a TLS server certificate
791 received in the context of a TLS probing step is no immediate criteria to complete the auto accept
792 mode.

793 Note: Certain SHIP nodes may only support one or a constrained number of simultaneous
794 connections. In this case, TLS probing may not be possible at certain times and a SHIP node SHALL
795 retry TLS probing later. SHIP nodes that only support one or a constrained number of simultaneous
796 connections SHOULD NOT interrupt an established connection for SHIP probing.

10 WebSocket

On top of TCP and TLS, WebSocket MUST be used. One of the goals achieved by using WebSocket is the ability of the protocol to maintain connections through local network gateways such as Network Address Translation (NAT) devices or firewalls.

Note that a number of "draft versions" of the WebSocket standard exist that are incompatible with the current standard. Therefore, this specification requires strict compliance with RFC 6455.

10.1 TLS Dependencies

A SHIP node client MUST use the Server Name Indication extension in the TLS handshake (RFC 6066). This is especially important for large-scale service providers such as cloud installations to be able to provide services for various server names. Please see section 9.4 for more details.

10.2 Opening Handshake

This section refers to sections 1.3 (non-normative) and 4 in RFC 6455.

Valid WebSocket Request-URIs for use with SHIP MUST follow the wss scheme (i.e., a valid SHIP URI will always start with "wss://") if TLS is used. This specification does not make any assumptions on the host, port and resource name properties of the request. A SHIP node will learn these properties via the SHIP discovery process as described in chapter 7. If a SHIP node decides to connect to another SHIP node, it SHALL present these properties in the exact same fashion as previously discovered.

The origin property MAY be absent in the request.

Each WebSocket request conforming to this specification MUST include the Sec-WebSocket-Version header with a fixed value of 13. Earlier WebSocket draft standard versions are not allowed.

Additionally, the value "ship" MUST be included in the Sec-WebSocket-Protocol header.

With this procedure, a SHIP node SHALL detect whether it is talking to another SHIP node at the earliest stage possible, preventing the overhead of useless communication with another node that implement the WebSocket protocol, but without the SHIP payload. The request MUST NOT contain any other subprotocol names.

The current version of this document does not specify any WebSocket extensions. Therefore, the request MUST NOT contain a Sec-WebSocket-Extensions header.

10.3 Data Framing

This section refers to section 5 in RFC 6455.

SHIP protocol messages are at least partially binary. Therefore, all data frames (i.e. non-control frames) used with this specification MUST be of type 0x2 (binary frames). A SHIP node that receives a data frame with another type (0x1) MUST terminate the connection with status code 1003 (unacceptable data). A SHIP node that receives a data frame with type (0x3–0x7, 0xB–0xF) MUST terminate the connection with status code 1002 (protocol error).

Since this specification does not allow any extensions, the reserved bits of the base framing protocol MUST be set accordingly (to value 0) and the reserved opcodes in the framing header MUST NOT be used.

For clarity, please note that while RFC 6455 requires clients to support fragmentation of messages and to support handling control frames in the middle of a fragmented message, it explicitly forbids interleaving of fragments belonging to different messages. The absence of SHIP protocol message interleaving ("multiplexing") is not considered a relevant issue in this SHIP version since SHIP protocol messages are expected to be relatively small (i.e., their transmission on a typical IP layer will only take a few milliseconds).

10.4 Connection Keepalive

A SHIP WebSocket connection SHALL be left established whenever local resource usage on the SHIP node permits this behaviour to reduce delay and reaction times of SHIP nodes.

In wide-area networking scenarios (e.g. for Cloud services), connections can typically only be established from a local SHIP node towards a remote one and not vice versa (i.e., only from the local device towards the Cloud, not vice versa because of a local firewall or NAT gateway). In this case, keeping up the connection is vital to be able to receive messages from the Cloud at any given time.

Furthermore, large-scale deployments might need to deploy fail-safe algorithms to detect server failures and re-route traffic to other nodes. A server failure may be detected quickly when using keep-alive connections, and re-routing will then usually occur before the connection is really needed for the next payload message, improving overall protocol resilience and user experience.

In addition to keeping connections alive whenever possible, a SHIP node SHALL make use of the ping and pong control frames to ensure that the underlying transport is really operational.

A SHIP node MUST NOT send ping messages at intervals smaller than 50 seconds on a single connection. The typical timeout waiting period for a pong message after sending a ping message SHALL be 10 seconds.

11 Message Representation Using JSON Text Format

11.1 Introduction

Many SHIP messages are sent using a JSON-UTF8 representation as a basis. However, the SHIP protocol is prepared to allow other formats, such as JSON-UTF16 or ASN.1.

For different reasons, which are beyond the scope of this document, some parts of the messages are defined using XSD (XML Schema Definition). This language permits the description of XML structures and content for specific purposes. Several tools can be found that permit creation of XMLs or even so-called "data binders" from XSDs.

JSON, or Java Script Object Notation, like XML, is an interchange format to describe data objects. It has been described in RFC 4627 since 2006. Because of its small set of formatting rules, it is easy to implement. Code for parsing and generating JSON is available in most programming languages.

In order to benefit from the advantages of XSD and JSON, this chapter describes which JSON types must be used and how to generate JSON text representations from the XSD. In general, it is rather difficult to map every feature of an XML to a corresponding JSON representation. However, there are some ways to retain the semantics of most XSD elements.

11.2 Definitions

JSON consists of six basic types.

1. Number
2. String (double-quoted)
3. Boolean
4. Array (ordered sequence)
5. Object (unordered collection)
6. Null

The data representation consists of key:value pairs and is built on two structures: An unordered collection surrounded by left and right curly brackets or an ordered sequence surrounded by square brackets. The members inside the structures are separated by commas.

11.3 Examples For Each Type

1. Number
{"age" : 12, "height" : 1.73}
2. String (double-quoted)
{"name" : "JSON in WebSocket"}
3. Boolean
{"checked" : true}

- 889 4. Array (ordered sequence)
 890 {"item" : ["one", "two", "three"]}
- 891 5. Object (unordered collection)
 892 {"Name" : "Crockford", "First name" : "Douglas"}
- 893 6. Null
 894 {"nillable" : null}

895 11.4 XML to JSON Transformation

896 11.4.1 Scope

897 The transformation rules in this chapter apply to all data structures in section 13.4 that are explicitly
 898 defined using XSD. These structures are called "explicit SHIP structures".

899 The SHIP Message Exchange permits conveying payload of an external (i.e. non-SHIP) specification
 900 within the element "data.payload" (see section 13.4.5.2). The corresponding specification is
 901 announced using the element "protocolId". An external specification MAY apply the specific
 902 conversion rules of this chapter as well. For each protocolId, it is permitted to define deviating rules
 903 for the content of "payload".

904 11.4.2 XSD Types

905 The mapping of XSD types is described in Table 4.

XSD types	JSON types
xs:boolean	Boolean
xs:double, xs:byte, xs:unsignedByte, xs:short, xs:unsignedShort, xs:integer, xs:unsignedInt, xs:nonNegativeInteger, xs:unsignedLong	Number
xs:dateTime, xs:duration, xs:time	String
xs:language, xs:string	String
xs:hexBinary	String
xs:anyType	Results in corresponding types
xs:simpleType	See next section 11.4.4.
xs:complexType	See next section 11.4.5.

906 *Table 4: Mapping from the XSD Types to JSON Types.*

907 11.4.3 Element Occurrences

908 Elements with a specified type can contain the attributes "minOccurs" and "maxOccurs". These
 909 attributes specify how often the field can or, respectively, must be added. E.g. "minOccurs=0" means
 910 the field is optional and may be omitted. If the "minOccurs" attribute is omitted for an element, it is
 911 implicitly set to 1, which means the field is mandatory.

If "maxOccurs" is set to a value greater than 1, the element is transformed to a JSON array, which contains items of the corresponding type. In that case, "maxOccurs" defines the maximum length of the array, where "unbounded" means there is no upper limit. If the "maxOccurs" attribute is omitted, it is implicitly set to 1, which means no JSON array is generated for the element, but of course the corresponding JSON type.

If "minOccurs" and "maxOccurs" are both set to 0, the element is ignored.

11.4.4 Simple Types

Simple types are specified with the <xs:simpleType> item and always refer to simple types like <xs:integer> or <xs:string>. Simple types can specify restrictions on the value or can define a list or a union of one or more simple types.

XSD item in xs:simpleType	JSON types
xs:restriction	Type corresponding to the base type. Restricts the possible values.
xs:list	Array
xs:union	String

Table 5: Transformation of a simple type.

Restrictions contain XSD items like <xs:minLength>, <xs:maxLength>, <xs:enumeration>, etc. These items limit the possible values of the type and apply to JSON and XML in the same way.

11.4.5 Complex Types

Complex types consist of a combination of sub-elements ("particles" in the XML specification), which can be arranged in different ways. These combinations are called compositors, which are: sequence, choice, and all. Some of them can also be nested. Depending on their usage, these compositors result in different JSON representations:

Used in: Compositor:	xs:complexType	xs:sequence	xs:choice	xs:all
xs:sequence	Array	+	+	Not allowed
xs:choice	Array	+	+	Not allowed
xs:all	Object	Not allowed	Not allowed	+
None	Array	-	-	-

Table 6: Mapping from the XSD compositors to JSON Types.

"+" means that the sub-elements are only integrated in the superset without creating a new hierarchy level. "Not allowed" means that XML Schema 1.0 prohibits this combination. "-" means that the item is omitted.

Furthermore, complex types can be derived from other simple or complex types with further extensions or restrictions. For that, the complex type consists of a <xs:simpleContent> or

<xs:complexContent> item with a nested <xs:restriction> or <xs:extension>. These items themselves are not transformed into JSON components.

In a complex content, restrictions delimit the base type to a set of sub-elements and/or delimit the possible values of elements. Extensions add elements to a sub-type. With that, the JSON structure MUST be transformed from the derived version of the type. This means that the JSON values MUST follow the restrictions and extensions in the same way as for XML. Elements added by an extension MUST also be transformed to corresponding JSON items.

In a simple content, restrictions delimit the possible values of the element like for simple types. Extensions can only add attributes to simple types, so they are omitted from the JSON transformation.

11.4.6 Rules

Generating a JSON representation based upon an XSD is defined as follows. In addition, different coding styles have to be considered, e.g., in XML, closing angle brackets are protected.

1. Element names become usual names, which are part of objects.
2. Any numbers are recognized and used as a JSON number.
3. Booleans are recognized and used as JSON booleans.
4. Empty elements get an empty JSON array as value.
5. "nil" elements get a JSON null value.
6. Elements which may occur in the same place more than once become a JSON array.
7. Attributes get absorbed.
8. Groups are integrated in the used places without creating additional representations. When the "maxOccurs" attribute of the group is greater than 1, it can be integrated several times.
9. The following rules apply on the use of namespaces and namespace prefixes of "explicit SHIP structures" (see section 11.4.1). These rules have an impact on the use of element names:
 - a. SHIP namespace definitions of XMLs are not transformed into a JSON representation.
 - b. SHIP namespace prefixes (including the colon, e.g. "ship:") of XMLs are omitted for the transformation into JSON. I.e. element names of explicit SHIP structures do not contain a SHIP namespace prefix in a JSON representation.
10. The following rules apply to the use of namespaces and namespace prefixes of "external specifications" used within the element "data.payload" (see section 11.4.1). These rules have an impact on the use of element names. Whether namespace definitions or namespace prefixes of external specifications are transformed into JSON can be specified per protocolId (see section 11.4.1):
 - a. By default, it is assumed that no such transformation is required.

- b. If a namespace definition is to be transformed into JSON, it is RECOMMENDED to transform it to a JSON object as follows:

```
{"@xmlns:namespacePrefix" : "schemaReference"}
```

where "namespacePrefix" is a dedicated namespace prefix and "schemaReference" contains the reference from the XML.

- c. If a namespace prefix is to be transformed into JSON, it is RECOMMENDED to transform it to a JSON object as follows: The element names of the XML shall be used including the namespace prefix and the colon, if present.

Example: If an XML contains a tag "xyz:foo", where "xyz" is the prefix, the proper JSON element name would also be "xyz:foo".

11.4.7 Example Transformations

The following table shows and compares examples for the corresponding XML and JSON representations of the XSD elements:

XSD element	XML representation	JSON representation
<code><xs:element name="height" type="xs:double"/></code>	<code><height>1.73</height></code>	<code>{ "height":1.73 }</code>
<code><xs:element name="checked" type="xs:boolean"/></code>	<code><checked>true</checked></code>	<code>{ "checked":true }</code>
<code><xs:element name="empty"> <xs:complexType> </xs:complexType> </xs:element></code>	<code><empty></empty></code> or <code><empty/></code>	<code>{ "empty":[] }</code>
<code><xs:element name="nillable" nillable="true"/></code>	<code><nillable xsi:nil="true"/></code>	<code>{ "nillable":null }</code>
<code><xs:element name="items"> <xs:complexType> <xs:sequence> <xs:element maxOccurs="unbounded" name="item" type="xs:unsignedInt"/> </xs:sequence> </xs:complexType> </xs:element></code>	<code><items> <item>1</item> <item>2</item> <item>3</item> </items></code>	<code>{ "items": [{ "item": [1, 2, 3] }] }</code>

XSD element	XML representation	JSON representation
<pre><xs:element name="items"> <xs:complexType> <xs:sequence> <xs:element name="item1" type="xs:unsignedInt"/> <xs:element name="item2" type="xs:unsignedInt"/> <xs:element name="item3" type="xs:unsignedInt"/> </xs:sequence> </xs:complexType> </xs:element></pre>	<pre><items> <item1>1</item1> <item2>2</item2> <item3>3</item3> </items></pre>	<pre>{ "items": [{"item1": 1}, {"item2": 2}, {"item3": 3}] }</pre>
<pre><xs:element name="items"> <xs:complexType> <xs:choice> <xs:element name="item1" type="xs:unsignedInt"/> <xs:element name="item2" type="xs:unsignedInt"/> <xs:element name="item3" type="xs:unsignedInt"/> </xs:choice> </xs:complexType> </xs:element></pre>	<pre><items> <item1>1</item1> </items> or <items> <item2>1</item2> </items> ...</pre>	<pre>{ "items": [{"item1": 1},] } or { "items": [{"item2": 1 },] }</pre>
<pre><xs:element name="element"> <xs:complexType> <xs:sequence> <xs:element name="item" type="xs:unsignedInt"/> </xs:sequence> <xs:attribute name="min" type="xs:unsignedInt"/> </xs:complexType> </xs:element></pre>	<pre><element min="3"> <item>5</item> </element></pre>	<pre>{ "element": [{"item": 5 }] }</pre>
<pre><xs:element name="items"> <xs:simpleType> <xs:list itemType="xs:unsignedInt"/> </xs:simpleType> </xs:element></pre>	<pre><items>1 2 3</items></pre>	<pre>{ "items": [1, 2, 3] }</pre>

XSD element	XML representation	JSON representation
<pre><xs:element name="items"> <xs:complexType> <xs:all> <xs:element name="item1" type="xs:unsignedInt"/> <xs:element name="item2" type="xs:unsignedInt"/> <xs:element name="item3" type="xs:unsignedInt"/> </xs:all> </xs:complexType> </xs:element></pre>	<pre><items> <item3>3</item3> <item2>2</item2> <item1>1</item1> </items> ...</pre>	<pre>{ "items": { "item3": 3, "item2": 2, "item1": 1 } } ...</pre>

983 Table 7: Examples for XML and JSON representations.

984 The following example shows the transformation of an XSD that combines several item types:

XSD
<pre><xs:complexType name="ComplexDataType"> <xs:sequence> <xs:element name="itemDouble" type="xs:double" minOccurs="0"> </xs:element> <xs:element name="itemString" type="xs:string" minOccurs="0"/> </xs:sequence> </xs:complexType> <xs:element name="complexData" type="ComplexDataType"></xs:element> <xs:complexType name="ComplexListDataType"> <xs:sequence> <xs:element maxOccurs="unbounded" minOccurs="0" ref="complexData"/> </xs:sequence> </xs:complexType> <xs:element name="complexListData" type="ComplexListDataType"/> <xs:group name="TestGroup"> <xs:sequence> <xs:element name="itemEmpty"> <xs:complexType></xs:complexType> </xs:element> <xs:element name="itemFixed" type="xs:string" fixed="predefined value"></xs:element> <xs:element name="optionalItem" type="xs:boolean" minOccurs="0"/> </xs:sequence> </xs:group> <xs:element name="example"> <xs:complexType> <xs:sequence> <xs:element ref="complexListData"/> <xs:group ref="TestGroup"/> </xs:sequence> </xs:complexType> </xs:element></pre>

XML representation
<pre><example> <complexListData> <complexData> <itemDouble>1.6</itemDouble> <itemString>abc</itemString> </complexData> <complexData> <itemDouble>2.4</itemDouble> <itemString>def</itemString> </complexData> </complexListData> <itemEmpty/> <itemFixed>predefined value</itemFixed> </example></pre>
JSON representation
<pre>{ "example": [{ "complexListData": [{ "complexData": [[{ "itemDouble": 1.6 }, { "itemString": "abc" }], [{ "itemDouble": 2.4 }, { "itemString": "def" }]] }] }, { "itemEmpty": [] }, { "itemFixed": "predefined value" }] }</pre>

985 Table 8: Example transformation of several combined XSD item types.

11.5 JSON to XML Transformation

11.5.1 Scope

Converting JSON to a corresponding XML representation can be ambiguous as the expressiveness of the two formats differs. E.g. JSON allows to model empty arrays while XML does not. The transformation rules in section 11.5 aim to reduce these ambiguities.

11.5.2 Rules

Generating an XML representation based upon a JSON is defined by reversing the rules defined in section 11.4. Additionally, the following rules apply:

1. Empty JSON arrays where the corresponding XSD element may occur more than once MUST be ignored.

11.5.3 Example Transformation

The following table shows examples for particular transformations from JSON to XML for given XSD elements.

Table 9 compares two JSON representations "a" and "b" leading to the same XML representation according to the rules in section 11.5.2 for a given XSD permitting multiple occurrences of an element. Representation "a" is the regular representation as it follows the transformation principles from section 11.4 for an XML with absent list items: As the XML contains no list item at all, the same level of information should also be present in the JSON representation. However, as JSON naturally permits representations like "b", it needs to be considered equivalent to "a".

XSD element	JSON representation	XML representation
<pre><xs:element name="items"> <xs:complexType> <xs:sequence> <xs:element maxOccurs="unbounded" name="item" type="xs:unsignedInt"/> </xs:sequence> </xs:complexType> </xs:element></pre>	<pre>a) { "items": [] } b) { "items": [{ "item": [] }] }</pre>	<pre><items> </items></pre>

Table 9: Example for JSON to XML transformation.

12 Key Management

The essential credentials of the key management in this specification consist of the following key material:

1. **Public keys:** Used by SHIP nodes in the first step of authentication to validate the authenticity of signatures and perform an ECDH key agreement. Also, the corresponding SKI values of the public keys are used for a user verification, identification and to trust the public keys itself.
2. **Private keys:** Each SHIP node has a private key corresponding to its public key, which is used to generate signatures and perform ECDH key agreements.
Note: While a public key has no requirements regarding confidentiality and can be transmitted in band, the private key must be kept secret and should never be transmitted. In general, a secure element should be used to protect the stored private key.
3. **Symmetric keys:** Used for mutual authentication and symmetric encryption during efficient reconnection and runtime communication.
4. **PIN:** Optionally used in the second step of authentication (independent from TLS) to improve the trust level. E.g. if auto accept (which only offers a relatively low trust level) is used by a SHIP node in the first authentication step, a PIN can be used to reach mutual authentication and e.g. enable SHIP commissioning.

12.1 Certificates

12.1.1 SHIP Node Certificates

A SHIP node MUST have a certificate. No matter if the node acts as SHIP client or SHIP server, a SHIP node MUST always provide a certificate during the TLS handshake for mutual authentication.

A self-signed or PKI based certificate MUST be used.

SHIP node certificates MUST include a SHIP node specific public key.

One public/private key pair SHALL NOT be used for more than one certificate.

One SHIP node certificate SHALL NOT be used for more than one SHIP node.

This SHIP specification does not specify a mandatory PKI.

If a SHIP node does not know the PKI of another SHIP node, the corresponding PKI based certificate is just treated like a self-signed certificate (excluding the signature verification). Hence, interoperability within SHIP does not depend on using a certain PKI.

A SHIP node MAY also receive and manage a revocation list from a web server. If a SHIP node has a synchronized time, it MAY also check whether a certificate is still valid. A SHIP node without synchronized time SHOULD NOT set a validity for self-signed certificates. A self-signed certificate without validity SHOULD set notAfter to 99991231235959Z, see TLS, as described in RFC5280. Other means of certificate evaluation MAY also be used by a SHIP node. However, the additional evaluation of a certificate is only a manufacturer specific topic in this SHIP version.

1044 In general, a SHIP node MUST at least verify the public key of the certificate with the SKI trusted by
1045 one of the public key verification modes (see section 12.2 for details). Any other evaluation of the
1046 certificate is optional or manufacturer specific and SHALL NOT affect the general SHIP authentication
1047 and communication (see Note 1 below). This includes certificate lifetime checks, PKI checks, and
1048 other checks of the certificate. This means if optional or manufacturer specific checks fail, but the
1049 received public key is authenticated correctly, the SHIP node SHOULD still allow communication with
1050 the other device (see Note 1 below). An invalid PKI certificate SHOULD be handled like a self-signed
1051 certificate (excluding the signature verification), as trust in SHIP relies on the SHIP node specific
1052 public key and not a PKI. E.g. a PKI certificate with an invalid lifetime SHOULD just be handled like a
1053 self-signed certificate (no "PKI trust" is given, but if the public key is trusted, this certificate MAY still
1054 offer "user trust"). This also applies if a SHIP node does not support a synchronized time to check the
1055 lifetime of a PKI certificate. If an optional check fails, a SHIP node MAY inform the user about the
1056 reasons for a failed optional check.

1057 Note 1: The SHIP specification defines a baseline interoperability for general SHIP authentication and
1058 communication. In this baseline, certificate validation is limited to verifying the public key and
1059 Subject Key Identifier (SKI). Additional checks, such as verifying the certificate's expiry date, could
1060 violate the baseline interoperability. Manufacturers are advised to strictly adhere to the baseline
1061 requirements. However, specific regulatory or proprietary requirements may demand additional
1062 validations, including expiry date checks. Such checks are outside the scope of the SHIP specification.
1063 While these additional requirements cannot be prohibited by the SHIP specification, they could affect
1064 interoperability with devices following the baseline interoperability.

1065 Note 2: If a SHIP node certificate has a lifetime, the manufacturer SHOULD also implement update
1066 mechanisms for the certificate.

1067 The certificate of a SHIP node MAY be updated, which includes the update of the corresponding
1068 public key, SKI, and private key (see also section 12.1.3 on certificate updates). However, if the
1069 updated SKI cannot be provided to the user, the original public key, SKI, private key and certificate
1070 SHALL be restored with factory reset (e.g. if the SKI of the public key is only available printed on a
1071 device label).

1072 The CN (common name) field is out of scope for certificates within this SHIP specification. A SHIP
1073 node SHOULD ignore the CN (common name field) field of received certificates.

1074 **12.1.2 Web Server Based SHIP Node Certificates**

1075 A web server-based SHIP node has a special role, as a web server is usually not a local member of the
1076 private network and in such cases cannot act as a client. A web server based SHIP node SHOULD have
1077 a PKI based certificate.

1078 Therefore, SHIP nodes that want to communicate with a web server-based SHIP node SHOULD have a
1079 corresponding CA-certificate for the verification of the web server's certificate.

1080 Note: A CA-certificate has no requirements regarding confidentiality. However, a SHIP node MUST
1081 assure that the CA-certificate storage cannot be manipulated.

12.1.3 Certificate updates

This section and its sub-sections describe how a SHIP node "A" can report changes of its own certificates (i.e. not changes of other SHIP nodes) to another SHIP node "B".

Note: The mechanisms to report certificate changes to another SHIP node are introduced with SHIP version 1.1.0. They are not supported by SHIP nodes that are based upon a previous SHIP version. As regulation authorities increasingly require shortening the lifetime of certificates, it is expected that SHIP versions since 1.1.0 will gain more support as they provide mechanisms to automatically update certificates without breaking the trust between SHIP nodes and without a need of manual user interactions. However, vendors should do their best effort to support users if the SHIP node "A" trusts another SHIP node "C" that is based upon a version before 1.1.0. In order to avoid that a certificate update of "A" unexpectedly breaks communication with "C" (i.e. re-trusting would be required), the vendor of "A" may instead consider refraining from the certificate update, provided that this is permitted and acceptable. But if the certificate update of "A" cannot be avoided, the vendor of "A" should implement measures to guide the user through the necessary steps to establish trust again between "A" and "C" (in general all connected SHIP nodes of a version before 1.1.0). Please note that the SHIP version of a remote SHIP node is available from the SME "Protocol Handshake" Message (see section 13.4.4.2).

Note: These sections are ONLY about SHIP node certificates (see section 12.1.1). They are NOT about certificates for other protocols or services. They are also NOT about root or intermediate certificates if a SHIP node certificate is signed by another certificate. They do also NOT describe measures to change a SHIP node's own certificates.

12.1.3.1 Overview

A SHIP node's certificates MAY be updated over the course of time. The specification differentiates between 4 different certificates:

- **initial certificate:** The initial certificate of a SHIP node, i.e. the certificate that the device was originally delivered with.
- **predecessor certificate:** The previous certificate of a SHIP node, i.e. a certificate that was in use, but is not in use any more.
- **current certificate:** The certificate that is currently used in the TLS handshake to secure the communication.
- **successor certificate:** The new certificate that needs to be provided to other SHIP nodes before it can replace the current certificate.

For each supported curve in TLS, there SHALL be an according current certificate, e.g. in case secp256r1 and brainpoolP256r1 are supported a SHIP node needs to have a current secp256r1 certificate and current brainpoolP256r1 certificate. If a current certificate is updated, it SHALL be updated with a successor certificate that uses the same curve as the current certificate.

Note: This specification does not require to terminate an active connection just because the SHIP node replaces the certificate with a new certificate. Switching to a new certificate is rather relevant for new connections.

1121 A SHIP node that updates its key material SHALL transmit the updated key material state to trusted
1122 peers. As an additional mechanism, the connection state "hello" contains information about a SHIP
1123 node's key material state, see section 13.4.4.1.

1124 SHIP nodes SHALL offer a transition stage for certificate updates of at least 1 month to reduce the
1125 probability of failed updates and the update frequency. The transition stage SHALL start by
1126 transmitting SHIP SME "keyMaterialState" messages to trusted SHIP nodes. After the transition stage
1127 the new certificate(s) SHALL replace the current certificate(s). Additionally, if the secp256r1
1128 certificate changes, the according SKI in the TXT record SHALL be updated, see section 7.3.2.

1129 A SHIP node MAY add new curves or drop old curves during runtime. This is because additional key
1130 material of other supported cryptographic curves is conveyed based on the initially trusted key
1131 material. However, if a SHIP node intends to replace one curve by another curve, the transition stage
1132 is necessary to give connected SHIP nodes the possibility to switch from the current certificate of the
1133 old curve to the new certificate of the new curve. This requires the new certificate of the new curve
1134 to be announced as "successor". E.g. if brainpoolP256r1 is switched to brainpoolP384r1 a transition
1135 stage is necessary to prevent a fallback to secp256r1.

1136 Note that removing a certificate or support for a cryptographic curve completely, e.g. in a case in
1137 which an according cryptographic curve is no longer supported by the SHIP node, is considered a
1138 certificate update, too. In that situation, "keyMaterialState" messages will not contain certificates for
1139 the according cryptographic curve any more. Note that in this situation, an already existing trust
1140 between two nodes can only persist if both nodes have also trusted each other for another common
1141 cryptographic curve before.

1142 **12.1.3.2 Transmitting certificate updates**

1143 If a SHIP node has a successor certificate, the SHIP node SHALL transmit the SKI of the successor
1144 certificates to trusted peers with the SHIP SME "keyMaterialState" message. The SHIP SME
1145 "keyMaterialState" message SHALL also contain the SKI of the current certificates and MAY contain
1146 the SKI of the predecessor certificates. The SHIP SME "keyMaterialState" message SHALL NOT
1147 contain foreign SKIs. This approach requires an already existing, trusted relationship between the
1148 peers, that has formerly been established via any public key verification mode or a commissioning
1149 tool.

1150 Note: As already described in section 12.1.3, the SHIP SME "keyMaterialState" message is only used
1151 to provide updated key material used in SHIP itself.

1152 In a factory new state, the current certificate is the initial certificate and there is no predecessor
1153 certificate. Once a certificate update process starts, the SHIP node SHALL, for a transition time of one
1154 month, mark the new key material as "successor" of the key material marked as "current". After the
1155 transition time of one month, the "current" certificate SHALL become the "predecessor" certificate
1156 and the "successor" certificate SHALL become the "current" certificate and be denoted as such.

1157 All the available certificates, predecessor, current and successor, SHALL be transmitted in exactly one
1158 "keyMaterialState" message. The "keyMaterialState" message has a field denoting the
1159 "updateCounter", which is representative of the current overall state of the key material of a SHIP

1160 node. Section 12.1.3.3 describes that a peer must respond to a "keyMaterialState" message with a
1161 SHIP SME "keyMaterialStateResponse" message.

1162 If a SHIP node has a currently active connection to a peer it wants to transmit the updated key
1163 material to, it SHALL utilize this already active connection to transfer the "keyMaterialState"
1164 message. If no SHIP SME "keyMaterialStateResponse" is received 30 seconds after a SHIP SME
1165 "keyMaterialState" message was transmitted, the SHIP SME "keyMaterialState" message SHOULD be
1166 resent. If no "keyMaterialStateResponse" message is received after 30 seconds again, the current
1167 connection SHALL be closed and a new connection SHOULD be established for subsequent retries.
1168 The re-connection SHALL follow the same rules as denoted below for currently closed connections to
1169 peers.

1170 If a SHIP node has no currently active connection to a peer it wants to transmit the updated key
1171 material to, it SHALL try to open a connection to provide the updated key material. Due to different
1172 circumstances, e.g. the other peer being non-reachable, opening a connection may fail. A SHIP node
1173 SHALL NOT attempt to open connections to its peers for the sole sake of updating the key material
1174 more frequently than every 5 minutes. If the connection succeeds, the rules laid out in the above
1175 paragraph apply.

1176 In any already established connection, the keyMaterialState message might be sent or received
1177 multiple times, e.g. if a node's key material changes while a connection is still open.

1178 **12.1.3.3 Receiving certificate updates**

1179 Foreign successor and current SKIs of a received keyMaterialState message SHALL be stored for the
1180 curve negotiated for the current TLS connection to the sender of the keyMaterialState message.
1181 Foreign SKIs of a received keyMaterialState message SHALL be stored together with the trust level of
1182 the sender of the keyMaterialState message.

1183 Foreign SKIs of other curves than the one negotiated for the current TLS connection with the sender
1184 SHOULD NOT be trusted (see Note below). This minimizes the potential attack surface, particularly in
1185 cases in which any weakness of a curve other than the currently used one is discovered. If foreign
1186 SKIs of other curves than the one negotiated for the current TLS connection with the sender are
1187 stored and trusted, they SHALL be stored together with the trust level of the keyMaterialState
1188 message of the sender.

1189 Note: The sender of such a keyMaterialState message should be aware that the recipient will
1190 probably NOT trust a different curve this way automatically. This means that this should not be
1191 considered a common approach to add trust in another curve or even to prepare switching
1192 completely from one curve to another curve (at least not without user interaction). The recipient
1193 may trust another curve anyhow, but this is not guaranteed and would need to be verified
1194 separately. Subsequent versions of the SHIP specification may add or change recommendations to
1195 trust other curves automatically.

1196 Foreign predecessor SKIs of a received keyMaterialState message for supported curves MAY be
1197 deleted, but SHALL be untrusted.

1198 After receiving a "keyMaterialState" message from another SHIP node, a
1199 "keyMaterialStateResponse" message SHALL be sent to the other SHIP node within 20 seconds.
1200 Transmitting a "keyMaterialStateResponse" message just expresses the receipt of a
1201 "keyMaterialState" message. This message does not express whether the SKIs of the
1202 "keyMaterialState" message have been accepted or not.

1203 A SHIP node SHALL NOT trust foreign key material of a SHIP node that is not contained in the
1204 received "keyMaterialState" anymore.

1205 The "keyMaterialState" message contains an "updateCounter", which is representative of the overall
1206 key material state of a SHIP node. If a SHIP node A transmits its "keyMaterialState" to SHIP node B,
1207 SHIP node B can evaluate the "updateCounter" and compare it with a previous "updateCounter" it
1208 received from A. If the received "updateCounter" is greater than the previous "updateCounter" of A,
1209 the key material of the SHIP node A changed. The "updateCounter" is additionally transmitted in the
1210 connection state "hello", thus making it possible to detect changed key material on a new
1211 connection.

1212 **12.1.3.4 Recognizing certificate updates**

1213 A SHIP node MAY, due to different circumstances (see below), deem the locally stored key material
1214 of a foreign node as possibly outdated. Whenever that is the case, a SHIP node MAY issue a
1215 "keyMaterialStateRequest" message within the SME "connection data exchange" state to the
1216 currently connected, foreign node.

1217 A notable and precise reason for deeming the locally stored certificates of a foreign node as possibly
1218 outdated is observing the foreign node sent a higher "keyMaterialState.updateCounter" within the
1219 "hello" connection state.

1220 A SHIP node that supports at least SHIP version 1.1 reports in the SME connection state "hello" an
1221 element "keyMaterialState.updateCounter" with its current value. The
1222 "keyMaterialState.updateCounter" is representative of the overall state of the key material of a
1223 node, also refer to section 13.4.7.9. It is thus transferred in every connection establishment of a SHIP
1224 node.

1225 If the received "keyMaterialState.updateCounter" of a SHIP node in the "hello" state is higher than
1226 the locally known and stored "keyMaterialState.updateCounter" of that trusted SHIP node, the
1227 according key material has been updated and must be retrieved. In such a case, a SHIP node SHALL
1228 issue a "keyMaterialStateRequest" message to the according foreign node within the following SME
1229 "connection data exchange" state. A sent "keyMaterialStateRequest" message SHALL contain the
1230 currently stored and trusted "updateCounter" of the foreign node.

1231 If a received "updateCounter" inside a "keyMaterialStateRequest" message is different from the
1232 according current value of the receiving node, the receiving node SHALL send a "keyMaterialState"
1233 message as laid out in section 12.1.3.2. This accounts for possible racing conditions in which a SHIP
1234 node sends an unsolicited "keyMaterialState" message, while the remote node sends a
1235 "keyMaterialStateRequest" message.

1236 Note that it is not intended to downgrade certificates. Thus, a decrease in the "updateCounter" is
1237 generally not seen as a valid and trustworthy action and a "keyMaterialState" message with
1238 decreased counter SHALL be ignored.

1239 **12.1.3.5 Effects for verification modes**

1240 A certificate update might partly invalidate the content of a label or QR code on the device as the
1241 printed SKI might become obsolete through the update. The obsoleted printed SKI(s) can then not be
1242 used anymore for verification modes that require an SKI. This is in particular problematic if the SHIP
1243 node is to be registered at another SHIP node "B" which only supports user verification with pre-trust
1244 approach (there may be further problematic constellations which will not be detailed here).

1245 As a consequence, it is STRONGLY RECOMMENDED that the vendor of a SHIP node enables a user to
1246 easily get updated SKIs. The following (non-exhaustive) list provides some examples to achieve this:

- 1247 • Web page (locally at the SHIP node or vendor's homepage) to display the current SKI(s)
- 1248 • An application to display the current SKI(s) (using a SHIP node's local or cloud based
1249 interface)
- 1250 • Display on the device

1251 **12.2 SHIP Node Specific Public Key**

1252 A SHIP node MUST have a SHIP node specific public key per certificate. The public keys have no
1253 requirements regarding confidentiality and can be transmitted in-band.

1254 The Subject Key Identifier SHALL be generated as described in RFC 3280, section 4.2.1.2 method (1)
1255 (i.e. the SKI is the SHA-1 value of the public key).

1256 SHIP nodes SHALL check if a received public key matches with a trusted SKI*¹.

1257 *¹ Note: This means an implementation that receives a certificate must extract the public key from
1258 this certificate and calculate the SHA1-1 value from the public key. The result must be identical to an
1259 already trusted SKI (and of course the SKI of the certificate itself, see section 12.2.1). A mismatch
1260 means that the certificate is not valid and hence the communication must be aborted as the
1261 certificate was probably created by an adversary.

1262 **12.2.1 Public Key Presentation**

1263 The Subject Key Identifier (SKI) of the SHIP node specific public key MUST be provided to the user.

1264 SKI values SHALL be made accessible to the user in full length.

1265 The SKI provided for user verification SHALL match with the SKI of the current certificate used in
1266 TLS*¹.

1267 *¹ Note: This means an implementation must verify if the certificate extension "subject key
1268 identifier" (SubjectKeyIdentifier) of a received certificate contains an SKI value that is identical to the
1269 SKI trusted by the user. Matching values are a necessary condition to continue the communication,
1270 but this is not sufficient to verify the authenticity of the certificate: See esp. section 12.2 on the
1271 verification of a received public key.

1272 The 20 byte-long SKI of the public key SHALL be provided to the user as 40 hexadecimal digits in the
 1273 following form (see also section 12.1.3.5 for updated SKI values). To increase the readability of the
 1274 SKI and provide interoperability from a user perspective, spaces SHALL be inserted each 4
 1275 hexadecimal digits, as shown below:

1276 XXXX XXXX XXXX XXXX XXXX XXXX XXXX XXXX XXXX

1277 Depending on the ECC curve the following prefix SHALL be added followed by a separating space in
 1278 front of the 40 hexadecimal digits:

- 1279 - "SKI" for secp256r1 SKIs
- 1280 - "BSKI" for brainpoolP256r1 SKIs
- 1281 - "B2SKI" for brainpoolP384r1 SKIs

1282 Note: In SHIP versions before 1.1 there was no prefix for secp256r1 SKIs.

1283 Examples of a secp256r1 SKI user representation on a display or device label:

1284 secp256r1 SKI value: 0x1234AAAAFFFF1111CCCC3333EEEEDDDD99992222

1285 ⇒ secp256r1 displayed SKI string: SKI 1234 AAAA FFFF 1111 CCCC 3333 EEEE DDDD 9999 2222

1286 Examples of a brainpoolP256r1 SKI user representation on a display or device label:

1287 brainpoolP256r1 SKI value: 0x1234AAAAFFFF1111CCCC3333EEEEDDDD99992222

1288 ⇒ brainpoolP256r1 displayed SKI string: BSKI 1234 AAAA FFFF 1111 CCCC 3333 EEEE DDDD
 1289 9999 2222

1290 Remark: The SHIP node SKI MAY also be integrated into a QR code together with the PIN, as
 1291 described in 12.7.

1292 At least one of the following measures MUST be applied to make the SKI of the public key accessible
 1293 for the user:

- 1294 - Label: Access to the SKI of the SHIP node specific public key is provided via a label on the
 1295 SHIP node.
- 1296 - User interface: Access to the SKI of the SHIP node specific public key is provided via a user
 1297 interface (e.g. display) of the SHIP node.
- 1298 - Local communication interface: Access to the SKI of the SHIP node specific public key is
 1299 provided via a local user communication interface of the SHIP node, e.g. NFC. The user MUST
 1300 be able to easily access the public key of the SHIP node. Therefore, the local communication
 1301 must provide easy and user friendly access to the SKI.
- 1302 - Cloud based user interface: Access to the SKI of the SHIP node specific public key is provided
 1303 via a user interface in the cloud. The public key must be accessed via the serial number, or
 1304 some other SHIP node-specific distinctive characteristic.

1305 Note: If a SHIP node can only provide the SKI with a label, the SKI label needs to be updated if the
1306 certificate is updated. If this is not feasible a SHIP node with an SKI label should not update the initial
1307 certificate or allow to restore the initial certificate via factory default.

1308 **12.2.2 Public Key Storage**

1309 A SHIP node has its own certificate, public key and SKI, but also needs to be able to store the SKIs of
1310 foreign public keys for authentication of other SHIP nodes. A SHIP node can express with an SME "key
1311 material" in the element "keyMaterial.maxEntries" the maximum number of SKIs (and related data)
1312 that it can store in its memory (see section 13.4.7.5).

1313 Trusted foreign SKIs (verified or updated) of other SHIP nodes SHALL be stored together with their
1314 trust level.

1315 A SHIP node SHALL also store the received "updateCounter" if SKIs from a keyMaterialState message
1316 are stored.

1317 Note: For a given foreign SHIP node there might be several sets of key material stored (one per state
1318 and curve). The different key material for a given SHIP node MAY have different trust levels.

1319 To avoid re-verification by user interaction, persistent storage of mandatory key material is
1320 STRONGLY RECOMMENDED.

1321 Untrusted foreign SKIs MAY also be stored and SHALL issue a "user trust level" of "0".

1322 A SHIP node SHOULD offer the user a possibility to remove certain trusted foreign public keys at any
1323 time. At least the SHIP node SHALL offer a possibility to delete all stored foreign public keys (e.g. via
1324 factory reset).

1325 **12.2.2.1 Optional Storage**

1326 Beside the SKI and trust level there are further values that are not strictly necessary but can help in
1327 certain situations. The following list shows which further values may be of interest for storage:

- 1328 - Public key: As public keys in SHIP have a key length of 256 Bit or more, storing foreign public
1329 keys for each SKI can be utilized to improve the uniqueness and security of the
1330 authentication compared to 160 Bit SKIs.
- 1331 - Curve: The curve of a foreign SKI's public key may also be stored. Especially for non
1332 secp256r1 SKIs this allows a better presentation to the user. Additionally, the curve allows to
1333 derive if the SKI can be found with mDNS or if TLS probing is needed. However, this can also
1334 be achieved by storing a non-secp256r1 flag.
- 1335 - Non-secp256r1 flag: If no curve information of an SKI is available, the non-secp256r1 flag
1336 allows to derive if the SKI can be found with mDNS or if TLS probing is needed.
- 1337 - SHIP ID: If a non-secp256r1 SKI is used between two SHIP nodes, storing which SKI belongs to
1338 which SHIP ID improves the re-discovery of a SHIP node that has changed its IP address. With
1339 the SHIP ID, re-discovery of a non-secp256r1 connected SHIP node is possible over mDNS.
1340 Without SHIP ID, TLS probing has to be used. Please refer to section 7.4 for more
1341 information. Additionally, storing which SKIs belong to which SHIP ID allows to identify if
1342 multiple SKIs are stored for the same SHIP node.

Note: The persistent storage of SHIP IDs might be required (mandated) by further specifications that are based upon or related to the SHIP specification. In particular this might be the case if the constant SHIP ID is used together with additional authentication information. It can also be expected that the persistent storage of SHIP IDs will be mandated in general by the SHIP specification itself in a subsequent version.

- Key material state: The state of each SKI may also be stored (successor, predecessor, current).
- Certificate: the certificate of a SHIP node itself may also be stored. This might be especially important if a PKI certificate is used.

12.2.3 Prevent Double Connections with SKI Comparison

The public key SHALL also be used to prevent double connections. If a SHIP node recognizes after a successful TLS handshake that there are two or more simultaneous connections to another SHIP node, the SHIP node with the bigger 160 bit SKI value SHALL only keep the most recent connection open and close all other connections to the same SHIP node (a previous release of this SHIP specification may permit a different preference). If an older connection is already in the SME "connection data exchange" state, the SHIP node with the bigger SKI value SHOULD initiate a connection termination as described in section 13.4.8. In general, each SHIP node may close a connection – even the SHIP node with the smaller SKI value – if a timeout was detected or the SHIP node with the bigger SKI value does not close double or multiple connections to the same SHIP node within 3 seconds. After a timeout of 3 seconds, the device with the smaller SKI value SHALL send a WebSocket ping. Connections that are not pingable (i.e. where no proper Pong frame is received as response) SHOULD be closed. If multiple connections are still pingable, the SHIP node with the smaller SKI value MAY close the older connection. If an older connection is already in the SME "connection data exchange" state, the SHIP node with the smaller SKI value SHOULD initiate a connection termination as described in section 13.4.8.

12.3 Verification Procedure

SHIP nodes that possess one or more CA-certificates MAY check whether a received certificate is a PKI or self-signed certificate.

A communication partner with a matching PKI certificate MAY gain additional PKI trust, depending on the trustworthiness of the corresponding CA.

As described in section 12.1.1, an invalid PKI certificate should be handled like a self-signed certificate (excluding the signature verification), as trust in SHIP relies on the public keys trusted by a user.

A SHIP node MUST always verify the public key of the communication partner with one of the following verification modes (see also section 12.2).

12.3.1 Public Key Verification Modes

Each public key verification mode provides a certain user trust level, the "user trust". While the verification mode describes the concrete mode, the user trust level maps the different modes on a generic value. In each of the public key verification modes, user interaction is necessary to establish user trust between two SHIP nodes to ensure user consent.

1383 When a stored public key is reused, it MUST be possible to derive the trust level of the public key.
1384 The following public key verification modes exist:

- 1385 1. Auto accept: "User trust" level = 8
- 1386 2. Auto accept + PIN: "Second factor trust" level 16-32 AND "User trust" 8
- 1387 3. User verify: "User trust" level = 32
- 1388 4. Manufacturer specific commissioning: "User trust" level = 32-96 (depending on the
1389 trustworthiness of the "commissioning tool")
- 1390 5. User input: "User trust" level = 64
- 1391 6. SHIP commissioning
- 1392 a) "Second factor trust" level 32 AND "User trust" 8
- 1393 b) "User trust" level 32

1394 Note: A PKI trust is no verification mode. For PKI trust level please see section 12.3.2.

1395 Note: The trust provided by SHIP commissioning depends on the verification that was used to verify
1396 the SHIP commissioning tool at a SHIP node. The "SHIP commissioning" trust level reflects the trust
1397 level that was assigned to the public key of a SHIP commissioning tool. As specified in 12.3.1.6 and
1398 12.3.2, for verification of a SHIP commissioning tool either a) auto accept + PIN or b) user verification
1399 is used.

1400 The user trust level of a public key can be adjusted during runtime whenever a public key is re-
1401 verified in a more secure manner. If for example an "auto accepted" public key is later re-verified
1402 successfully as a "commissioned" public key, the trust level SHALL be adjusted to the "commissioned"
1403 trust level.

1404 A SHIP node MUST implement at least one of the verification modes. Two or more verification modes
1405 MAY be active in parallel during runtime.

1406 **12.3.1.1 Auto Accept**

1407 This mode should only be implemented by very simple SHIP nodes without any user interface (UI), as
1408 it is the least secure verification mode. This mode SHALL NOT be implemented by SHIP nodes that
1409 have a user interface that would allow other verification modes, e.g. user verify.

1410 If "auto accept" is triggered in a SHIP node (e.g. by push button) by the user, the SHIP node SHALL
1411 open a time window in which it MUST automatically accept an unknown foreign public key that is
1412 received during registration.

1413 Note: If multiple curves are supported by both peers the highest ranked ECC curve and SKI is chosen
1414 between peers.

1415 Only one foreign public key SHALL be accepted during a single "auto accept" time window. Hence,
1416 after accepting one unknown foreign public key during "auto accept", the "auto accept" mode SHALL
1417 instantly be deactivated.

1418 As "auto accept" skips the public key verification of the received unknown public key (thus
1419 introducing the potential risk of man-in-the-middle attacks), the duration for the auto accept time
1420 window SHALL be kept as low as possible from a usability perspective and MUST be lower than or
1421 equal to 2 minutes. However, please keep in mind that man-in-the-middle attacks are still possible
1422 even with a shorter time window.

1423 **12.3.1.2 Auto Accept with PIN**

1424 To counter man in the middle attacks it is STRONGLY RECOMMENDED to use auto-accept only in
1425 combination with a PIN.

1426 Note: Beside auto accept + PIN another verification procedure needs to be supported, as other SHIP
1427 nodes might not be able to enter a PIN, as described below.

1428 If "SHIP commissioning" is also supported by the SHIP node (i.e. besides "auto accept") a SHIP node
1429 MAY set SME connectionPinState.pinState to "required". If "SHIP commissioning" is not supported,
1430 SME connectionPinState.pinState SHALL NOT be set to "required", as described in 13.4.4.3. This is
1431 necessary from an interoperability perspective, as most SHIP nodes might not be able to enter a PIN.
1432 If SHIP node "A" does not provide a possibility to enter a PIN, but SHIP node "B" has its SME
1433 connectionPinState.pinState set to "required", then a "SHIP commissioning tool" can be used for
1434 trust establishment between SHIP nodes "A" and "B", provided that SHIP node "B" supports "SHIP
1435 commissioning".

1436 If a SHIP node always requires a PIN after auto accept, the duration for the auto accept time window
1437 MAY also be longer than 2 minutes. If a SHIP node always requires a PIN after auto accept, the SHIP
1438 node SHALL only accept an unknown public key / SKI if the owner of the public key was also able to
1439 send the correct PIN. When a communication partner with unknown public key was able to send the
1440 correct PIN, the auto accept time window SHALL be closed. Unknown public keys / SKIs received from
1441 communication partners, that were not able to send the correct PIN, SHALL be discarded as
1442 untrustworthy public keys / SKIs. The latter (discarding untrustworthy public keys / SKIs) SHALL NOT
1443 have any effect to the auto accept time window (i.e. the auto accept time window is neither reduced
1444 nor extended).

1445 **12.3.1.3 User Verification**

1446 This mode SHOULD be implemented by any SHIP node with an appropriate display or communication
1447 interface for the user.

1448 If "user verification" and other curves than secp256r1 are supported, TLS probing SHALL be used to
1449 discover whether a SHIP node uses an SKI related to a different curve in TLS.

1450 If "user verification" is supported, the SHIP node MUST inform the user when unknown foreign SKI
1451 values of public keys are presented during local service discovery of other nodes via mDNS or that
1452 have been found during a TLS handshake (including TLS probing). The information MUST include the
1453 SKI value of the public key. In case SKIs for different curves are available from a peer, only the SKI

1454 matching with the highest ranked supported ECC curve at the local SHIP node SHALL be provided to
1455 the user.

1456 Example 1: SHIP node A supports secp256r1 and brainpoolP256r1. SHIP node B supports secp256r1
1457 and brainpoolP384r1. In this case the secp256r1 is the highest ranked curve between SHIP node A
1458 and B. This means if SHIP node B supports user verification, the SKI value related to the secp256r1
1459 curve needs to be provided to the user.

1460 Example 2: SHIP node A supports secp256r1 and brainpoolP256r1. SHIP node B support secp256r1,
1461 brainpoolP256r1 and brainpoolP384r1. In this case the brainpoolP256r1 is the highest ranked curve
1462 between SHIP node A and B. This means if SHIP node B supports user verification, the SKI value
1463 related to the brainpoolP256r1 curve needs to be provided to the user.

1464 "User verification" can also be applied to already stored public keys to increase the user trust level or
1465 discard public keys.

1466 **12.3.1.4 Manufacturer Specific Commissioning**

1467 While "SHIP commissioning" provides an interoperable commissioning of trusted foreign key material
1468 and allows to handle multiple SHIP nodes by the same commissioning tool, also a manufacturer
1469 specific commissioning can be described. In this case a manufacturer may have its own
1470 commissioning tool, that only works with the SHIP nodes of this manufacturer. Manufacturer specific
1471 commissioning is called "commissioning" within the SHIP specification.

1472 Note: The trust level of a public key that was verified via "commissioning" mode depends on the
1473 trustworthiness of the used commissioning tool.

1474 If SKI values of public keys are received from a commissioning tool, those SKI values SHALL be stored
1475 persistently together with the user trust level of the commissioning tool and used during verification
1476 to identify matching public keys. A SHIP node SHALL also check if there are already known SHIP nodes
1477 with a matching public key and adjust the trust level accordingly.

1478 A commissioning tool MAY also be used to provide access to a revocation list or to update key
1479 material of a SHIP node.

1480 Note: As "commissioning" provides user trust, the user SHALL be part of the commissioning
1481 procedure and user consent is required.

1482 **12.3.1.5 User Input**

1483 This mode does usually not need to be implemented by a SHIP node that does not act as a SHIP
1484 commissioning tool. However, this mode can particularly be useful for a SHIP commissioning tool, as
1485 explained in the SHIP commissioning tool section 12.6.

1486 **12.3.1.6 SHIP Commissioning**

1487 SHIP commissioning allows to exchange trustworthy key material (that is used for SHIP
1488 communication) over a secured SHIP connection between a trusted SHIP node that acts as "SHIP
1489 commissioning tool" and a SHIP node that supports the "SHIP commissioning" public key verification
1490 mode. The key material is exchanged via SME key material exchange, as described below (see section
1491 13.4.7 for message details). In this case the SHIP commissioning tool provides a unified user

1492 interface, for all SHIP nodes that support SHIP commissioning. Therefore, it is not necessary for SHIP
1493 nodes that support SHIP commissioning to provide an own user interface if auto accept + PIN is used.

1494 A SHIP node MAY support the "SHIP commissioning" public key verification mode.

1495 A SHIP node "A" that supports the "SHIP commissioning" public key verification mode SHALL support
1496 the following SME key material exchange messages:

- 1497 - Receive SME "commissioning request"
- 1498 - Send SME "commissioning response"
- 1499 - Providing key material that is foreign for node "A" to a SHIP commissioning tool:
 - 1500 o Receive SME "key material request"
 - 1501 o Send SME "key material"
- 1502 - Receiving key material that is foreign for node "A" from a SHIP commissioning tool:
 - 1503 o Send SME "key material request"
 - 1504 o Receive SME "key material"
 - 1505 o Send SME "key material response"
 - 1506 o Receive SME "key material delete"
 - 1507 o Send SME "key material delete response"

1508 However, a SHIP node that supports the "SHIP commissioning" public key verification mode SHALL
1509 only accept an SME "commissioning request" message from another SHIP node if at least one of the
1510 following trust level requirements is fulfilled:

- 1511 a) "User trust" level ≥ 32
- 1512 b) "Second factor trust" level ≥ 32 AND "User trust" ≥ 8

1513 To fulfil this trust level requirement, one of the other public key verification modes MUST be
1514 supported to establish initial trust to the "SHIP commissioning tool". Therefore, a SHIP node that
1515 supports the "SHIP commissioning" public key verification mode MUST additionally support one of
1516 the following verification modes:

- 1517 - User Verify: User trust = 32
- 1518 - Auto Accept + PIN after factory default: Second factor trust = 32

1519 Note: Auto accept without PIN cannot be used for authentication, because the user trust level = 8 is
1520 too low. However, when a PIN is used in combination with auto accept, the trust level requirement
1521 can be fulfilled by reaching a second factor trust level = 32.

- 1522 A SHIP node that supports "SHIP commissioning" MAY deactivate other public key verification
1523 modes, once a "SHIP commissioning tool" was successfully accepted until factory default is triggered.
- 1524 If an SME "commissioning request" message is received and the trust level requirements are fulfilled,
1525 a SHIP node that supports the "SHIP commissioning" public key verification mode, SHOULD accept
1526 the requester as "SHIP commissioning tool" and send an SME "commissioning response" message to
1527 the requester with commissioningResponse.accept = true. A commissioned SHIP node SHOULD store
1528 the acceptance as SHIP commissioning tool (e.g. in form of a flag). The acceptance as SHIP
1529 commissioning tool SHALL be stored at least for the duration of the TLS session.
- 1530 A SHIP node that supports the "SHIP commissioning" public key verification mode, SHALL add and
1531 update foreign SKIs / public keys according to an SME "key material" message received from an
1532 accepted and trustworthy "SHIP commissioning tool", with following exceptions:
- 1533 a) curve is not supported
 - 1534 b) curve is not used between nodes
 - 1535 c) memory constraints (see below)
- 1536 As explained in sections 12.2.2 and 13.4.7.5, a SHIP node that supports "SHIP commissioning" can
1537 express with its "keyMaterial.maxEntries" the maximum number of SKIs (and related data) that it can
1538 store in its memory. If a SHIP node "A" that supports "SHIP commissioning" receives in an SME "key
1539 material" message with more "entry" items in the received "keyMaterial" than stated by the value of
1540 "keyMaterial.maxEntries" of SHIP node "A", then SHIP node "A" MAY discard upon its own discretion
1541 received "entry" items that do not fit into its memory.
- 1542 The trust level of rejected SKIs SHALL be set to zero in the SME "key material response" message.
- 1543 Note: The SME "key material" message of a SHIP commissioning tool may sometimes just contain
1544 selected SKI/public keys. I.e. it may not contain the whole list of entries known by a SHIP
1545 commissioning tool (e.g. depending on user selection).
- 1546 A SHIP node that supports the "SHIP commissioning" public key verification mode, SHALL delete all
1547 foreign SKIs / public keys according to an SME "key material delete" message received from an
1548 accepted and trustworthy "SHIP commissioning tool".
- 1549 A SHIP node that supports the "SHIP commissioning" public key verification mode MAY request SKI
1550 updates by sending SME "key material request" to an accepted and trustworthy "SHIP commissioning
1551 tool".
- 1552 Note: A SHIP node that supports "SHIP commissioning" cannot distinguish whether a "key material"
1553 is based upon a "key material request" or whether it has been provided unsolicitedly.
- 1554 If a SHIP node that supports the "SHIP commissioning" public key verification mode receives an SME
1555 "key material request" message from an accepted and trustworthy "SHIP commissioning tool", it
1556 SHALL send an SME "key material" message to the requesting "SHIP commissioning tool", with all
1557 trusted (user trust > 0) foreign SKIs / public keys.
- 1558 **Trust limitation:**

1559 For the user trust level of SKIs received with a SME "key material" message, certain limitation rules
 1560 need to be applied. For the limitation the following trust levels SHALL be considered for each
 1561 received SKI "S":

- 1562 a) user trust level of the SHIP commissioning tool's SKI at the commissioned SHIP node
- 1563 b) second factor trust level of the SHIP commissioning tool's SKI at the commissioned SHIP node
- 1564 c) already existing user trust level of the SKI "S" at the commissioned SHIP node (or 0 if the SKI
 1565 was unknown to the commissioned SHIP node before)

1566 The highest value of a, b, c SHALL set the limit "x" for the (new) user trust level for the SKI "S"
 1567 received with an SME "key material" message from a SHIP commissioning tool. If the received trust
 1568 level "d)" of the received SKI "S" is higher than "x", then the trust level SHALL be truncated to the
 1569 limit "x". The resulting user trust level (truncated or not) is then expressed within the SME "key
 1570 material response" message.

1571 Note: The limitation rule above of the trust level is important because a user may add SKIs in a SHIP
 1572 commissioning tool with a trust level that is higher than the trust of the SHIP commissioning tool at
 1573 the commissioned SHIP node. For example: When a user adds an SKI in the SHIP commissioning tool
 1574 with user input, the user trust level of the SKI is 64. If this SKI is transmitted to a commissioned SHIP
 1575 node that used auto accept + PIN to trust the SHIP commissioning tool, the user trust level "a)" is
 1576 only 8 and the second factor trust level "b)" is 32. We assume the SKI is new to the commissioned
 1577 SHIP node (i.e. "c)" is 0). In this case, the received SKI's trust level "d)" of 64 needs to be limited to
 1578 the higher value of user / second factor trust level. This means the received user trust level "d)" of 64
 1579 needs to be limited to 32 in this example.

1580 12.3.2 Trust Level

1581 The trust level expresses the trust in a certain communication partner using generic values. The
 1582 higher the values, the stronger the trust in the corresponding communication partner.

1583 The trust level consists of different categories, which include different mechanisms and permit a
 1584 differentiated view of the trustworthiness of a communication partner. Currently, there are the
 1585 following trust level categories in SHIP:

- 1586 1. User trust
- 1587 2. PKI trust
- 1588 3. Second factor trust

1589 The trust levels are applied as described in the following table:

Mechanism	Trust level
untrusted	"User trust" level 0
auto accept	"User trust" level 8
auto accept + PIN	"User trust" level 32 and "Second factor trust" level 16 or 32 (see section 12.5: "32" reserved for first PIN transmitter)

Mechanism	Trust level
user verified	"User trust" level 32
commissioned	"User trust" level 32-96 (depending on trustworthiness of commissioning tool)
user input	"User trust" level 64
SHIP commissioning	a) "User trust" 8 AND "Second factor trust" level 32 b) "User trust" level 32
optional PKI	"PKI trust" level 0-65535 depending on SHIP node policy / trust in certain PKI

Table 10: Trust Levels

Note: optional PKI is no verification mode, as it does not offer the necessary user trust.

Note: The trust provided by SHIP commissioning depends on the verification that was used to verify the SHIP commissioning tool at a SHIP node. Verifications used in combination with a SHIP commissioning tool is a) auto accept + PIN or b) user verification.

If multiple mechanisms are used from the same category, only the mechanism which offers the highest trust level in this category SHALL be accounted for. E.g. if a SHIP node has verified a public key with "auto accept" and "user verify", only "user verify" is accounted for and therefore the "user trust" value is "32".

A "user trust" of "8" is the minimal "user trust" that is required for general SHIP communication. This means if the "user trust" is less than "8", the communication SHALL be aborted.

For commissioning via SHIP, a trust level of "32" or higher MUST be achieved in the "user trust level" or "second factor trust level" category. E.g. a "second factor trust level" of "32" would allow commissioning over SHIP, but also a "user trust level" of "32" would allow commissioning.

The PKI trust depends on the manufacturer's policy. PKI certificates are not mandatory, hence general communication SHALL also be possible without the use of a trusted PKI and a "PKI trust level" of "0".

The PIN is currently the only second factor authentication mechanism and MAY provide an additional trust level value of "16-32" in the "second factor trust" category, as described in section 12.5.

A layer above SHIP can use the trust level to control access to certain functionality. The trust level requirements MAY differ depending on the feature, the kind of application/use case, and legal or device related security requirements. Some privacy relevant use cases might require a high "user trust" while manufacturer specific use cases might have a high requirement regarding the "PKI trust level".

12.4 Symmetric Key

Depending on the chosen security method, a SHIP node SHOULD store the necessary key material in order to reconnect in an efficient manner.

If TLS was used, the session state of this connection SHOULD be stored and reused during a reconnection, as described in section 9.6.

The session state SHALL be stored in alignment with the public key and the trust level of the corresponding communication partner.

12.5 SHIP Node PIN

The SHIP node PIN provides a very user friendly way to reach mutual authentication, e.g. via QR code scan between a smart phone and a SHIP node that uses auto accept and additionally waits for a valid PIN. The SHIP node PIN transmission is described in section 13.4.4.3.

The SHIP node PIN is bound to the public keys/SKIs of the SHIP node. Therefore, the SHIP node PIN SHALL only be transmitted to a SHIP node that is the cryptographically proven owner of the corresponding public key. After the SHIP node PIN was transmitted, the sender SHOULD discard the PIN.

The PIN is an authentication secret that must be kept confidential and SHALL only be shared with authenticated and authorized communication partners. Therefore, the SHIP node PIN SHALL NOT be transmitted if the public key of the corresponding communication partner has a user trust level that is less than "32".

The first communication partner after factory default that sends the SHIP node PIN MAY gain a higher second factor trust level of "32" and therefore MAY gain access to special functionality. E.g. a communication partner that has a second factor trust of "32" MAY act as a "SHIP commissioning tool" towards the SHIP node. In SHIP, it is not possible that two SHIP nodes may gain a second factor trust of "32" with the SHIP node PIN. Any SHIP node that sends the PIN afterwards SHALL only get a second factor trust of "16". If another communication partner should be the one with a second factor trust of "32", a factory reset must be performed. After factory reset, the first communication partner that sends the SHIP node PIN MAY gain a higher second factor trust level of "32".

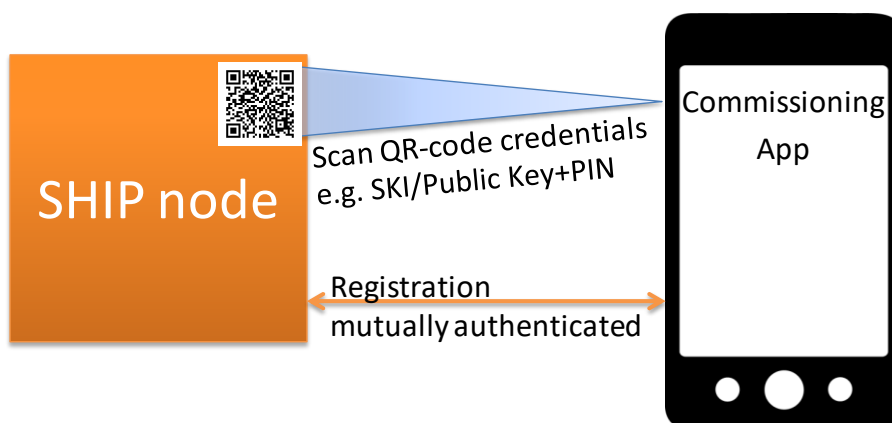


Figure 5: Easy Mutual Authentication with QR codes and Smart Phone

1643 The SHIP node PIN SHALL be provided to the user as 8-16 hexadecimal digits in the following form
1644 (equivalent to 32-64 Bit PIN). To increase the readability of the PIN and provide interoperability from
1645 a user perspective, spaces should be inserted between every 4 hexadecimal digits in a graphical
1646 presentation as shown below:

1647 XXXX XXXX (8 digits)

1648 XXXX XXXX X (9 digits)

1649 ...

1650 XXXX XXXX XXXX XXXX (16 digits)

1651 Additionally, the prefix "PIN" SHALL be used in a graphical representation in front of the PIN,
1652 separated with a space.

1653 Example of a 40-Bit PIN user representation on display or device label:

- 1654 • Graphical representation: PIN 5555 AAAA FF
- 1655 • Corresponding 40-Bit PIN value in hexadecimal format: 0x5555AAAAFF

1656 Remark: The SHIP node PIN MAY also be integrated into a QR code together with the SKI, as
1657 described in 12.7.

1658 This subsection only addresses the PIN in the scope of key management. The actual transmission
1659 format is described in subsection 13.4.4.3 of the SHIP data exchange chapter.

1660 The PIN of a SHIP node MAY be changed by the user, e.g. via a user interface. However, via a factory
1661 reset the original PIN MUST be restored again. This is especially important if the original PIN is
1662 printed on a device label.

1663 If a SHIP node PIN is changed during runtime, this SHALL NOT affect any trust level of already trusted
1664 SHIP nodes. If a user wants other SHIP nodes to reenter the PIN, the user MUST force a re-
1665 registration of the corresponding SHIP node by deleting the trust relationship to the corresponding
1666 SHIP node.

1667 **12.6 SHIP Commissioning Tool**

1668 Commissioning allows to exchange trustworthy key material over a secured SHIP connection,
1669 between a trusted SHIP node that acts as "SHIP commissioning tool" and a SHIP node that supports
1670 the "SHIP commissioning" public key verification mode. The key material is exchanged via SME key
1671 material exchange, as described in section 13.4.7.

1672 The following verification modes are relevant for a "SHIP commissioning tool":

- 1673 - User Input MAY be supported:
 - 1674 o If User Input is supported by a SHIP commissioning tool it SHALL be possible to scan
 - 1675 label and QR codes with SKIs and PINs

- 1676 - User Verification SHALL be supported
- 1677 o Present all SKIs from SHIP nodes found via mDNS to user over display
- 1678 o Allow the user to select trusted SKIs
- 1679 Note: For updated SKIs (i.e. invalid labels/QR codes) see section 12.1.3.5.
- 1680 A "SHIP commissioning tool" SHALL also support reverification of SKIs trusted from commissioned
1681 SHIP nodes:
- 1682 - Present trusted SKIs from commissioned SHIP nodes to user over display
- 1683 - If a user reverifies an SKI, the SHIP commissioning tool SHALL assign a user trust level
1684 according to the user verification.
- 1685 - Allow user to delete untrusted / deprecated SKIs
- 1686 Beside the mandatory SME messages, a SHIP node that can be used as "SHIP commissioning tool"
1687 SHALL support the following SME messages:
- 1688 - Send SME "PIN input" Message as described in section 13.4.4.3
- 1689 - Send SME "commissioning request"
- 1690 - Receive SME "commissioning response"
- 1691 - Read key material
- 1692 o Sending SME "key material request": request from a commissioned SHIP node "A" its
1693 trusted foreign key material
- 1694 o Receiving SME "key material": received trusted foreign key material needs to be
1695 provided to the user for reverification.
- 1696 - Providing foreign key material
- 1697 o Receiving SME "key material request": receive key material request from a
1698 commissioned SHIP node "A"
- 1699 o Sending SME "key material": provide trusted key material that is foreign for SHIP
1700 node "A" to the commissioned SHIP node "A"
- 1701 o Sending SME "key material delete": delete untrusted key material that is foreign for
1702 SHIP node "A" at a commissioned SHIP node "A"
- 1703 o Receiving SME "key material delete response": receive delete response for untrusted
1704 foreign key material from commissioned SHIP node "A"
- 1705 To gain commissioning rights a "SHIP commissioning tool" SHALL send an SME "commissioning
1706 request" to the corresponding SHIP node. With this message a SHIP node also communicates its

1707 capability to act as "SHIP commissioning tool". SME "commissioning request" SHOULD be sent on
1708 every connection to a SHIP node.

1709 If a "SHIP commissioning tool" receives a "commissioning response" with the
1710 "commissioningResponse.accept" value set to "true", "SHIP commissioning tool" may request, add or
1711 delete trustworthy key material in the SHIP node that has send the "commissioning response
1712 message".

1713 If a "SHIP commissioning tool" receives a "commissioning response" with the
1714 "commissioningResponse.accept" value set to "false", the requested SHIP node has rejected the
1715 "SHIP commissioning tool", e.g. because the trust level was too low.

1716 If a "SHIP commissioning tool" receives no "commissioning response" at all, the requested SHIP node
1717 might not support "SHIP commissioning".

1718 **Key material reverification:**

1719 If the user wants to reverify the key material of a commissioned SHIP node, a SHIP commissioning
1720 tool SHALL send a key material request to the corresponding commissioned SHIP node.

1721 If a SME "key material" message is received at a "SHIP commissioning tool", it SHALL present the
1722 trustworthy SKIs from the sender of this message to the user, so the user can re-verify the SKIs of the
1723 sender of this message. Based on this information a user may decide to delete certain keys or to add
1724 further keys at the sender of this message.

1725 **Add foreign SKIs:**

1726 Adding new key material SHALL only be triggered by user interaction with the SHIP node (e.g. user
1727 establishes trust to a SHIP commissioning tool) or with the SHIP commissioning tool (e.g. trustworthy
1728 key material is added, user establishes trust to a SHIP node).

1729 A SHIP node that supports "SHIP commissioning" MAY use the SME "key material request" message,
1730 to request key material from a SHIP commissioning tool. If a SME "key material request" message is
1731 received by a "SHIP commissioning tool", it SHALL reply with an SME "key material" message with
1732 foreign SKIs / public keys from trustworthy SHIP nodes to the requester. If the "SHIP commissioning
1733 tool" does not have any relevant, trustworthy SKIs to provide, the SME "key material" message
1734 SHALL have zero entries in the list.

1735 If a "SHIP commissioning tool" wants to add new trustworthy foreign SKIs / public keys to a SHIP
1736 node that supports "SHIP commissioning", it SHALL send those SKIs / public keys with an SME "key
1737 material" message to this SHIP node. However, a SHIP node "A" that supports "SHIP commissioning"
1738 expresses with its "keyMaterial.maxEntries" element an upper bound of SKIs / public keys that it can
1739 store (see section 12.3.1.6). Therefore, a "SHIP commissioning tool" SHOULD request the value of
1740 "keyMaterial.maxEntries" of SHIP node "A" (by sending an SME "key material request" message)
1741 before it tries to add new trustworthy foreign SKIs / public keys at SHIP node "A".

1742 If a "SHIP commissioning tool" wants to add a new SHIP commission tool to a SHIP node that
1743 supports "SHIP commissioning", it SHALL send the SKI / public key of the additional SHIP

1744 commissioning tool with an SME "key material" message and set
1745 keyMaterial.entry.commissioningTool to true. A second SHIP commissioning tool provides
1746 redundancy and allows to avoid problems if the other SHIP commissioning tool can no longer be
1747 used.

1748 **Delete foreign SKIs:**

1749 If a "SHIP commissioning tool" wants to delete trustworthy foreign SKIs / public keys on a SHIP node
1750 that supports "SHIP commissioning", it SHALL send those SKIs with an SME "key material delete"
1751 message to the SHIP node where such trustworthy SKIs shall be deleted.

1752 **Avoidance of automatisms (informative):**

1753 The definition of the SHIP commissioning tool is intended for user-controlled actions. It is not
1754 intended for automatic key material changes between single nodes or even across a whole SHIP
1755 network.

1756 One problem of automatic changes can occur if there are two SHIP commissioning tools "X" and "Y"
1757 established by the user. The key problem in this case is a proper synchronization between "X" and
1758 "Y". As no synchronization rules are defined in this specification, an automatism may lead to an
1759 infinite loop of key changes by "X" and "Y" if they have keys of other nodes but with different states.
1760 A SHIP node "A" cannot know which state is newer. Likewise, "X" and "Y" cannot determine from the
1761 rules of the current specification which state is newer.

1762 **12.7 QR Code**

1763 QR Code Model 2 with at least "low" ECC level SHOULD be used.

1764 The size of each module of the QR code SHALL be at least $330 \cdot 10^{-6}$ metres. This equals a module size
1765 of 4 pixels when printed with 300dpi.

1766 The quiet zone SHALL have a width of at least 4 modules.

1767 Certain SHIP specific data MAY be integrated into a QR code. The main advantage of the QR code is
1768 that it allows a very user friendly and mutually authenticated connection establishment with smart
1769 phones via "scan", ideally also enabling the smart phone as "commissioning tool".

1770 If SHIP data is integrated, the following encoding rules MUST be fulfilled:

1771 1) **SHIP prefix:** The SHIP specific data in the QR code SHALL start with a SHIP prefix, the string
1772 "SHIP;" in UTF-8 encoding.

1773 2) **secp256r1 SKI (mandatory):** After the SHIP prefix, the SKI MUST follow. The secp256r1 SKI MUST
1774 be encoded as follows:

1775 a) The first 4 bytes MUST include the string "SKI:" in UTF-8 encoding.

1776 b) The next bytes MUST include the SKI value as a non-prefixed hexadecimal string in UTF-8
1777 encoding with an additional space every 4 hexadecimal digits, as also described in section
1778 12.2. Upper or lower case letters MAY be used (0-9, A-F, a-f).

- 1779 c) The last byte MUST include the character ';' (semicolon) in UTF-8 encoding. This marks the
1780 end of the hexadecimal SKI string.
- 1781 d) Example: "SKI:5555 AAAA FfFf 1111 CCCC 3333 EEeE ddDD 9999 2222;"
- 1782 3) **PIN (mandatory if SHIP node has a PIN)**: If the SHIP node also has a PIN, the encoded PIN SHALL
1783 follow after the encoded SKI. The PIN MUST be encoded as follows:
- 1784 a) The first 4 bytes MUST include the string "PIN:" in UTF-8 encoding.
- 1785 b) The next bytes MUST include the PIN as non-prefixed hexadecimal string in UTF-8 encoding
1786 with an additional space every 4 characters, as also described in section 12.5. Upper or lower
1787 case characters MAY be used (0-9,A-F,a-f).
- 1788 c) The last byte MUST include the character ';' (semicolon) in UTF-8 encoding. This marks the
1789 end of the hexadecimal PIN string.
- 1790 d) Example: "PIN:5555 AaAa FF;"
- 1791 4) **ID (recommended)**: The SHIP ID. After the PIN, if present, or otherwise after the SKI, if the PIN is
1792 not present, the ID SHOULD follow. The ID MUST be encoded as follows:
- 1793 a) The first 3 bytes MUST include the string "ID:" in UTF-8 encoding.
- 1794 b) The next bytes MUST include the ID value as string in UTF-8 encoding. The ID value itself
1795 MUST NOT contain a semicolon character!
- 1796 c) The last byte MUST include the character ';' (semicolon) in UTF-8 encoding. This marks the
1797 end of the ID string.
- 1798 d) Example: "ID:EXAMPLEBRAND-EEB01M3EU-001122334455;"
- 1799 5) **BRAND (optional)**: MAY be used after ID to provide brand information about the device in the
1800 QR code.
- 1801 a) The first 6 bytes MUST include the string "BRAND:" in UTF-8 encoding.
- 1802 b) The next bytes MUST include the brand information as string in UTF-8 encoding. The brand
1803 information itself MUST NOT contain a semicolon character!
- 1804 c) The last byte MUST include the character ';' (semicolon) in UTF-8 encoding. This marks the
1805 end of the BRAND string.
- 1806 d) Example: "BRAND:EXAMPLEBRAND;"
- 1807 6) **TYPE (optional)**: MAY be used after BRAND to provide type information about the device in the
1808 QR code.
- 1809 a) The first 5 bytes MUST include the string "TYPE:" in UTF-8 encoding.

- 1810 b) The next bytes MUST include the type information as string in UTF-8 encoding. The type
1811 information itself MUST NOT contain a semicolon character!
- 1812 c) The last byte MUST include the character ';' (semicolon) in UTF-8 encoding. This marks the
1813 end of the TYPE string.
- 1814 d) Example: "TYPE:DISHWASHER;"
- 1815 7) **MODEL (optional)**: MAY be used behind TYPE to provide model information about the device in
1816 the QR code.
- 1817 a) The first 6 bytes MUST include the string "MODEL:" in UTF-8 encoding.
- 1818 b) The next bytes MUST include the model information as string in UTF-8 encoding. The model
1819 information itself MUST NOT contain a semicolon character!
- 1820 c) The last byte MUST include the character ';' (semicolon) in UTF-8 encoding. This marks the
1821 end of the MODEL string.
- 1822 d) Example: "MODEL:EEB01M3EU;"
- 1823 8) **brainpoolP256r1 SKI (optional)**: MAY be used behind MODEL. The brainpoolP256r1 SKI MUST be
1824 encoded as follows:
- 1825 a) The first 5 bytes MUST include the string "BSKI:" in UTF-8 encoding.
- 1826 b) The next bytes MUST include the SKI value as a non-prefixed hexadecimal string in UTF-8
1827 encoding with an additional space every 4 hexadecimal digits, as also described in section
1828 12.2. Upper or lower case letters MAY be used (0-9, A-F, a-f).
- 1829 c) The last byte MUST include the character ';' (semicolon) in UTF-8 encoding. This marks the
1830 end of the hexadecimal SKI string.
- 1831 d) Example: "BSKI:5555 AAAA Ffff 1111 CCCC 3333 EEeE ddDD 9999 1111;"
- 1832 9) **brainpoolP384r1 SKI (optional)**: MAY be used behind BSKI. brainpoolP384r1 SKI MUST be
1833 encoded as follows:
- 1834 a) The first 6 bytes MUST include the string "B2SKI:" in UTF-8 encoding.
- 1835 b) The next bytes MUST include the SKI value as a non-prefixed hexadecimal string in UTF-8
1836 encoding with an additional space every 4 hexadecimal digits, as also described in section
1837 12.2. Upper or lower case letters MAY be used (0-9, A-F, a-f).
- 1838 c) The last byte MUST include the character ';' (semicolon) in UTF-8 encoding. This marks the
1839 end of the hexadecimal SKI string.
- 1840 d) Example: "B2SKI:5555 AAAA Ffff 1111 CCCC 3333 EEeE ddDD 9999
1841 0000;"

1842 Example QR code encoding with only SKI and PIN:

1843 "SHIP;SKI:5555 AAAA FFFF 1111 CCCC 3333 EEEE DDDD 9999 2222;PIN:5555
 1844 AAAA FF;"



1845

1846 *Figure 6: QR Code Model 2, "low" ECC level, 0.33mm/Module, with SKI and PIN*

1847 The QR code with SKI and PIN has a size of 33x0.33mm = 10.89mm (without quiet zone).

1848

1849 Example QR code encoding including optional values:

1850 "SHIP;SKI:5555 AAAA FFFF 1111 CCCC 3333 EEEE DDDD 9999 2222;PIN:5555
 1851 AAAA FF;ID:EXAMPLEBRAND-EEB01M3EU-
 1852 001122334455;BRAND:EXAMPLEBRAND;TYPE:DISHWASHER;MODEL:EEB01M3EU;"



1853

1854 *Figure 7: QR Code Model 2, "low" ECC level, 0.33mm/module, with all values*

1855 The QR code with all values has a size of 47x0.33mm = 15.51mm (without quiet zone). In a different
 1856 example, the size might vary because of the variable length of ID, BRAND, TYPE and MODEL.

1857

1858 Example QR code encoding including brainpoolP256r1 SKI and brainpoolP384r1 SKI:

1859 "SHIP;SKI:5555 AAAA FFFF 1111 CCCC 3333 EEEE DDDD 9999 2222;PIN:5555
 1860 AAAA FF;BSKI:5555 AAAA FfFf 1111 CCCC 3333 EEeE ddDD 9999
 1861 1111;B2SKI:5555 AAAA FfFf 1111 CCCC 3333 EEeE ddDD 9999 0000;"



1862

1863 *Figure 8: QR Code Model 2, "low" ECC level, 0.33mm/module, with brainpoolP256r1 SKI and brainpoolP384r1 SKI*

1864

13 SHIP Data Exchange

13.1 Introduction

This section (13.1 only) is informative only for the underlying revision of the specification. It is normative for the development of a successor of this specification (though the successor may well adjust this section accordingly).

The concept makes use of the following assumptions:

1. The exchange of data between two SHIP nodes is considered (i.e. no routing to other SHIP nodes and no routing/branching within SHIP nodes are considered).
2. The communication between two SHIP nodes is connection oriented.
3. The connection is bidirectional.
4. Only so-called SHIP messages are exchanged (i.e. no streams).
5. SHIP Transport assumes a communication channel that is chosen or defined and used in a manner to permit safe message separation for every supported message format (this is relevant for binary formats).
6. The communication channel is reliable (verification of successful data transmission).
7. SHIP messages are delivered in the same order as they were submitted.

The concept is designed to permit modifications on the assumptions and mechanisms in future versions of the specification as long as extensibility and compatibility mechanisms are properly considered.

13.2 Terms and Definitions

"Server", "Client", connection role

In this chapter, the terms "server" and "client" are primarily used with regards to an underlying connection technology, i.e. they are usually NOT used as functional roles such as "light switch" or "time information server". See 13.4.1 for details.

Message Definition

This specification provides definitions for so-called SHIP messages. The definitions make use of miscellaneous description concepts (XSD, binary structure, table, ...). For each message, there can also be a process definition on the use of the message.

Message Parts and Composition

A message is composed of message parts. The parts can have different requirements regarding representation.

Example: The "message type" part can be a byte whereas the "message value" can be represented with JSON-UTF8 or another agreed format.

1898 Representation

1899 A representation is an instance of a message part in an "official" format (which implies rules on its
1900 use). Each format is a "wire format".

1901 Depending on the scope or message variant (see below) different representations are permitted
1902 (JSON-UTF8, JSON-UTF16, binary, "mixed"/message dependent).

1903 Note: For a given message variant, all permitted representations must be equivalent (i.e. there must
1904 be a lossless conversion available between the representations)!

1905 Note: Subsequent versions may use further representations.

1906 Message Wire Format

1907 The message wire format determines the wire format of the whole message, i.e. the composition of
1908 all message parts. This requires that the message format allows to determine all representations.

1909 Unless stated otherwise, the term "SHIP message" refers to a wire format. In the wire format, a SHIP
1910 message is of limited size and has a start and an end.

1911 Format Descriptor

1912 For each message wire format, a format descriptor is defined. The format descriptor can be used
1913 during protocol handshake to agree on the wire format for subsequent communication.

1914 SHIP Message Facets

1915 This specification defines different variants and types of SHIP messages.

1916 The variant of a SHIP message is uniquely determined by the SHIP transmission context or by the
1917 message itself (using a message variant identifier).

1918 Examples for SHIP message variants: CMI message (see 13.4.3), connectionHello (SME "Hello"
1919 message, see 13.4.4.1), etc.

1920 Each variant belongs to a SHIP message type. These types ("init", "control", etc.) are defined in Table
1921 11.

1922 Extensibility

1923 Definitions of messages and message parts can prescribe if or how they can be extended with
1924 content not explicitly specified in this specification.

1925 Unless stated otherwise, extension rules serve to achieve and preserve forward and backward
1926 compatibility between devices belonging to different SHIP releases. Rules for manufacturer-specific
1927 extensions are given separately and are marked for this purpose.

1928 "RFU" - "Reserved for Future Use"

1929 Definitions that are marked with "RFU" denote potential future extensions of the specification. Such
1930 extensions can be defined by the specification authority ONLY. Under no circumstances shall such
1931 extensions be used for manufacturer-specific extensions.

1932 Remark: This rule is crucial in order to prevent ambiguities and to keep interoperability. Therefore,
1933 for subsequent releases of the specification, the specification authority can give concrete
1934 specifications for regions formerly marked with "RFU", regardless of any manufacturer-specific use.

1935 **Default Structure Extensibility**

1936 A structure consists of a "parent" element with zero or more optional or mandatory "child" elements.
1937 The child elements have no meaning without the parent element.

1938 A well-defined structure is a structure definition of this specification. Among other things, a
1939 definition imposes rules for the unique identification of parent and child elements, on the order and
1940 presence of child elements, and on their types. The child elements of a well-defined parent are called
1941 "known children".

1942 By default, no other child than a "known child" is permitted in a well-defined parent.

1943 The "default structure extensibility" is a property that can be associated with a given structure. This
1944 property is defined as follows: It marks the structure as being extensible by the specification
1945 authority in a specific way. A future version of the specification may define further child elements
1946 beyond the last "known child" of the current revision's parent element. This property applies to
1947 immediate children of the parent only (i.e. not to second-degree children, e.g.). The "default
1948 structure extensibility" applies only where explicitly mentioned.

1949 The property "default structure extensibility (recursive)" extends the "default structure extensibility"
1950 recursively, i.e. down to all children that are themselves parents for their children.

1951 Remark: This simply means that a future specification may append new children to a parent that
1952 permits default structure extensibility. It does **not** mean that new children can appear before or
1953 between known children. This is most relevant for a serialized form of a structure instance. The
1954 default structure extensibility also does **not** mean that manufacturer-specific children are permitted.

1955 **13.3 Protocol Architecture / Hierarchy**

1956 **13.3.1 Overview**

1957 The following protocol architecture is used to define responsibilities for data exchange and interfaces
1958 or algorithms:

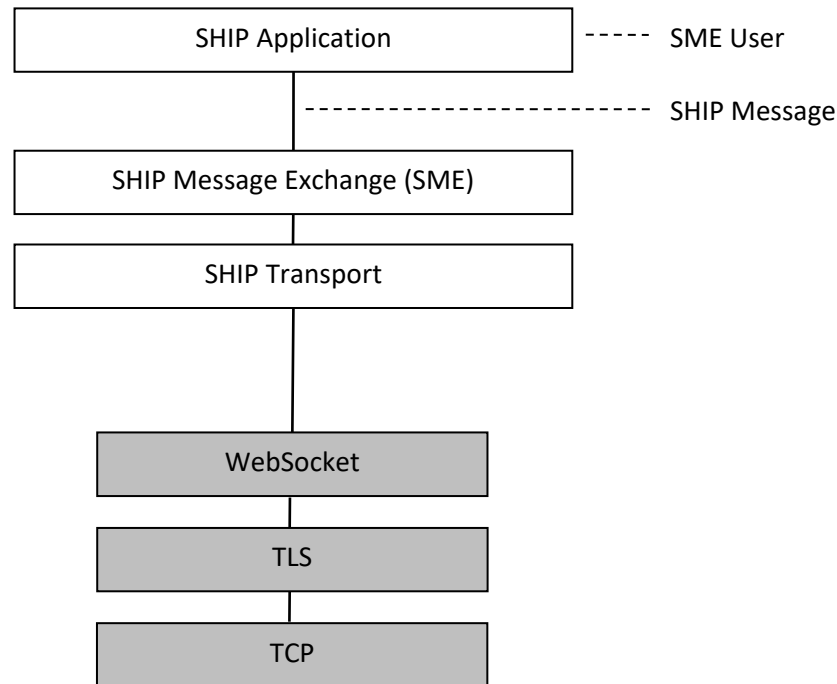


Figure 9: Protocol Architecture and Hierarchy

This chapter focuses on the elements with white boxes. The grey boxes are for reference only.

This specification does not require implementations to provide modules or functionalities to be structured as shown by the architecture. Rather, the architecture provides clear definitions to also ensure extensibility and flexibility in future releases of this specification.

13.3.2 SHIP Message Exchange (SME), SME User

The interface of SME towards a SHIP Application (i.e. the "SME User") is called "SME-AP". Using the ISO-OSI model terminology, SME-AP is the service access point of the SHIP Message Exchange Instance (SME-I). The SME-AP indicates received SHIP messages and takes SHIP messages for the transport to another SME-AP. A SHIP Application uses SME-AP to receive or send SHIP messages.

The SME-I protocol is the SHIP message in the wire format. SME-I itself requires a service access point to a SHIP Transport Instance to perform the message exchange.

Technical details on SME are described in 13.4.

Remark: Future versions of the specification may also define the exchange of a "SHIP Stream" as a further service. The service would probably be parallel to SME-I in terms of a layered hierarchy. It would also require a (properly adjusted) SHIP Transport Instance.

13.3.3 SHIP Transport

A SHIP Transport Instance is responsible for extracting SHIP messages from received data and offer it to SME-I. For the opposite direction, it is responsible for taking SHIP messages from SME-I and transmit proper data to the communication partner.

In this revision of the specification, the SHIP Transport Instance is implemented as follows:

SHIP Transport itself uses the message frame of WebSocket for data exchange (dependent on the used security method). Exactly one SHIP message (in the wire format) is exchanged in exactly one WebSocket message. This means exactly the last WebSocket fragment of a SHIP message has the FIN bit set.

Remark (informative): This basically means that SHIP Transport defines how WebSocket shall be used for SHIP Transport specific needs. This applies to at least this revision of the specification. Future versions of the specification may use other technologies as an alternative to WebSocket. The support of a "SHIP Stream" may also require changes. This can, for example, result in the definition of an additional frame around the SHIP messages. As a consequence, this version of the specification must provide at least a possibility for future extensions on the transport layer and initial compatibility rules. One step towards the preparation for future extension is the definition of "CMI".

13.4 SHIP Message Exchange

13.4.1 Basic Definitions and Responsibilities

Underlying Connection States

This specification does not define how a connection between the SME instances of two SHIP nodes is established. It just assumes that a lower layer or internal mechanisms will provide a connection that is either open or closed.

SME Connection and States

An SME connection is a connection between the SME instances of two SHIP nodes. Depending on the underlying connection state, the SME connection state is also open or closed. In addition to these basic states, each SME instance can have further states. These states can change depending on exchanged SHIP messages. In the following sections, the messages and states are explained in more detail.

SME Connection and Roles

It is REQUIRED that each SME instance has a unique role assigned for each connection. This role is either "server" or "client". If one of the communication partners has the role "server", the other communication partner MUST have the role "client". The role is constant as long the connection is not closed. Unless stated otherwise, it is NOT required that a reconnection between the communication partners assigns the same roles as the previous connection.

Note: A SHIP node MAY have multiple (distinct) SME connections to different communication partners. The SHIP node is permitted to have different roles per connection.

Note: The role described in this section relates to the SME connection only. It does NOT impose any "functional role" related with application specific messages (e.g. a role as "energy management server" would be independent from the SME connection role).

Remark: This specification does not describe how this information is gained from lower layers or implementations.

2017 Responsibilities

2018 All messages and processes described within section 13.4 are the responsibility of the user of the
2019 SME-AP (i.e. of SME User), unless stated otherwise.

2020 Remark (informative): First of all, the use of "SME User" instead of "SHIP Application" shall help make
2021 clear that it is SME that imposes rules on its use. I.e. the rules are not determined by the SHIP
2022 Application. Secondly, in general, a layer should avoid mentioning a specific upper layer.

2023 13.4.2 Basic Message Structure

2024 A SHIP message is formally defined using ABNF:

2025 `Message = MessageType MessageValue`

2026 `MessageType = OCTET`

2027 `MessageValue = 1*OCTET`

2028 The following values are defined for MessageType:

Value	Name
0	init
1	control
2	data
3	end
4-255	RFU

2029 *Table 11: MessageType Values*

2030 If an SME User receives a message with unknown type, it SHALL silently discard it or close the
2031 communication channel.

2032 Remark (informative): The leading type identifier is primarily a preparation for future binary formats.

2033 The subsequent sections define several messages with their type and value. In most cases, the
2034 proper definition for MessageValue is given in a text-based format. However, as will be seen later on,
2035 the SHIP protocol is prepared for multiple formats.

2036 13.4.3 Connection Mode Initialisation (CMI)

2037 As soon as an SME instance has opened a connection to a communication partner, it enters the SME
2038 connection state "CMI_INIT_START", which immediately refers to a connection role specific state.

2039 Remark (informative): CMI is designed to permit more efficient reconnections or format agreements
2040 in subsequent versions of this specification. The concept ensures the definition of a compatibility
2041 concept between these versions through fall-back mechanisms. I.e. devices based upon newer
2042 versions of the specification will benefit from more efficient procedures while even a
2043 firmware/specification downgrade will not do any harm because of a robust fall-back to version 1.0.

2044 A CMI message is defined as follows:

2045 `MessageType = %x00 ; init`

2046 `MessageValue = CmiHead CmiRemainder`

2047 `CmiHead = OCTET`

2048 `CmiRemainder = *OCTET`

2049 The permitted content and meaning of `MessageValue` is defined as follows:

2050 1. If the first byte (`CmiHead`) is 0: State "Connection data preparation".

2051 2. Else: RFU

2052 In this version of the specification, only `CmiHead` of `MessageValue` of a received message is analysed.

2053 `CmiRemainder` is reserved for future use. The following process (including state information) is

2054 defined and SHALL be performed:

2055 1. SME connection role "client":

2056 1.1. `CMI_STATE_CLIENT_SEND`: Client sends a CMI message with `MessageValue = 0` to "server"

2057 and enters state `CMI_STATE_CLIENT_WAIT` immediately afterwards.

2058 2. SME connection role "server":

2059 2.1. `CMI_STATE_SERVER_WAIT`: "Server" waits for the CMI message from "client".

2060 2.2. `CMI_STATE_SERVER_EVALUATE`: "Server" evaluates the received message.

2061 2.2.1. If the received `MessageType` is not 0: "Server" sends a CMI message with

2062 `MessageValue = 0` to "client" and closes the connection afterwards.

2063 2.2.2. If the received `CmiHead` has the decimal value 0: "Server" sends a CMI message with

2064 `MessageValue = 0` to "client" and enters the SME connection state "Connection data

2065 preparation".

2066 2.2.3. If the received `CmiHead` has a decimal value greater than 0: "Server" sends a CMI

2067 message with `MessageValue = 0` to "client" and closes the connection afterwards.

2068 3. SME connection role "client":

2069 3.1. `CMI_STATE_CLIENT_WAIT`: "Client" waits for the CMI message from "server".

2070 3.2. `CMI_STATE_CLIENT_EVALUATE`: "Client" evaluates the received message.

2071 3.2.1. If the received `MessageType` is not 0: "Client" closes the connection immediately.

2072 3.2.2. If the received `CmiHead` has the decimal value 0: "Client" enters the SME connection

2073 state "Connection data preparation".

2074 3.2.3. If the received `CmiHead` has a decimal value greater than 0: "Client" closes the

2075 connection immediately.

2076 Timeout procedure:

2077 The CMI process above begins as soon as a connection has been established (entering state
2078 CMI_INIT_START). This corresponds to the process steps CMI_STATE_CLIENT_SEND and
2079 CMI_STATE_SERVER_WAIT, respectively. Then, the following rules apply:

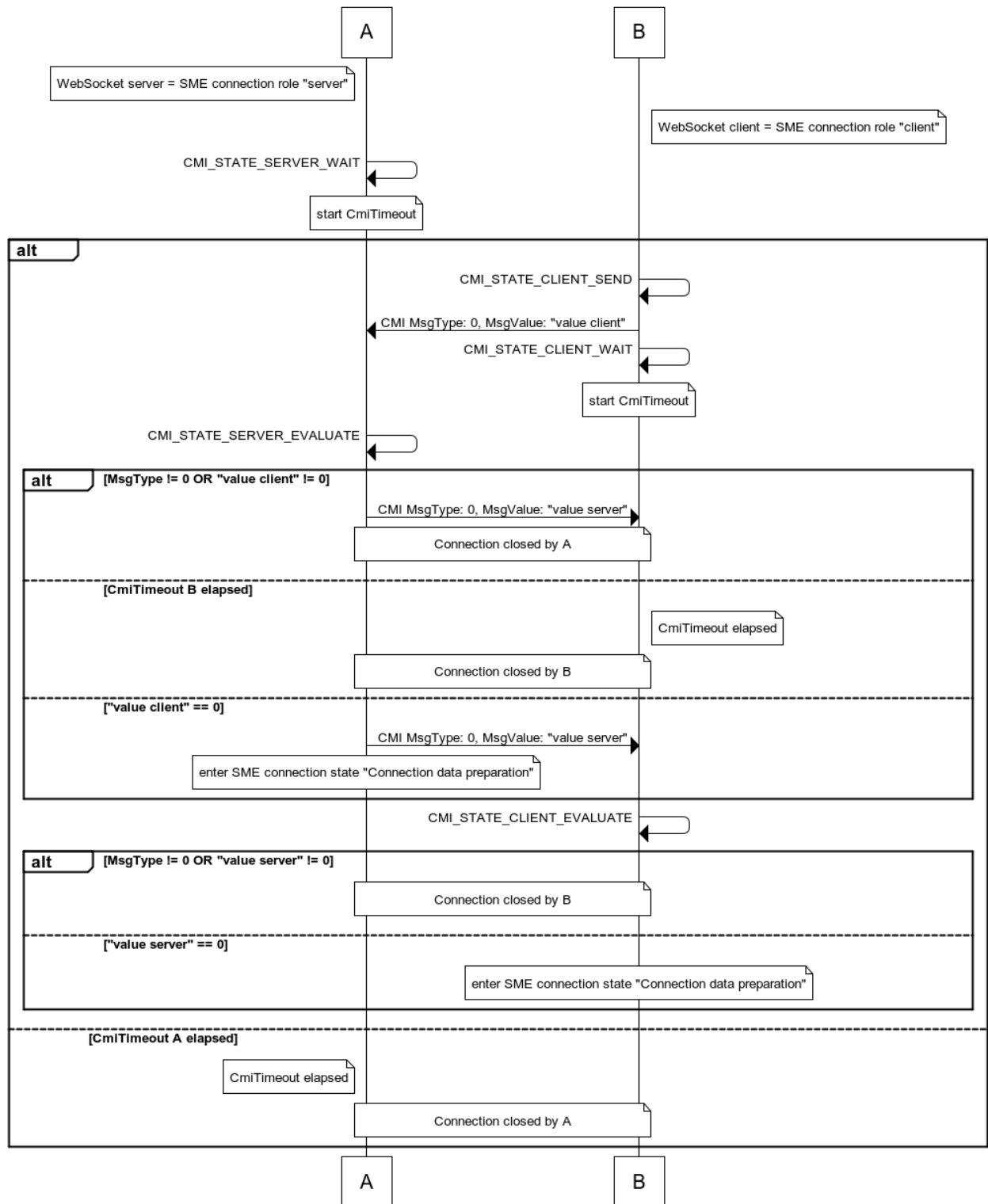
- 2080 1. For process step CMI_STATE_SERVER_WAIT, a timeout of CmiTimeout applies. If the server does
2081 not receive a message before this time elapses, it SHALL close the connection immediately.
- 2082 2. For process step CMI_STATE_CLIENT_WAIT, a timeout of CmiTimeout applies. If the client does
2083 not receive a message before this time elapses, it SHALL close the connection immediately.

2084 An SME User SHALL assign any value from 10 seconds to 30 seconds to CmiTimeout.

2085 Remark on CmiHead "RFU" and potential definitions in subsequent specification versions:

2086 In a subsequent version of this specification, values other than "0" may be defined. In fact, it is
2087 possible to define a multi-byte "extended CmiHead". Compatibility as well as fault tolerance can be
2088 preserved through the rule to reconnect with CmiHead "0" if a previous connection with another
2089 value failed.

2090 A brief overview of the CMI procedure is given in the following sequence diagram.



2091

2092 *Figure 10: CMI Message Sequence Example*2093 **13.4.4 Connection Data Preparation**

2094 If an SME User enters this state, it SHALL proceed with "Connection state Hello" (see 13.4.4.1).

2095 **13.4.4.1 Connection State "Hello"**2096 **13.4.4.1.1 Basic Definitions**

2097 In this state, the SME Users negotiate whether they allow to continue with the next communication
 2098 state or not. This concept enables implementations to give a user of a SHIP node some time to decide
 2099 whether the communication partner can be trusted or not. From a process point of view, a
 2100 connection that is not (yet) trusted is considered "PENDING", whereas it is "READY" as soon the
 2101 connection is trusted (the terms "READY" and "PENDING" are defined in section 13.4.4.1.2).

2102 Remark (informative): Trusting a device or not is typically related to the phase when a device just
 2103 presented its certificate and public key (see section 12) in order to allow a user to verify this
 2104 information (provided there is a proper user interface). I.e. this procedure can start even before
 2105 Connection Mode Initialisation begins (and in case of "auto accept", it can also be finished before
 2106 CMI begins). In case of a real user based (manual) verification, it is likely that such a verification time
 2107 is required at least for the very first establishment of a connection between two SHIP nodes.

2108 This state uses an SME "hello" message, which is defined as follows:

2109 MessageType = %x01 ; control

2110 MessageValue = SmeHelloValue

2111 SmeHelloValue = *OCTET

2112 The content of SmeHelloValue is defined as follows: The structure is defined by the SHIP root tag
 2113 "connectionHello" (including the root tag "connectionHello") of the XSD
 2114 "SHIP_TS_TransferProtocol.xsd". The default structure extensibility applies to this structure. The
 2115 format of this structure MUST be JSON-UTF8.

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
connectionHello.phase	M	The sender's phase during the "hello" process (enumeration: pending, ready, aborted). See 13.4.4.1.2.
connectionHello.waiting	O	Remaining time (in milliseconds) granted by the sender.
connectionHello.prolongationRequest	O	Request to prolong the remaining time.
connectionHello.keyMaterialState	M/O ^{*1}	
connectionHello.keyMaterialState.updateCounter	M/O ^{*1}	Current state of the key material update counter, see also section 12.1.3.

2116 *Table 12: Structure of SmeHelloValue of SME "hello" Message.*

2117 *1: For SHIP versions ≥ 1.1 , it is mandatory to send this field. For SHIP versions < 1.1 , this field is
2118 undefined (thus "optional"). A recipient SHALL NOT consider it an error if the element is absent.

2119 The SME "hello" process does not require knowledge of the connection role (server or client).
2120 Instead, each of the SME Users SHALL execute the process as described below.

2121 *13.4.4.1.2 Process Overview*

2122 When this process is entered, an SME User SHALL be in exactly one of the following basic "Hello"
2123 states for a given connection to another node (depending on the trust in the connection, see
2124 13.4.4.1.1):

2125 1. **READY**

2126 This state MUST ONLY be entered if the communication partner is already trusted. In this state,
2127 the SME User is ready to proceed with the next state after "Hello" as soon as it receives from the
2128 communication partner that it is ready to proceed as well. The SME User will only wait for this
2129 information from the communication partner for a limited time. However, the communication
2130 partner can request a prolongation of this time.

2131 2. **PENDING**

2132 In this state, the SME User is not yet ready to proceed with the next state after "Hello".
2133 Nevertheless, it will only wait for a final "READY" information from the communication partner
2134 for a limited time, although the communication partner can request a prolongation of this time
2135 (this is the same principle as for state "READY"). Furthermore, the SME User must ensure to

2136 a. EITHER switch into state "READY" and inform the communication partner accordingly in
2137 time,

2138 b. OR request a prolongation of the time the communication partner waits for the "READY"
2139 information,

2140 c. OR report the abortion of this process if the SME User finally decides not to trust the
2141 communication partner.

2142 The third basic state "HELLO_OK" can only be accessed if certain conditions are fulfilled as described
2143 in the following sections.

2144 Please note: The connection state "Hello" is defined independent from any authentication
2145 procedures of any lower layer. It just assumes the knowledge whether a communication partner is
2146 trusted or not. This means that lower layers need to specify the procedures and conditions to trust
2147 another device and whether to keep this information persistently or not.

2148 *13.4.4.1.3 Process Details*

2149 **Timer Overview:**

2150 The following timers are defined:

2151 1. Wait-For-Ready-Timer

- 2152 Default value: T_hello_init (see below).
- 2153 Purpose: The communication partner must send its "READY" state (or request for prolongation")
2154 before the timer expires.
- 2155 2. Send-Prolongation-Request-Timer
- 2156 Purpose: Local timer to request for prolongation at the communication partner in time (i.e.
2157 before the communication partner's Wait-For-Ready-Timer expires).
- 2158 3. Prolongation-Request-Reply-Timer
- 2159 Purpose: Detection of response timeout on prolongation request.
- 2160 Each timer operates in a countdown mode. If a timer is activated, it MUST be initialized with a value
2161 greater than 0 in general, and according to the rules of the corresponding process step specifically.
2162 Details on the use of the timers are described for the appropriate process steps. See also
2163 "Implementation Advice".
- 2164 The following symbols for constant time values are used:
- 2165 1. T_hello_init: Any value from 60 seconds up to (and including) 240 seconds. An implementation
2166 can choose for any value of the specified range. However, this value SHALL be constant during a
2167 connection. This value SHOULD also be constant across different connections.
- 2168 2. T_hello_inc: The same value as T_hello_init.
- 2169 3. T_hello_prolong_thr_inc: 30 seconds.
- 2170 4. T_hello_prolong_waiting_gap: 15 seconds.
- 2171 5. T_hello_prolong_min: 1 second.
- 2172 **States and Sub-states Overview:**
- 2173 Depending on the basic state, the following sub-states are defined:
- 2174 1. Sub-states of basic state "READY":
- 2175 1.1. SME_HELLO_STATE_READY_INIT
- 2176 1.2. SME_HELLO_STATE_READY_LISTEN
- 2177 1.3. SME_HELLO_STATE_READY_TIMEOUT
- 2178 2. Sub-states of basic state "PENDING":
- 2179 2.1. SME_HELLO_STATE_PENDING_INIT
- 2180 2.2. SME_HELLO_STATE_PENDING_LISTEN
- 2181 2.3. SME_HELLO_STATE_PENDING_TIMEOUT

2182 General information on the basic state "READY":

2183 In this case, the SME User need not request a prolongation with the communication partner. Thus,
2184 Send-Prolongation-Request-Timer is not needed. Of course, this requires submitting information on
2185 the own "READY"-state to the communication partner. Consequently, any sub-element "waiting" for
2186 an SME "hello" message received from the communication partner can be ignored.

2187 General information on the basic state "PENDING":

2188 In this case, the use of Send-Prolongation-Request-Timer is required as soon as the "waiting" sub-
2189 element from the communication partner is available, independent from the communication
2190 partner's phase (except for "aborted").

2191 As described above, an SME User with state "PENDING" requires the communication partner to also
2192 announce its basic state "READY" in time. Thus, the Wait-For-Ready-Timer is required as long as this
2193 information has not been received.

2194 **Sub-state SME_HELLO_STATE_READY_INIT:**

2195 This is the first state an SME User SHALL enter if it is in the basic state "READY". In this state, it SHALL

- 2196 1. initialise Wait-For-Ready-Timer to the default value and start the timer,
2197 2. deactivate Send-Prolongation-Request-Timer and Prolongation-Request-Reply-Timer,
2198 3. send an SME "hello" update message (see definition below),
2199 4. enter state SME_HELLO_STATE_READY_LISTEN.

2200 Please note that this state does not evaluate any received messages. I.e. SME "hello" messages that
2201 are received before or during this state are subject of subsequent states.

2202 **Sub-state SME_HELLO_STATE_READY_LISTEN:**

2203 In this state, the SME User SHALL evaluate SHIP messages received from the communication partner.

2204 Only SME "hello" messages are considered here. The following rules apply:

- 2205 1. If the received message has sub-element "phase" set to "ready": Enter state "HELLO_OK".
2206 2. If the received message has sub-element "phase" set to "pending" and NO sub-element
2207 "prolongationRequest" is set: No specific action is required here (i.e. ignore the message).
2208 3. If the received message has sub-element "phase" set to "pending" and sub-element
2209 "prolongationRequest" is set to "true":
2210 a. Execute the common procedure to decide an incoming prolongation request.
2211 b. Execute the common procedure to send an SME "hello" update message.
2212 4. If the received message has sub-element "phase" set to "aborted": Execute the common "abort"
2213 procedure.

- 2214 If a received message is not an SME "hello" message while in this state, the SME User SHALL execute
2215 the common "abort" procedure.
- 2216 **Sub-state SME_HELLO_STATE_READY_TIMEOUT:**
2217 This state SHALL be entered if Wait-For-Ready-Timer expired. The SME User SHALL execute the
2218 common "abort" procedure.
- 2219 **Sub-state SME_HELLO_STATE_PENDING_INIT:**
2220 This is the first state an SME User SHALL enter if it is in the basic state "PENDING". In this state, it
2221 SHALL
- 2222 1. initialise Wait-For-Ready-Timer to the default value and start the timer,
2223 2. deactivate Send-Prolongation-Request-Timer and Prolongation-Request-Reply-Timer,
2224 3. send an SME "hello" update message (see definition below),
2225 4. enter state SME_HELLO_STATE_PENDING_LISTEN.
- 2226 **Sub-state SME_HELLO_STATE_PENDING_LISTEN:**
2227 In this state, the SME User SHALL evaluate SHIP messages received from the communication partner.
2228 Only SME "hello" messages are considered here. The following rules apply:
- 2229 1. If the received message has sub-element "phase" set to "ready" and NO sub-element "waiting":
2230 Execute the common "abort" procedure.
- 2231 2. If the received message has sub-element "phase" set to "ready" and sub-element "waiting" is set:
2232 a. Deactivate Wait-For-Ready-Timer and Prolongation-Request-Reply-Timer.
2233 b. If the received sub-element "waiting" is greater than or equal to T_hello_prolong_thr_inc:
2234 Initialize Send-Prolongation-Request-Timer to a new value as described below and (re-)start
2235 the timer. Otherwise (i.e. the received sub-element "waiting" is less than
2236 T_hello_prolong_thr_inc): Deactivate Send-Prolongation-Request-Timer.
- 2237 3. If the received message has sub-element "phase" set to "pending" and sub-element "waiting" is
2238 set and NO sub-element "prolongationRequest" is set:
2239 a. Deactivate Prolongation-Request-Reply-Timer.
2240 b. If the received sub-element "waiting" is greater than or equal to T_hello_prolong_thr_inc:
2241 Initialize Send-Prolongation-Request-Timer to a new value as described below and (re-)start
2242 the timer. Otherwise (i.e. the received sub-element "waiting" is less than
2243 T_hello_prolong_thr_inc): Deactivate Send-Prolongation-Request-Timer.
- 2244 4. If the received message has sub-element "phase" set to "pending" and NO sub-element "waiting"
2245 and sub-element "prolongationRequest" is set to "true":
2246 a. Execute the common procedure to decide an incoming prolongation request.

- 2247 b. Execute the common procedure to send an SME "hello" update message.
- 2248 5. If the received message has sub-element "phase" set to "aborted": Execute the common "abort"
2249 procedure.
- 2250 6. If the received message does not match any of the aforementioned schemes: Execute the
2251 common "abort" procedure.
- 2252 In addition, the following rules apply:
- 2253 1. If a received message is not an SME "hello" message while in this state, the SME User SHALL
2254 execute the common "abort" procedure.
- 2255 2. If an SME User finally decides to not trust the communication partner, the SME User SHALL
2256 execute the common "abort" procedure.
- 2257 The following rules SHALL be applied to calculate a new value for Send-Prolongation-Request-Timer:
- 2258 1. The new value SHALL be by T_hello_prolong_waiting_gap lower than the value from the received
2259 sub-element "waiting".
- 2260 2. Under normal operation, the value calculated above should be positive. However, in case the
2261 result is less than T_hello_prolong_min, the SME User SHALL disable the Send-Prolongation-
2262 Request-Timer.
- 2263 **Sub-state SME_HELLO_STATE_PENDING_TIMEOUT:**
- 2264 This state SHALL be entered if any of the timers expired:
- 2265 1. If Wait-For-Ready-Timer expired: The SME User SHALL execute the common "abort" procedure.
- 2266 2. If Send-Prolongation-Request-Timer expired:
- 2267 a. The SME User SHALL send an SME "hello" message with the following content:
- 2268 i. Sub-element "phase" set to "pending".
- 2269 ii. Sub-element "prolongationRequest" set to "true".
- 2270 iii. No further sub-element shall be set.
- 2271 b. Initialize Prolongation-Request-Reply-Timer to the value of the last received sub-element
2272 "waiting" of the communication partner. If no sub-element "waiting" was received so far, the
2273 1.1-fold of the current value of Wait-For-Ready-Timer SHALL be used as the initialization
2274 value.
- 2275 c. Start Prolongation-Request-Reply-Timer.
- 2276 d. Return to the previous state.
- 2277 3. If Prolongation-Request-Reply-Timer expired: The SME User SHALL execute the common "abort"
2278 procedure.

2279 **Switching Between Basic States "READY" and "PENDING":**

- 2280 1. It is NOT permitted to switch from basic state "READY" and its sub-states to basic state
 2281 "PENDING" and any of its sub-states.
- 2282 2. If an SME User switches from basic state "PENDING" to "READY", it SHALL
- 2283 a. deactivate Send-Prolongation-Request-Timer and Prolongation-Request-Reply-Timer,
- 2284 b. send an SME "hello" update message (with its new state "READY"),
- 2285 c. enter state
- 2286 i. EITHER "HELLO_OK" (only if one of the previously received SME "hello" messages had
 2287 sub-element "phase" set to "ready")
- 2288 ii. OR SME_HELLO_STATE_READY_LISTEN otherwise.

2289 **State HELLO_OK:**

2290 All timers of connection state "Hello" can be deactivated. The SME User SHALL continue with
 2291 "Connection state Protocol handshake" (see 13.4.4.2).

2292 **Common "abort" procedure:**

2293 This procedure SHALL be executed where referenced. The SME User SHALL

- 2294 1. deactivate all SME specific timers of connection state "Hello",
- 2295 2. send an SME "hello" message with sub-element "phase" set to "aborted" (further sub-elements
 2296 SHALL NOT be set) if the connection is not already closed,
- 2297 3. close the connection (if the connection is not already closed).

2298 **Common Procedure for Sending an SME "hello" Update Message:**

2299 This procedure SHALL be executed where referenced. The SME User SHALL send an SME "hello"
 2300 message with the following content:

- 2301 1. Sub-element "phase" set to
- 2302 a. "ready" in case of the basic state "READY",
- 2303 b. or "pending" in case of the basic state "PENDING".
- 2304 2. Sub-element "waiting" set to the current value of Wait-For-Ready-Timer if Wait-For-Ready-Timer
 2305 is active. This sub-element SHALL NOT be set if the timer is not active.
- 2306 Further sub-elements SHALL NOT be set.

2307 **Common Procedure to Decide an Incoming Prolongation Request:**

2308 This procedure SHALL be executed where referenced.

- 2309 1. If an SME User accepts the prolongation request: It SHALL increase its Wait-For-Ready-Timer by
 2310 T_hello_inc.

2311 2. Otherwise: No specific action required.

2312 Further rules apply:

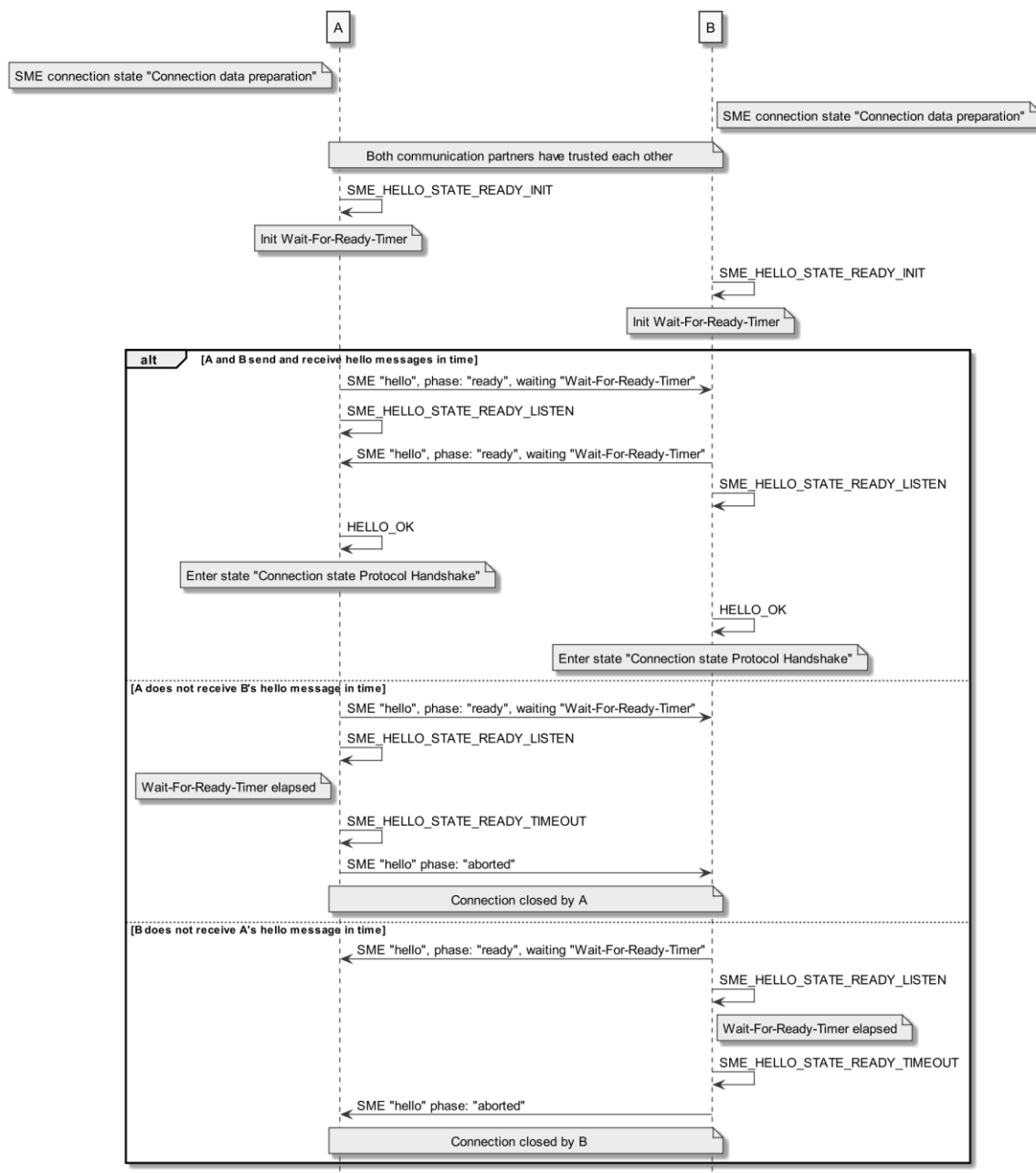
2313 1. An SME User SHALL accept at least two prolongation requests.

2314 **Implementation Advice:**

2315 Several rules and procedures described above include instructions on the report of a state or
 2316 response. The timers of this section SHALL NOT be used to delay such reports more than necessary.

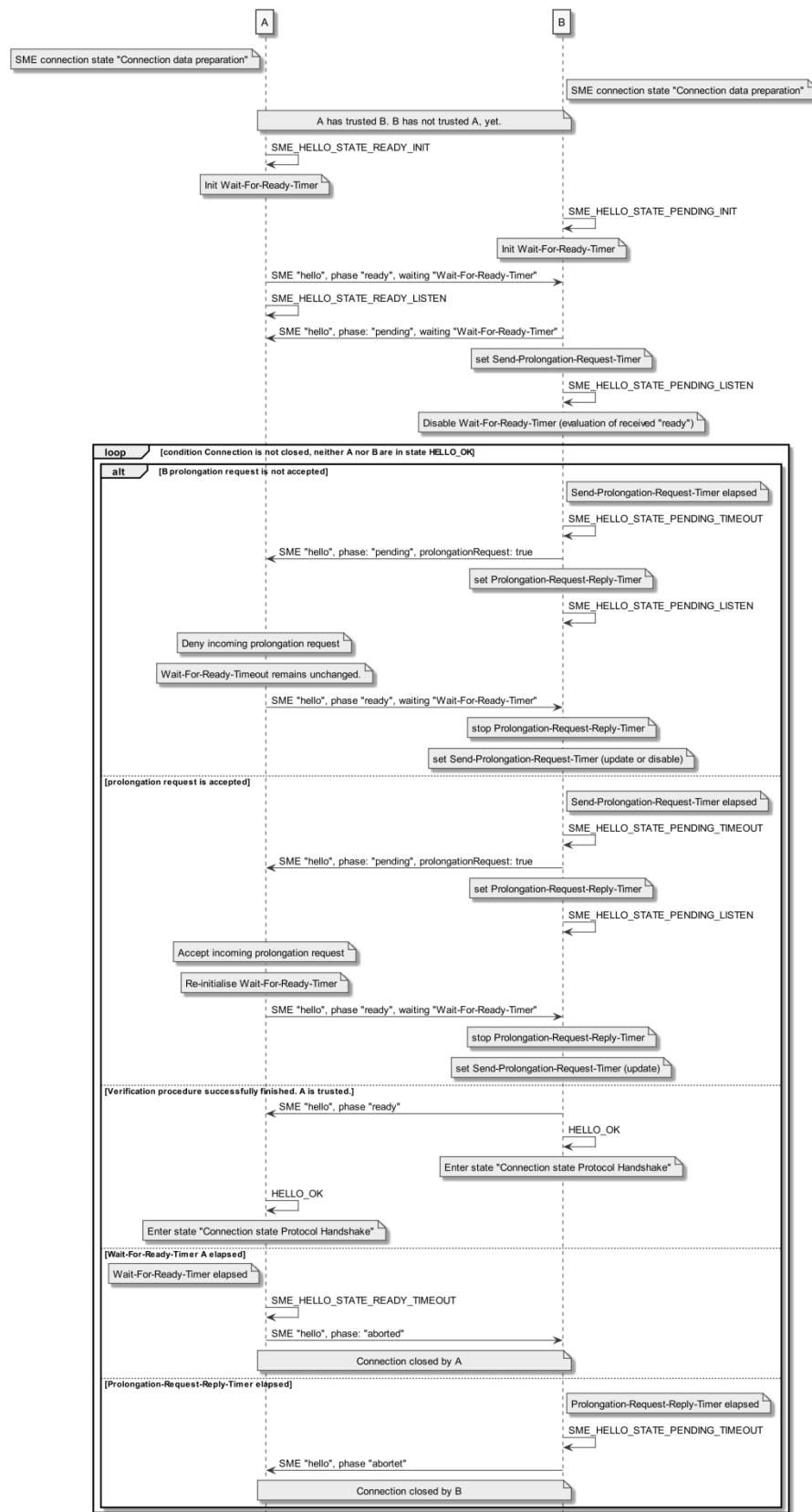
2317 **Example Sequence Diagrams:**

2318 Example sequence diagrams for connection state "Hello" are shown in the subsequent figures:



2319

2320 *Figure 11: Connection State "Hello" Sequence Example Without Prolongation Request: "A" and "B" already trust each other;*
 2321 *"B" is slower/delayed.*



2322

2323 *Figure 12: Connection State "Hello" Sequence Example With Prolongation Request.*

2324 **13.4.4.2 Connection State "Protocol handshake"**

2325 **13.4.4.2.1 Basic Definitions**

2326 In this state, the communication partners agree on the further SHIP protocol version and on the
2327 message format to continue with.

2328 This state uses an SME "protocol handshake" message and a dedicated error message.

2329 **SME "Protocol Handshake" Message:**

2330 The SME "protocol handshake" message is defined as follows:

2331 `MessageType = %x01 ; control`

2332 `MessageValue = SmeProtocolHandshakeValue`

2333 `SmeProtocolHandshakeValue = *OCTET`

2334 The content of SmeProtocolHandshakeValue is defined as follows: The structure is defined by the
2335 SHIP root tag "messageProtocolHandshake" (including the root tag "messageProtocolHandshake") of
2336 the XSD "SHIP_TS_TransferProtocol.xsd". The default structure extensibility applies to this structure.
2337 The format of this structure MUST be JSON-UTF8.

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
messageProtocolHandshake.handshakeType	M	The kind of the handshake information (enumeration: announceMax, select).
messageProtocolHandshake.version	M	Parent element of SHIP specification version information.
messageProtocolHandshake.version.major	M	Version information: Major version part.
messageProtocolHandshake.version.minor	M	Version information: Minor version part.
messageProtocolHandshake.formats	M	Protocol format(s).
List of "format" (1..unbounded)	M	In general, the subsequent child element "format" SHALL be present at least one time and CAN be present more than one time. However, the number of permitted occurrences finally depends on the phase of the protocol

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
		handshake. See definitions in the text for details! The "format" instances of the list SHALL have a unique value, i.e. no two "format" values may be identical.
messageProtocolHandshake.formats.format	M	Protocol format. See text for permitted values.

2338 *Table 13: Structure of SmeProtocolHandshakeValue of SME "Protocol Handshake" Message.*

2339 Permitted values for the child element "format" are "JSON-UTF8" and "JSON-UTF16" (without the
2340 quotation marks), but only "JSON-UTF8" is REQUIRED to be supported (see also 13.4.4.2.2). Other
2341 values are reserved for future use. An empty string is NOT a permitted value.

2342 **SME "Protocol Handshake Error" Message:**

2343 The SME "protocol handshake error" message is defined as follows:

2344 Message_{Type} = %x01 ; control

2345 Message_{Value} = SmeProtocolHandshakeErrorValue

2346 SmeProtocolHandshakeErrorValue = *OCTET

2347 The content of SmeProtocolHandshakeErrorValue is defined as follows: The structure is defined by
2348 the SHIP root tag "messageProtocolHandshakeError" (including the root tag
2349 "messageProtocolHandshakeError") of the XSD "SHIP_TS_TransferProtocol.xsd". The default
2350 structure extensibility applies to this structure. The format of this structure MUST be JSON-UTF8.

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
messageProtocolHandshakeError.error	M	Error number.

2351 *Table 14: Structure of SmeProtocolHandshakeErrorValue of SME "Protocol Handshake Error" Message.*

2352 **13.4.4.2.2 Compatibility Aspects**

2353 In this version of the specification, the exchange of SME "protocol handshake" messages is
2354 exclusively executed with JSON-UTF8 as described above.

2355 Each communication partner MUST support each SHIP specification version from "1.0" up to and
2356 including their own maximum supported SHIP specification version.

- 2357 Each communication partner MUST support the format JSON-UTF8 (see also 13.4.4.2.1).
- 2358 Remark (informative): Subsequent versions of the SHIP specification may define a Connection Mode
2359 Initialization that permits a different protocol handshake. Even the omission of a handshake might be
2360 defined dependent on the circumstances. However, a fall-back mechanism to version 1.0 of the SHIP
2361 specification must be defined and preserved.
- 2362 *13.4.4.2.3 Protocol Handshake Process*
- 2363 **Connections Roles:**
- 2364 The concept requires the knowledge of the SME connection role (client, server).
- 2365 **Timer overview:**
- 2366 The following timer is defined:
- 2367 1. Wait-Timer
- 2368 Default value: 10 seconds.
- 2369 Purpose: The communication partner must provide the required protocol handshake information
2370 before the timer expires.
- 2371 **State SME_PROT_H_STATE_SERVER_INIT:**
- 2372 This is the first state a server SME User SHALL enter. In this state, it SHALL
- 2373 1. initialize Wait-Timer to the default value and start the timer,
- 2374 2. enter state SME_PROT_H_STATE_SERVER_LISTEN_PROPOSAL.
- 2375 **State SME_PROT_H_STATE_CLIENT_INIT:**
- 2376 This is the first state a client SME User SHALL enter. In this state, it SHALL
- 2377 1. send an SME "protocol handshake" message with the following content:
- 2378 a. Sub-element "handshakeType" set to "announceMax".
- 2379 b. Sub-element version (and its children) set to the maximum supported SHIP specification
2380 version: In this version of the specification, this means "major" MUST be set to "1" and
2381 "minor" MUST be set to "1".
- 2382 c. Sub-element "formats" set to all format values supported by the client.
- 2383 2. initialise Wait-Timer to the default value and start the timer,
- 2384 3. enter state SME_PROT_H_STATE_CLIENT_LISTEN_CHOICE.
- 2385 **State SME_PROT_H_STATE_SERVER_LISTEN_PROPOSAL:**
- 2386 In this state, a server SME User evaluates received messages from the client:
- 2387 1. If the received message is a valid SME "protocol handshake" message: The server SME User
2388 SHALL

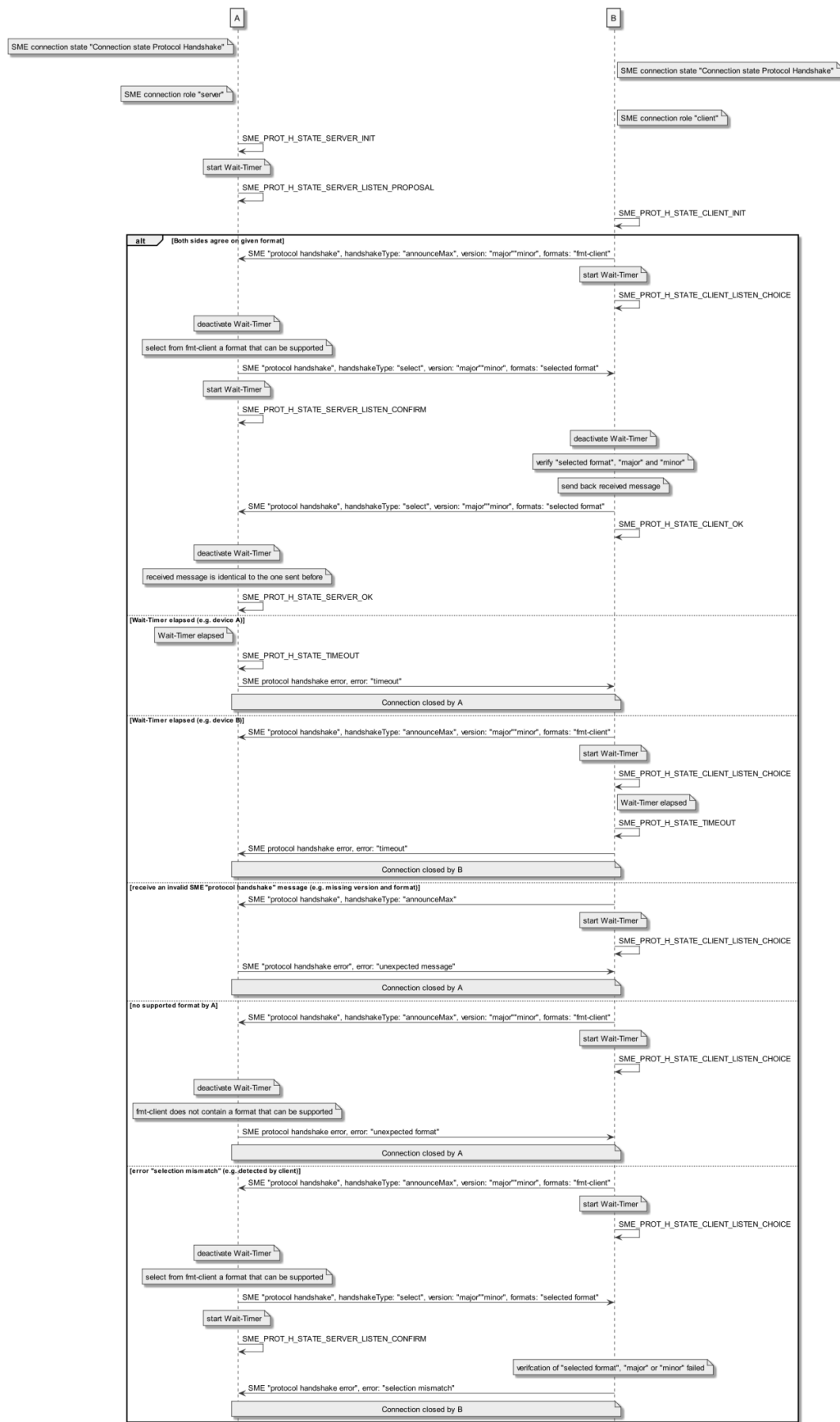
- 2389 a. deactivate Wait-Timer,
- 2390 b. select the maximum supported SHIP specification version supported by both communication
2391 partners,
- 2392 c. select a single format supported by both communication partners,
- 2393 d. send an SME "protocol handshake" message with the following content:
- 2394 i. Sub-element "handshakeType" set to "select".
- 2395 ii. Sub-element version (and its children) set to the selected SHIP specification version.
- 2396 iii. Sub-element "formats" set to the selected format.
- 2397 e. re-initialize Wait-Timer to the default value and start the timer,
- 2398 f. enter state SME_PROT_H_STATE_SERVER_LISTEN_CONFIRM.
- 2399 2. Otherwise: Execute the common "abort" procedure with error type "unexpected message".
- 2400 **State SME_PROT_H_STATE_CLIENT_LISTEN_CHOICE:**
- 2401 In this state, a client SME User evaluates received messages from the server to analyse the server's
2402 selection:
- 2403 1. If the received message is a valid SME "protocol handshake" message: The client SME User SHALL
- 2404 a. deactivate Wait-Timer,
- 2405 b. verify if the received sub-element "handshakeType" is set to "select" and – in case the
2406 verification succeeded – continue with the next step,
- 2407 c. verify if the received sub-element "version" matches the client's capability and – in case the
2408 verification succeeded – continue with the next step,
- 2409 d. verify if the received sub-element "formats" contains a single format and matches the client's
2410 capability and – in case the verifications succeeded – continue with the next step,
- 2411 e. send the received SME "protocol handshake" message back to the server (this denotes the
2412 confirmation of the server's choice),
- 2413 f. enter state SME_PROT_H_STATE_CLIENT_OK.
- 2414 If any of the aforementioned verifications failed, the node SHALL execute the common "abort"
2415 procedure with error type "selection mismatch".
- 2416 2. Otherwise: Execute the common "abort" procedure with error type "unexpected message".
- 2417 **State SME_PROT_H_STATE_SERVER_LISTEN_CONFIRM:**
- 2418 In this state, a server SME User evaluates received messages from the client:

- 2419 1. If the received message is a valid SME "protocol handshake" message: The server SME User
2420 SHALL
- 2421 a. deactivate Wait-Timer,
- 2422 b. verify if the received message is identical to the message the server previously (in state
2423 SME_PROT_H_STATE_SERVER_LISTEN_PROPOSAL) submitted to the client and – in case the
2424 verification succeeded – continue with the next step,
- 2425 c. enter state SME_PROT_H_STATE_SERVER_OK.
- 2426 If any of the aforementioned verifications failed, the SME User SHALL execute the common
2427 "abort" procedure with error type "selection mismatch".
- 2428 2. Otherwise: Execute the common "abort" procedure with error type "unexpected message".
- 2429 **Sub-state SME_PROT_H_STATE_TIMEOUT:**
- 2430 This state SHALL be entered if Wait-Timer expired. The SME User SHALL execute the common "abort"
2431 procedure with error type "timeout".
- 2432 **States SME_PROT_H_STATE_CLIENT_OK, State SME_PROT_H_STATE_SERVER_OK:**
- 2433 As soon as an SME User enters this state, it SHALL switch to the selected SHIP specification version
2434 and selected format. Then, it SHALL proceed with "Connection state PIN verification".
- 2435 Example (informative): Before this state is reached, all messages are exchanged with the format
2436 JSON-UTF8 (in SHIP specification version 1.0). For this example, we assume there is a new SHIP
2437 specification version 2.1 available and a binary format "ASN.1-PER" and both SME Users agreed on
2438 this version and format. This means all messages MUST follow the protocol and format requirements
2439 as soon as the "..._OK" state is reached. This may include new values for MessageType and non-
2440 textual (but compact, i.e. efficient) content of MessageValue.
- 2441 **Common "abort" procedure:**
- 2442 This procedure SHALL be executed where referenced. The SME User SHALL
- 2443 1. deactivate all SME specific timers of connection state "Protocol handshake",
- 2444 2. send an SME "protocol handshake error" message with sub-element "error" set to the proper
2445 value (see Table 15),
- 2446 3. close the connection.

Value	Error type
0	RFU
1	Timeout
2	unexpected message
3	selection mismatch

Value	Error type
4-255	RFU

2447 Table 15: Values of Sub-element "error" of messageProtocolHandshakeError.



2448

2449 Figure 13: Connection State "Protocol Handshake" Message Sequence Example

2450 **13.4.4.3 Connection State "PIN Verification"**

2451 *13.4.4.3.1 Introduction (Informative)*

2452 The exchange of a PIN is a common procedure to gain a certain (minimum) trust level required to
 2453 permit the exchange of sensitive data between authorized communication partners only. In general,
 2454 a PIN based authorization can be considered a more secure methodology than a simple "push
 2455 button" based method with "auto accept". Consequently, successful PIN exchange might be
 2456 mandatory for certain data that is "too sensitive" to be exchanged between communication partners
 2457 that are introduced by "push button/auto accept" only. See chapter 12 for more details.

2458 *13.4.4.3.2 Basic Definitions*

2459 In this state, the communication partners exchange information on their PIN requirements.

2460 This state uses SME messages for the PIN state, PIN input, and a dedicated error message. Broadly
 2461 speaking, "PIN state" is used to inform the communication partner whether a PIN is expected. The
 2462 message "PIN input" serves for the transport of a PIN to the owner of the PIN. The error message
 2463 indicates if a PIN is wrong.

2464 **SME "PIN state" Message:**

2465 The SME "PIN state" message is defined as follows:

```
2466     MessageType = %x01 ; control
2467     MessageValue = SmeConnectionPinStateValue
2468     SmeConnectionPinStateValue = *OCTET
```

2469 The content of SmeConnectionPinStateValue is defined as follows: The structure is defined by the
 2470 SHIP root tag "connectionPinState" (including the root tag "connectionPinState") of the XSD
 2471 "SHIP_TS_TransferProtocol.xsd". The default structure extensibility applies to this structure. The
 2472 format of this structure MUST be the format agreed with the protocol handshake (see 13.4.4.2).

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
connectionPinState.pinState	M	The originator's PIN state (in relation to the recipient). See 13.4.4.3.5.1.
connectionPinState.inputPermission	O	Information whether PIN receipt is currently permitted or not. See 13.4.4.3.5.1.

2473 *Table 16: Structure of SmeConnectionPinStateValue of SME "Pin state" message.*

2474 **SME "PIN input" Message:**

2475 The SME "PIN input" message is defined as follows:

2476 `MessageType = %x01 ; control`

2477 `MessageValue = SmeConnectionPinInputValue`

2478 `SmeConnectionPinInputValue = *OCTET`

2479 The content of `SmeConnectionPinInputValue` is defined as follows: The structure is defined by the
 2480 SHIP root tag "connectionPinInput" (including the root tag "connectionPinInput") of the XSD
 2481 "SHIP_TS_TransferProtocol.xsd". The default structure extensibility applies to this structure. The
 2482 format of this structure MUST be the format agreed with the protocol handshake (see 13.4.4.2).

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
connectionPinInput.pin	M	The receiver's PIN.

2483 *Table 17: Structure of SmeConnectionPinInputValue of SME "Pin input" message.*

2484 The sub-element "pin" MUST be set and interpreted as follows: The value range of the PIN is defined
 2485 by section 12.5. The format of the PIN within sub-element "pin" of "PIN input" message is the same
 2486 as the written format of section 12.5, but without any separators. Thus, the sub-element "pin" is a
 2487 string of 8-16 contiguous hexadecimal characters (as specified in section 12.5), i.e. the decimal
 2488 characters '0' to '9' and the Latin characters 'a' to 'f' and 'A' to 'F'. The verification of a PIN (i.e. the
 2489 value of element "pin") SHALL NOT be case-sensitive.

2490 Examples (informative): If the written PIN (for the user) is "12AB C34D" (without quotation marks),
 2491 the following strings show valid content for the element "pin": "12abc34d", "12ABc34d",
 2492 "12ABC34D", etc. (without quotation marks; examples not exhaustive).

2493 **SME "PIN error" Message:**

2494 The SME "PIN error" message is defined as follows:

2495 `MessageType = %x01 ; control`

2496 `MessageValue = SmeConnectionPinErrorValue`

2497 `SmeConnectionPinErrorValue = *OCTET`

2498 The content of `SmeConnectionPinErrorValue` is defined as follows: The structure is defined by the
 2499 SHIP root tag "connectionPinError" (including the root tag "connectionPinError") of the XSD
 2500 "SHIP_TS_TransferProtocol.xsd". The default structure extensibility applies to this structure. The
 2501 format of this structure MUST be the format agreed with the protocol handshake (see 13.4.4.2).

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
connectionPinError.error	M	Error number.

Table 18: Structure of SmeConnectionPinErrorValue of SME "Pin error" message.

13.4.4.3.3 Basic Rules

The device that owns (requires) a device PIN MUST NOT communicate this PIN to another device.

Only the other direction is allowed. These rules can be expressed briefly as follows, where node "A" owns a "PIN A" and another node "X" is the communication partner:

1. NOT ALLOWED: Send "PIN A" from left to right: Node A -----> Node X

2. ALLOWED: Send "PIN A" from right to left: Node A <----- Node X

Common security related rules on PINs (examples: which kind of device needs to have a PIN and request it from the communication partner; how is a PIN made available to a user; which conditions are defined to prevent "too simple PIN values"; how does a PIN value contribute to a trust level) are subject of chapter 12.

13.4.4.3.4 Protection Against Brute Force Attempts

Every verified and invalid PIN received from the communication partner is counted. If a node verifies a received PIN and declares it as invalid, it SHALL proceed as specified for the state SME_PIN_STATE_CHECK_ERROR AND impose a penalty to the communication partner according to the following rules:

1. If the number of counted invalid PINs is less than three, NO penalty is required.
2. If the number of counted invalid PINs ranges from three to five, the node SHALL apply a penalty as follows: The node SHALL enter the state SME_PIN_STATE_CHECK_BUSY_WAIT for a period of at least 10 seconds. This period SHOULD exceed 15 seconds only in case of increased security requirements.
3. If the number of counted invalid PINs is greater than five, the node SHALL apply a penalty as follows: The node SHALL enter the state SME_PIN_STATE_CHECK_BUSY_WAIT for a period of at least 60 seconds. This period SHOULD exceed 90 seconds only in case of increased security requirements.

In addition, the node SHALL implement countermeasures against attempts to bypass the aforementioned penalties. Among others, disconnecting the node or switching it off (regularly or suddenly, e.g. through power loss) SHALL NOT disable or weaken the penalties towards the communication partner that sent the invalid PIN. This requirement holds regardless of the number of communication partners a node is capable of distinguishing and (potentially) storing.

Remark (informative): The aforementioned countermeasures may lead to modified PIN requirements (see 13.4.4.3.5.1) after re-powering/re-connection. E.g. if a penalty towards a communication partner was not completed before the disconnection, the "inputPermission" SHOULD be "busy"

2535 towards the re-connected communication partner until the penalty has been completed. In fact, if a
2536 node is not capable of persistently storing all communication partners with unfinished penalty, it
2537 may impose a general penalty (i.e. regardless of the communication partner/connection that is
2538 currently established) until the general penalty was completed.

2539 *13.4.4.3.5 Process Details*

2540 *13.4.4.3.5.1 PIN Requirement - Communicated PIN States*

2541 The sub-element connectionPinState.pinState of the SME "PIN state" message conveys the PIN
2542 requirement towards the communication partner, i.e. the communicated PIN state. Permitted values
2543 are:

2544 1. **required**

2545 The node requires to receive its own valid PIN from the communication partner. The next state
2546 "Connection data exchange" will not be reached until a received PIN was verified successfully.

2547 Note: Setting "pinState" to "required" should only be done for certain cases! In general, a SHIP
2548 node cannot know whether the other SHIP node has a user interface or equivalent possibility for
2549 PIN input. If a manufacturer of a SHIP device decides to set "pinState" to "required", the
2550 manufacturer should also provide a SHIP-based commissioning tool with PIN input.

2551 2. **optional**

2552 The node does not require its own valid PIN from the communication partner, but restricts data
2553 exchange. It is possible to proceed with the next state "Connection data exchange" without the
2554 correct PIN; however, the node limits the data exchange to only those data that do not require
2555 the PIN. As soon as the communication partner submits the valid PIN, the node grants access to
2556 all data.

2557 Note: This means that the communication partner is not forced to submit a PIN. I.e. the node
2558 would keep its "pinState" value to "optional" and would communicate according to state
2559 "Connection data exchange" (with reduced data) – but in parallel it would continue listening for
2560 potential PIN messages from the communication partner for a "late release" of the data
2561 exchange restrictions.

2562 3. **pinOk**

2563 The node already received its own valid PIN from the communication partner and grants
2564 unrestricted data exchange. It is possible to proceed with the next state "Connection data
2565 exchange" immediately.

2566 4. **none**

2567 The node does not have an own PIN and grants unrestricted data exchange. It is possible to
2568 proceed with the next state "Connection data exchange" immediately.

2569 Note: The above mentioned state "Connection data exchange" requires further conditions to be
2570 enabled. This is described in detail in the subsequent sections.

2571 The sub-element connectionPinState.inputPermission of the SME "PIN state" message expresses
2572 whether the owner of the PIN currently accepts an SME "PIN input" message for verification or not.
2573 Permitted values are:

2574 1. **busy**

2575 The node does currently not accept an SME "PIN input" message for verification.

2576 2. **ok**

2577 The node currently accepts an SME "PIN input" message for verification.

2578 The following dependencies between "pinState" and "inputPermission" are defined:

2579 1. If the value of "pinState" is "pinOk" or "none", the sub-element "inputPermission" MUST NOT be
2580 present. This case means the node does not accept an SME "PIN input" message.

2581 2. If the value of "pinState" is "required" or "optional", the sub-element "inputPermission" MUST
2582 be present. The value of "inputPermission" MUST be "busy" as long as a penalty towards the
2583 communication partner is in place (see 13.4.4.3.4); otherwise, it MUST be "ok".

2584 *13.4.4.3.5.2 Process States*

2585 Broadly speaking, a node needs to

2586 1. report its PIN requirement to a communication partner,

2587 2. verify received PINs (provided a PIN is required or at least optional),

2588 3. await the communication partner's PIN requirement,

2589 4. send a PIN to the communication partner (provided it is required or at least optional and the
2590 node wants to obtain unrestricted data exchange).

2591 The first two items belong to the major state SME_PIN_STATE_CHECK whereas the remaining items
2592 belong to the major state SME_PIN_STATE_ASK. These major states are independent from each
2593 other and SHALL also be executed in parallel. If connection state "PIN verification" is entered, the
2594 first sub-state of SME_PIN_STATE_CHECK to execute is SME_PIN_STATE_CHECK_INIT and the first
2595 sub-state of SME_PIN_STATE_ASK to execute is SME_PIN_STATE_ASK_INIT.

2596 **Sub-state SME_PIN_STATE_CHECK_INIT:**

2597 In this state, the node SHALL perform the following steps:

2598 1. It SHALL execute the common procedure to send the PIN requirement with the current
2599 requirements it has towards the communication partner.

2600 2. If the node's "pinState" is "required":

2601 a. State "Connection data exchange" MUST be disabled (i.e. this state must not be executed).

- 2602 b. State SME_PIN_STATE_CHECK_BUSY_WAIT SHALL be entered if the sub-element
2603 "inputPermission" is set to "busy", otherwise state SME_PIN_STATE_CHECK_LISTEN SHALL be
2604 entered.
- 2605 3. If the node's "pinState" is "optional":
- 2606 a. The common procedure to enable the state "Connection data exchange" SHALL be executed.
- 2607 b. State SME_PIN_STATE_CHECK_BUSY_WAIT SHALL be entered if the sub-element
2608 "inputPermission" is set to "busy", otherwise state SME_PIN_STATE_CHECK_LISTEN SHALL be
2609 entered.
- 2610 4. If the node's "pinState" is "pinOk" or "none", the common procedure to enable the state
2611 "Connection data exchange" SHALL be executed and state SME_PIN_STATE_CHECK_OK SHALL be
2612 entered.
- 2613 **Sub-state SME_PIN_STATE_CHECK_LISTEN:**
- 2614 In this state, the SME User SHALL evaluate SHIP messages received from the communication partner.
2615 When this state is entered, the sub-element "inputPermission" MUST already be set to "ok".
- 2616 Only SME "PIN verification" messages are considered here. The following rules apply:
- 2617 1. If the received PIN matches the node's PIN:
- 2618 a. Set the node's "pinState" to "pinOk" and remove (disable) the sub-element
2619 "inputPermission".
- 2620 b. Execute the common procedure to send the PIN requirement.
- 2621 c. Execute the common procedure to enable the state "Connection data exchange".
- 2622 d. Enter the state SME_PIN_STATE_CHECK_OK.
- 2623 2. If the received PIN DOES NOT match the node's PIN:
- 2624 a. Enter state SME_PIN_STATE_CHECK_ERROR.
- 2625 **Sub-state SME_PIN_STATE_CHECK_ERROR:**
- 2626 In this state, the SME User informs the communication partner that a wrong PIN has been received.
2627 The following steps SHALL be performed:
- 2628 1. Increase the number of counted invalid PINs.
- 2629 2. Execute the common procedure to send an SME "PIN error" message with the error code for
2630 "wrong PIN".
- 2631 3. If the number of counted invalid PINs DOES NOT require imposing a penalty according to the
2632 rules of section 13.4.4.3.4: Enter state SME_PIN_STATE_CHECK_LISTEN.
- 2633 4. If the number of counted invalid PINs DOES require imposing a penalty according to the rules of
2634 section 13.4.4.3.4: Enter state SME_PIN_STATE_CHECK_BUSY_INIT.

2635 Sub-state SME_PIN_STATE_CHECK_BUSY_INIT:

2636 In this state, the SME User prepares a penalty (see 13.4.4.3.4). The following steps SHALL be
2637 performed:

- 2638 1. Set the node's sub-element "inputPermission" to "busy".
- 2639 2. Execute the common procedure to send the PIN requirement.
- 2640 3. Enter state SME_PIN_STATE_CHECK_BUSY_WAIT.

2641 Sub-state SME_PIN_STATE_CHECK_BUSY_WAIT:

2642 In this state, the SME User shall impose a penalty according to 13.4.4.3.4. This means that the SME
2643 User remains in this state until the end of the penalty's duration. When this state is entered, the sub-
2644 element "inputPermission" MUST already be set to "busy". The following steps SHALL be performed:

- 2645 1. As long as the duration of the penalty has not expired:
 - 2646 a. For every received SME "PIN input" message, execute the common procedure to send the
2647 PIN requirement.
- 2648 2. As soon as the duration of the penalty expires:
 - 2649 a. Set the node's sub-element "inputPermission" to "ok".
 - 2650 b. Execute the common procedure to send the PIN requirement.
 - 2651 c. Enter state SME_PIN_STATE_CHECK_LISTEN.

2652 Note: In this state, received PINs are NOT evaluated and are NOT counted.

2653 Sub-state SME_PIN_STATE_CHECK_OK:

2654 The SME User SHALL silently discard any SME "PIN input" message.

2655 Note: The state branch "SME_PIN_STATE_CHECK" ends here, i.e. no further action is required.
2656 Whether a next state is enabled is not determined here. Instead, this is determined where the
2657 common procedure to enable the state "Connection data exchange" is referenced.

2658 Sub-state SME_PIN_STATE_ASK_INIT:

2659 In this state, the SME User SHALL wait for the receipt of an SME "PIN state" message before any
2660 other action of this state is performed. Until then, no other message of connection state "PIN
2661 verification" SHALL be evaluated. Afterwards, the SME User SHALL enter state
2662 SME_PIN_STATE_ASK_PROCESS.

2663 However, the SME User SHALL close the connection if this state (SME_PIN_STATE_ASK_INIT) lasts
2664 more than 10 seconds.

2665 Sub-state SME_PIN_STATE_ASK_PROCESS:

2666 In this state, the SME User evaluates and processes received messages according to the following
2667 rules:

- 2668 1. The SME User SHALL close the connection if any of the rules of section 13.4.4.3.5.1 on the sub-
2669 elements of a received message are not fulfilled.
- 2670 2. A received SME "PIN error" message with the sub-element "error" set to the value for "wrong
2671 PIN" SHALL be interpreted by the SME User as follows: An SME "PIN input" message previously
2672 submitted to the communication partner contained the wrong PIN. The SME User SHOULD wait
2673 for a new SME "PIN state" message.
- 2674 3. For every received SME "PIN state" message with sub-element "pinState" set to a value that is
2675 NOT "required", the SME User SHALL execute the common procedure to enable the state
2676 "Connection data exchange".
- 2677 4. A received SME "PIN state" message with the sub-element "inputPermission" present and set to
2678 "busy" SHALL be interpreted by the SME User as follows: The communication partner is currently
2679 not ready to evaluate any SME "PIN input" message. The node SHOULD wait until the
2680 communication partner indicates being ready for an SME "PIN input" message. However, the
2681 node CAN send an SME "PIN input" message before the communication partner indicates being
2682 ready for this message.
- 2683 Remark (informative): The latter case (sending a new PIN before the communication partner
2684 indicates being ready) only makes sense to get a confirmation of the state after an "unexpectedly
2685 long" period of the assumed penalty.
- 2686 5. A received SME "PIN state" message with the sub-element "inputPermission" present and set to
2687 "ok" SHALL be interpreted by the SME User as follows: The communication partner is currently
2688 ready to evaluate an SME "PIN state" message. The node SHALL also evaluate sub-element
2689 "pinState" and decide whether to send an SME "PIN input" message or not.
- 2690 6. A received SME "PIN state" message with the sub-element "pinState" set to "required" or
2691 "optional" SHALL be interpreted and processed by the SME User as follows: The node SHALL also
2692 evaluate sub-element "inputPermission" and decide whether to send an SME "PIN input"
2693 message or not.
- 2694 7. A received SME "PIN state" message with the sub-element "pinState" set to "pinOk" or "none"
2695 SHALL be interpreted and processed by the SME User as follows: The node SHALL enter the state
2696 SME_PIN_STATE_ASK_OK.
- 2697 8. If a node needs to decide whether to send an SME "PIN input" message or not, the following
2698 rules apply:
- 2699 a. If a received "pinState" value is set to "required", the SME User SHALL
- 2700 i. EITHER execute the common procedure to send an SME "Pin input" message (provided
2701 that sub-element "inputPermission" is set to "ok" and the SME User has a PIN to send)
- 2702 ii. OR close the connection.
- 2703 b. If a received "pinState" value is set to "optional", the SME User SHALL

- 2704 i. EITHER enter state SME_PIN_STATE_ASK_RESTRICTED_OK (i.e. the SME User does not
2705 require unrestricted data exchange with the communication partner)
- 2706 ii. OR execute the common procedure to send an SME "Pin input" message (provided that
2707 sub-element "inputPermission" is set to "ok" and the SME User has a PIN to send)
- 2708 iii. OR close the connection.

2709 9. If a received SME "PIN state" message had "pinState" value set to "required" or "optional" and
2710 the SME User sent an SME "PIN input" message, the SME User SHALL wait for a new SME "PIN
2711 state" message for at least 30 seconds and at most 120 seconds before deciding to continue in
2712 this state or close the connection.

2713 Note: This rule is independent from decisions on a retry to send a PIN. It just focuses on the lack
2714 of a "PIN state" update (esp. with "inputPermission" either "busy" or "ok") from the
2715 communication partner.

2716 10. This state remains enabled unless stated otherwise. This also means that the SME User will
2717 continue listening for incoming messages as described above.

2718 **Sub-state SME_PIN_STATE_ASK_RESTRICTED_OK:**

2719 The SME User SHALL silently discard any SME "PIN error" message. It SHALL keep a "PIN state"
2720 message (only the latest message is required; this message SHALL NOT be kept in case a connection
2721 is closed).

2722 Note: The state branch "SME_PIN_STATE_ASK" ends here, i.e. no further action is required. Whether
2723 a next state is enabled is not determined here. Instead, this is determined where the common
2724 procedure to enable the state "Connection data exchange" is referenced.

2725 However, if an SME User wants to submit a PIN on a later occasion, it can take the last SME "PIN
2726 state" message and enter state SME_PIN_STATE_ASK_PROCESS.

2727 **Sub-state SME_PIN_STATE_ASK_OK:**

2728 The SME User SHALL silently discard any SME "PIN error" or "PIN state" message.

2729 Note: The state branch "SME_PIN_STATE_ASK" ends here, i.e. no further action is required. Whether
2730 a next state is enabled is not determined here. Instead, this is determined where the common
2731 procedure to enable the state "Connection data exchange" is referenced.

2732 **Common Procedure to Send the PIN Requirement:**

2733 This procedure SHALL be executed where referenced.

2734 The SME User SHALL send an SME "PIN state" message with sub-element "pinState" set according to
2735 the node's PIN requirement towards the communication partner (see 13.4.4.3.5.1). The sub-element
2736 "inputPermission" SHALL be omitted or set as required and according to the rules expressed in
2737 13.4.4.3.5.1.

2738 **Common Procedure to Send an SME "PIN error" Message:**

2739 This procedure SHALL be executed where referenced.

2740 The SME User SHALL send an SME "PIN error" message with sub-element "error" set to the
 2741 appropriate value (see Table 19).

Value	Error type
0	RFU
1	wrong PIN
4-255	RFU

2742 *Table 19: Values of Sub-element "error" of connectionPinError.*

2743 **Common procedure to Send an SME "PIN input" message:**

2744 This procedure SHALL be executed where referenced.

2745 The SME User SHALL send an SME "PIN input" message with sub-element "pin" set to the value
 2746 required by the communication partner.

2747 **Common procedure to Enable the State "Connection data exchange":**

2748 This procedure SHALL be executed where referenced.

2749 The SME User SHALL enable the state "Connection data exchange" if and only if all of the following
 2750 rules are fulfilled:

- 2751 1. The node's own PIN requirement element "pinState" towards the communication partner is NOT
 2752 "required".
- 2753 2. The communication partner's PIN requirement is available AND its element "pinState" is available
 2754 AND is NOT "required".

2755 Enabling the state "Connection data exchange" means this state shall be executed regardless of any
 2756 (parallel) states of "Connection state PIN verification". Depending on the element "pinState", the
 2757 SME User SHALL also adjust the restriction of data exchange as mentioned in 13.4.4.3.5.1
 2758 accordingly.

2759 A brief overview of the PIN verification procedure is given in the following sequence diagrams.

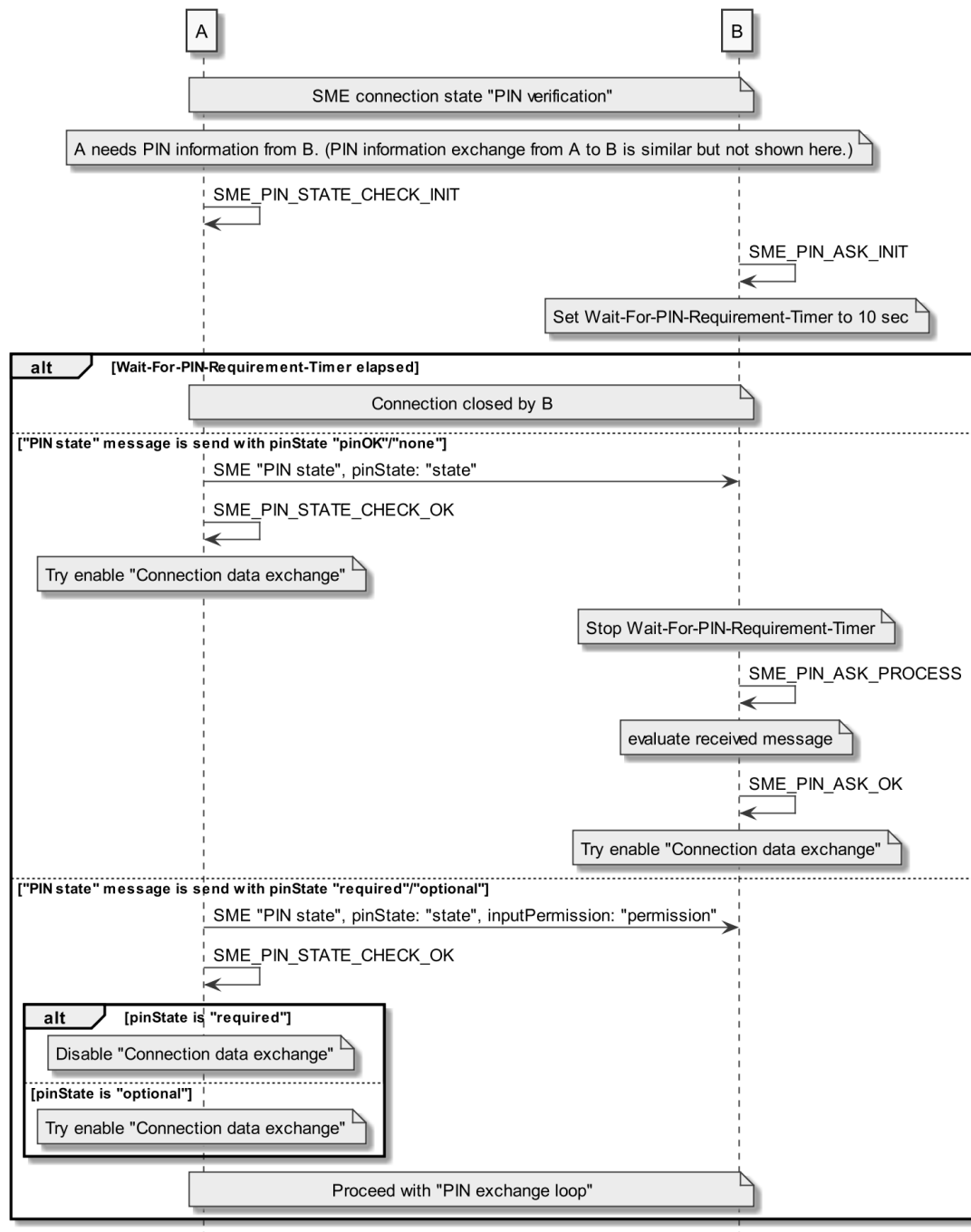


Figure 14: Connection State "PIN verification" Message Sequence Example (Begin)

13.4.5 Connection Data Exchange

13.4.5.1 General Rules

Specification Versions and "Base Specification"

A specification may exist in multiple versions. The first version is called "base specification" and denotes the unique origin of the other versions.

General Compatibility Rules for a Given Specification Sequence

Subsequently, the term "content" denotes a data instance that matches a given specification. The compatibility of specifications is immediately related to the question how "content" of different

2770 specification versions needs to be processed. Specification versions are compatible with each other if
 2771 the following rules apply:

- 2772 1. The base specification must provide rules for forward compatibility. This means it must define
 2773 how any succeeding version can define additional content without breaking compatibility with
 2774 existing implementations: An implementation that is based on the base specification must be
 2775 able to gracefully accept and process "content" that is based on a compatible successor of the
 2776 base specification. The processing of "content" SHALL include at least the parts that are already
 2777 defined in the base specification. It MAY well skip the unknown parts.
- 2778 2. Each compatible successor of the base specification must provide rules for forward compatibility
 2779 as well. These rules MUST NOT break the compatibility rules of the base specification.
- 2780 3. Each version of a specification has zero or one immediate successors. No more immediate
 2781 successors are permitted.
- 2782 4. Each version of a specification has zero or one immediate predecessors. No more immediate
 2783 predecessors are permitted.
- 2784 5. An implementation that is based on a given version of the specification must be able to gracefully
 2785 accept and process "content" that is based on a compatible predecessor of the version. The
 2786 processing of "content" SHALL include the whole content.

2787 **13.4.5.2 Message "data"**

2788 *13.4.5.2.1 Purpose and Structure*

2789 The element "data" of the XSD "SHIP_TS_TransferProtocol.xsd" is used to exchange higher level
 2790 protocol data (e.g. SPINE) between two SHIP nodes. The structure is briefly described in Table 20.
 2791 Details on the elements are given in subsequent sections.

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
data. header	M	See 13.4.5.2.3.
data. header. protocolId	M	Identifies how "payload" MUST be evaluated, see 13.4.5.2.4.
data. payload	M	Contains data of the protocol stated in the element protocolId (see 13.4.5.2.5).
data. extension	O	Parent element for manufacturer specific extensions, see 13.4.5.2.6.
data. extension. extensionId	O	Identifier for content of elements "binary", "string".
data. extension. binary	O	Binary data.
data. extension. string	O	Textual data.

2792 *Table 20: Structure of MessageValue of "data" Message.*

2793 The complete "data" message is defined as follows:

2794 MessageTypes = %x02 ; data

2795 MessageValue = DataValue

2796 DataValue = *OCTET

2797 The content of DataValue is defined as follows: The structure is defined by the SHIP root tag "data"
2798 (including the root element "data"). The default structure extensibility applies to this structure. The
2799 format of this structure MUST be the format agreed with the protocol handshake (see 13.4.4.2).

2800 *13.4.5.2.2 Extensibility Rules*

2801 The "default structure extensibility" applies for these parts:

- 2802 1. The first level of "data".
- 2803 2. The element data.header (recursive).
- 2804 3. The first level of "data.extension".

2805 *13.4.5.2.3 Element "header"*

2806 This element serves as header for the remaining content of element "data". The sender of a message
2807 MUST set all information as described in the table and as follows.

2808 *13.4.5.2.4 Element "protocolId"*

2809 **Introduction (Informative)**

2810 This element announces how the content of "payload" MUST be evaluated. Within a software
2811 implementation, it permits to select a specific parser esp. in case of potentially conflicting
2812 specifications. This applies even in case of potential future binary formats.

2813 **Rules on protocolId**

- 2814 1. Permitted values for protocolId are defined by the SHIP specification authority only.
- 2815 2. The value always denotes the "base specification" for the content of "payload", even if the
2816 content is based on a newer compatible version of the specification.
- 2817 3. A recipient that encounters an unknown value of protocolId SHALL silently skip the received
2818 message.

2819 Note: Only those specifications can be assigned a value for protocolId that fulfil further requirements
2820 demanded by the SHIP specification.

2821 *13.4.5.2.5 Element "payload"*

2822 This element takes content that MUST match a compatible version of the base specification as
2823 determined by the value of protocolId. The content of payload MUST itself permit to evaluate it
2824 properly with regards to the specification sequence determined by the value of protocolId.

2825 The extensibility rule of payload is determined by the authority of the base specification that is given
2826 by the value of protocolId. For content defined by the SHIP specification, the rule "default structure
2827 extensibility (recursive)" applies.

2828 **Remarks (Informative)**

2829 1. Typically, this means the content should start with a "unique" root tag (or "unique" type or data
2830 identification in case of a binary format). Together with the value of protocolId, it is then possible
2831 to identify which compatible definitions of the proper specification sequence apply. This also
2832 means that the content of payload and protocolId must be sufficient to identify the definition of
2833 the content. I.e. no further information (as from device discovery, e.g.) is required to know which
2834 definition applies.

2835 2. Please also note these rules only address the identification of the definition. They do NOT
2836 address questions on the purpose of the content (i.e. questions on specific processes or contexts
2837 that are related to the specific content).

2838 3. The development of a specification sometimes also includes the definition of new types or data
2839 (i.e. content) for a new version of the specification. Nevertheless, only the base specification
2840 needs to be referenced in protocolId as new definitions (rather than modifications of existing
2841 definitions) just extend a specification. However, such extensions must also be done according to
2842 all compatibility rules. This requires special care especially in case of binary formats.

2843 4. The SME Protocol Handshake includes an agreement of the SME message format to be used
2844 between two communication partners. This format applies to the SHIP "payload" element and its
2845 content as well. However, it does NOT determine which specific types or conversion rules have
2846 to be applied for the protocolId specific content of "payload".
2847 To give an example: The handshake may end in JSON-UTF16 as format, for example. In this case,
2848 the complete native SHIP MessageValue for "Data" exchange (see section 13.4.5.2.1) must be
2849 formatted in JSON-UTF16. This includes the first "data" key and also the "payload" key, among
2850 others. The value of "payload" must as well be formatted in JSON-UTF16. However, the SHIP
2851 specification DOES NOT rule which JSON type has to be used as value type of "payload". For an
2852 assumed protocolId "abc", there may be a specific transformation rule to use a JSON string in any
2853 case. For an assumed protocolId "def", it may be a JSON object, e.g. Finally, this means that it is
2854 recommended to consider "payload" as an opaque – but well-formatted – container.

2855 *13.4.5.2.6 Element "extension"*

2856 This element can be used to extend content from payload with manufacturer specific data. The sub-
2857 element "extensionId" MAY be used by a manufacturer to identify the kind of content in the
2858 (optional) sub-elements "binary" and "string". However, the use of these elements is manufacturer-
2859 specific and not detailed further in this specification.

2860 **13.4.6 Access Methods Identification**

2861 **13.4.6.1 Introduction**

2862 This section discusses a possibility for the device that is currently the connection server to fetch
2863 information from the current connection client for a potential "reverse re-connection". The following
2864 example shall explain this briefly.

2865 Subsequently, we assume SHIP device "A" got information on how to connect another SHIP device
2866 "B" (e.g. from service discovery), but has not had any connection with device "B" before. This means
2867 device "A" finally has the IP address and port number of device "B" and establishes a connection for
2868 the first time. Device "A" is the connection client and device "B" the connection server. Any intended
2869 connection termination processes or sudden interrupts are no problem for a reconnection as long as
2870 device "A" initiates the reconnection again. However, there may be cases where it is favoured or
2871 even required that device "B" initiates a connection to device "A" under certain circumstances. In
2872 such cases, there is a need for device "B" to have proper information on how to find and connect to
2873 device "A". In general, this cannot be derived from device "A"'s socket of the first connection (i.e.
2874 where device "A" is a connection client), as server and client sockets typically differ. Furthermore, IP
2875 addresses of devices change in many situations.

2876 To overcome this situation, the section describes a method to query device "A" for its access
2877 methods.

2878 **13.4.6.2 Basic Definitions**

2879 In this section, an SME User can request the "access methods" of the communication partner.
2880 However, the support of this methodology is not mandatory in all cases. Details on the support are
2881 explained in subsequent sections.

2882 The state "Access Methods Identification" can run in parallel to connection data exchange. In fact,
2883 this state MUST NOT be entered before connection data exchange is entered (i.e. the "access
2884 methods" exchange does NOT apply for earlier states like "hello" or "CMI", e.g.).

2885 **SME "Access methods request" Message:**

2886 The SME "Access methods request" message is defined as follows:

2887 `MessageType = %x01 ; control`

2888 `MessageValue = SmeConnectionAccessMethodsRequestValue`

2889 `SmeConnectionAccessMethodsRequestValue = *OCTET`

2890 The content of SmeConnectionAccessMethodsRequestValue is defined as follows: The structure is
2891 defined by the SHIP root tag "accessMethodsRequest" (including the root tag
2892 "accessMethodsRequest") of the XSD "SHIP_TS_TransferProtocol.xsd". The default structure
2893 extensibility applies to this structure. The format of this structure MUST be the format agreed with
2894 the protocol handshake (see 13.4.4.2).

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
accessMethodsRequest	M	The request for the recipient's access methods.
		Note: Subsequent versions of this specification may define (optional) sub-elements of accessMethodsRequest.

2895 Table 21: Structure of SmeConnectionAccessMethodsRequestValue of SME "Access methods request" message.

2896 **SME "Access methods" Message:**

2897 The SME "Access methods" message is defined as follows:

2898 MessageType = %x01 ; control

2899 MessageValue = SmeConnectionAccessMethodsValue

2900 SmeConnectionAccessMethodsValue = *OCTET

2901 The content of SmeConnectionAccessMethodsValue is defined as follows: The structure is defined by
 2902 the SHIP root tag "accessMethods" (including the root tag "accessMethods") of the XSD
 2903 "SHIP_TS_TransferProtocol.xsd". The default structure extensibility applies to this structure. The
 2904 format of this structure MUST be the format agreed with the protocol handshake (see 13.4.4.2).

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
accessMethods	M	The originator's access methods.
accessMethods.id	M	A SHIP node that supports SHIP version 1.1 or newer SHALL set this element to the unique SHIP ID. A SHIP node that just supports a SHIP version lower than 1.1 SHALL set this element to an empty string if and only if it has no unique SHIP ID. Otherwise, it SHALL set this element to the unique SHIP ID.
accessMethods.dnsSd_mDns	O	SHALL be present if the originator provides its SHIP service via service discovery as specified in chapter 7. Please note that this REQUIRES that the originator has a unique ID and consequently the element "accessMethods.id" MUST contain this value. Note: Subsequent versions of this specification may define (optional) sub-

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
		elements of accessMethodsRequest.dnsSd_mDns.
accessMethods.dns	O	SHALL be present if the originator provides its SHIP service with unicast DNS.
accessMethods.dns.uri	M	The URI where the originator provides its SHIP service. Please see also constraints defined in the text.

2905 *Table 22: Structure of SmeConnectionAccessMethodsValue of SME "Access methods" message.*

2906 The element "accessMethods.dns.uri" can take a URI as specified by IETF RFC 7320. However, this
2907 version of the SHIP specification considers only the URI scheme "wss" as used by WebSockets and
2908 defined by IETF RFC 6455.

2909 **Roles and Symbols:**

2910 Either side can request for the communication partner's "access methods" information. However,
2911 obligations on support and information differ depending on the role.

2912 Subsequently, the following symbols are used:

2913 1. **DEV-SERVER**

2914 This symbol is used for a device with the connection role "server".

2915 2. **DEV-CLIENT**

2916 This symbol is used for a device with the connection role "client".

2917 Please note the roles "server" and "client" denote only an aspect of the connection. They DO NOT
2918 denote an aspect of a specific functionality.

2919 Further symbols for two devices "A" and "B" will be defined in section 13.4.8.1.1.

2920 *13.4.6.2.1 Process Details*

2921 There is no specific requirement in which case an SME "Access methods request" message needs to
2922 be sent. However, please consider the recommendations in section 13.4.6.2.2.

2923 The recipient of an SME "Access methods request" message SHALL respond with an SME "Access
2924 methods" message. The sender of the aforementioned SME "Access methods request" message CAN
2925 close the connection according to section 13.4.8 if it does not receive a proper SME "Access
2926 methods" message within 60 seconds.

2927 Unsolicited SME "Access methods" message SHALL NOT be sent. I.e. it SHALL ONLY be sent upon a
2928 received SME "Access methods request".

2929 The recipient of an SME "Access methods" message SHALL store the received information
2930 persistently for cases where it needs to initiate a connection to the originator of the message.

2931 **13.4.6.2.2 Recommendations**

2932 The device DEV-SERVER SHOULD request for DEV-CLIENT's access methods if DEV-SERVER must be
2933 able to initiate a connection to the current DEV-CLIENT under certain circumstances but has no
2934 proper information so far.

2935 Note: This specification does not describe binding or subscription processes. However, such use
2936 cases are typical for higher layers. Imagine device "A" is DEV-CLIENT and connects to device "B" (DEV-
2937 SERVER) and subscribes to some data provided by device "B". I.e. it asks device "B" to submit data
2938 changes to device "A". Such cases typically intend that device "B" is also able to establish a
2939 connection to device "A". Thus, device "B" SHOULD request for device "A"'s "Access methods"
2940 information as long as device "B" is DEV-SERVER.

2941 **13.4.7 Key material exchange**

2942 **13.4.7.1 Introduction**

2943 SME key material exchange allows to exchange trustworthy key material between SHIP nodes.

2944 This enables

- 2945 a) an easy and user-friendly secure mechanism for trust establishment over a SHIP
- 2946 commissioning tool, as described in sections 12.3.1.6 and 12.6.
- 2947 b) seamless certificate updates without user interaction, as described in section 12.1.3.

2948 Key material exchange messages can be sent parallel to connection data exchange. However, key
2949 material exchange messages SHALL NOT be sent before connection data exchange is entered.

2950 **13.4.7.2 SME "commissioning request" Message**

2951 A SHIP commissioning tool SHALL support sending "commissioning requests".

2952 A SHIP node that supports "SHIP commissioning" SHALL support receiving "commissioning request".

2953 The SME "commissioning request" message is defined as follows:

2954 `MessageType = %x01 ; control`

2955 `MessageValue = SmeConnectionCommissioningRequestValue`

2956 `SmeConnectionCommissioningRequestValue = *OCTET`

2957 The content of SmeConnectionCommissioningRequestValue is defined as follows: The structure is
2958 defined by the SHIP root tag "commissioningRequest" (including the root tag
2959 "commissioningRequest") of the XSD "SHIP_TS_TransferProtocol.xsd". The default structure
2960 extensibility applies to this structure. The format of this structure MUST be the format agreed with
2961 the protocol handshake (see 13.4.4.2).

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation

commissioningRequest	M	Request to gain the commissioning right
		Note: Subsequent versions of this specification may define (optional) sub-elements of commissioningRequest.

2962 Table 23: Structure of SmeConnectionCommissioningRequestValue of SME "commissioning request" message.

2963 13.4.7.3 SME "commissioning response" Message

2964 A SHIP node that supports "SHIP commissioning" SHALL support sending "commissioning response".

2965 A SHIP commissioning tool SHALL support receiving "commissioning response".

2966 The SME "commissioning response" message is defined as follows:

2967 MessageType = %x01 ; control

2968 MessageValue = SmeConnectionCommissioningResponseValue

2969 SmeConnectionCommissioningResponseValue = *OCTET

2970 The content of SmeConnectionCommissioningResponseValue is defined as follows: The structure is
 2971 defined by the SHIP root tag "commissioningResponse" (including the root tag
 2972 "commissioningResponse") of the XSD "SHIP_TS_TransferProtocol.xsd". The default structure
 2973 extensibility applies to this structure. The format of this structure MUST be the format agreed with
 2974 the protocol handshake (see 13.4.4.2).

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
commissioningResponse	M	Response to grant or deny commissioning right to requester
commissioningResponse.accept	M	If true, the requester is accepted as commissioning tool. If false, the requester is not accepted as commissioning tool

2975 Table 24: Structure of SmeConnectionCommissioningResponseValue of SME "commissioning response" message.

2976 13.4.7.4 SME "key material request" Message

2977 The following SHIP nodes SHALL support receiving "key material request":

- 2978 - SHIP commissioning tools
- 2979 - SHIP nodes that support SHIP commissioning

2980 The following SHIP nodes SHALL support sending "key material request":

- 2981 - SHIP commissioning tools

2982 - SHIP nodes that support SHIP commissioning

2983 The SME "key material request" message is defined as follows:

2984 Message`Type` = %x01 ; control

2985 Message`Value` = SmeConnectionKeyMaterialRequestValue

2986 SmeConnectionKeyMaterialRequestValue = *OCTET

2987 The content of SmeConnectionKeyMaterialRequestValue is defined as follows: The structure is
 2988 defined by the SHIP root tag "keyMaterialRequest" (including the root tag "keyMaterialRequest") of
 2989 the XSD "SHIP_TS_TransferProtocol.xsd". The default structure extensibility applies to this structure.
 2990 The format of this structure MUST be the format agreed with the protocol handshake (see 13.4.4.2).

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
keyMaterialRequest	M	The request for trustworthy key material in possession of the recipient.
		Note: Subsequent versions of this specification may define (optional) sub-elements of keyMaterialRequest.

2991 *Table 25: Structure of SmeConnectionKeyMaterialRequestValue of SME "key material request" message.*

2992 **13.4.7.5 SME "key material" Message**

2993 The following SHIP nodes SHALL support receiving "key material":

2994 - SHIP commissioning tools

2995 - SHIP nodes that support SHIP commissioning

2996 The following SHIP nodes SHALL support sending "key material":

2997 - SHIP commissioning tools

2998 - SHIP nodes that support SHIP commissioning

2999 The SME "key material" message is defined as follows:

3000 Message`Type` = %x01 ; control

3001 Message`Value` = SmeConnectionKeyMaterialValue

3002 SmeConnectionKeyMaterialValue = *OCTET

3003 The content of SmeConnectionKeyMaterialValue is defined as follows: The structure is defined by the
 3004 SHIP root tag "keyMaterial" (including the root tag "keyMaterial") of the XSD
 3005 "SHIP_TS_TransferProtocol.xsd". The default structure extensibility applies to this structure. The
 3006 format of this structure MUST be the format agreed with the protocol handshake (see 13.4.4.2).

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
keyMaterial	M	Trustworthy key material
keyMaterial.maxEntries	M/O ^{*1}	The maximum number of "entry" items in "keyMaterial" that the sender of this message can store at maximum in its own memory. (*1): Mandatory if "keyMaterial" is sent from a SHIP node that supports SHIP commissioning. Optional otherwise.
keyMaterial.entry	0..	Includes a list of trustworthy SKIs and their corresponding trust level
keyMaterial.entry.curve	M	Contains number of enumeration defined by IANA TLS Supported Groups. 23 = secp256r1 26 = brainpoolP256r1 27 = brainpoolP384r1
keyMaterial.entry.ski	M	Subject Key Identifier of a SHIP node SHALL be provided as <xs:hexBinary>.
keyMaterial.entry.publicKey	O	Beside the SKI also the corresponding public key MAY be provided as <xs:hexBinary>. The public key SHALL match with the SKI.
keyMaterial.entry.userTrust	M	User trust of the corresponding SKI. userTrust value SHALL be greater than 0. To untrust SKIs, the keyMaterialDelete message is used.
keyMaterial.entry.commissioningTool	O	SHALL be used and set to "true" by a SHIP commissioning tool to introduce an additional SHIP commissioning tool to a SHIP node. SHALL be ignored if not received from a SHIP commissioning tool.
keyMaterial.entry.id	M	SHALL include the SHIP ID of the corresponding key material.
keyMaterial.entry.note	O	This field MAY be present to communicate further information of the key material. SHALL NOT exceed 256 characters in length.

Table 26: Structure of SmeConnectionKeyMaterialValue of SME "key material" message.

13.4.7.6 SME "key material response" Message

The following SHIP nodes SHALL support receiving "key material response":

- SHIP commissioning tools
- SHIP nodes that support SHIP commissioning

3012 SHIP nodes SHALL support sending "key material response":

3013 - if they support receiving "key material"

3014 The SME "key material response" message is defined as follows:

3015 `MessageType = %x01 ; control`

3016 `MessageValue = SmeConnectionKeyMaterialResponseValue`

3017 `SmeConnectionKeyMaterialResponseValue = *OCTET`

3018 The content of `SmeConnectionKeyMaterialResponseValue` is defined as follows: The structure is
 3019 defined by the SHIP root tag "keyMaterialResponse" (including the root tag "keyMaterialResponse")
 3020 of the XSD "SHIP_TS_TransferProtocol.xsd". The default structure extensibility applies to this
 3021 structure. The format of this structure MUST be the format agreed with the protocol handshake (see
 3022 13.4.4.2).

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
keyMaterialResponse	M	Received key material.
keyMaterialResponse.entry	0..	Includes a list of SKIs and their corresponding trust level
keyMaterialResponse.entry.curve	M	Contains number of enumeration defined by IANA TLS Supported Groups. 23 = secp256r1 26 = brainpoolP256r1 27 = brainpoolP384r1
keyMaterialResponse.entry.ski	M	Subject Key Identifier of a SHIP node
keyMaterialResponse.entry.public Key	O	Beside the SKI also the corresponding public key MAY be provided as <xs:hexBinary>. The public key SHALL match with the SKI.
keyMaterialResponse.entry.userTrust	M	User trust information of the SKI at the sender of this message.
keyMaterialResponse.entry.id	O	Includes the SHIP ID of the corresponding key material.
keyMaterialResponse.entry.note	O	This field MAY be present to communicate certain special conditions, e.g. failed optional certificate checks to the user in a user friendly readable way. SHALL NOT exceed 256 characters in length.

3023 *Table 27: Structure of `SmeConnectionKeyMaterialResponseValue` of SME "key material response" message.*

3024 **13.4.7.7 SME "key material delete" Message**

3025 The following SHIP nodes SHALL support receiving "key material delete":

- 3026 - SHIP nodes that support SHIP commissioning

3027 The following SHIP nodes SHALL support sending "key material delete":

- 3028 - SHIP commissioning tools

3029 The SME "key material delete" message is defined as follows:

3030 `MessageType = %x01 ; control`

3031 `MessageValue = SmeConnectionKeyMaterialDeleteValue`

3032 `SmeConnectionKeyMaterialDeleteValue = *OCTET`

3033 The content of SmeConnectionKeyMaterialDeleteValue is defined as follows: The structure is defined
 3034 by the SHIP root tag "keyMaterialDelete" (including the root tag "keyMaterialDelete") of the XSD
 3035 "SHIP_TS_TransferProtocol.xsd". The default structure extensibility applies to this structure. The
 3036 format of this structure MUST be the format agreed with the protocol handshake (see 13.4.4.2).

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
keyMaterialDelete	M	Key material that is to be deleted at the recipient of this message.
keyMaterialDelete.entry	1..	Includes list of deleted keys
keyMaterialDelete.entry.curve	M	Contains number of enumeration defined by IANA TLS Supported Groups. 23 = secp256r1 26 = brainpoolP256r1 27 = brainpoolP384r1
keyMaterialDelete.entry.ski	M	Subject Key Identifier of a SHIP node SHALL be provided as <xs:hexBinary>.
keyMaterialDelete.entry.note	O	SHALL NOT exceed 256 characters in length.

3037 *Table 28: Structure of SmeConnectionKeyMaterialDeleteValue of SME "key material delete" message.*

3038

3039 **13.4.7.8 SME "key material delete response" Message**

3040 The following SHIP nodes SHALL support receiving "key material delete response":

- 3041 - SHIP commissioning tools

3042 SHIP nodes SHALL support sending "key material delete response":

3043 - if they support receiving "key material delete"

3044 The SME "key material delete response" message is defined as follows:

3045 Message_{Type} = %x01 ; control

3046 Message_{Value} = SmeConnectionKeyMaterialDeleteResponseValue

3047 SmeConnectionKeyMaterialDeleteResponseValue = *OCTET

3048 The content of SmeConnectionKeyMaterialDeleteResponseValue is defined as follows: The structure
 3049 is defined by the SHIP root tag "keyMaterialDeleteResponse" (including the root tag
 3050 "keyMaterialDeleteResponse") of the XSD "SHIP_TS_TransferProtocol.xsd". The default structure
 3051 extensibility applies to this structure. The format of this structure MUST be the format agreed with
 3052 the protocol handshake (see 13.4.4.2).

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
keyMaterialDeleteResponse	M	Implemented key material deletion.
keyMaterialDeleteResponse.entry	1..	Includes a list of SKIs and their corresponding trust level
keyMaterialDeleteResponse.entry. curve	M	Contains number of enumeration defined by IANA TLS Supported Groups. 23 = secp256r1 26 = brainpoolP256r1 27 = brainpoolP384r1
keyMaterialDeleteResponse.entry. ski	M	Subject Key Identifier of a SHIP node SHALL be provided as <xs:hexBinary>.
keyMaterialDeleteResponse.entry. publicKey	O	Beside the SKI also the corresponding public key MAY be provided as <xs:hexBinary>. The public key SHALL match with the SKI,
keyMaterialDeleteResponse.entry. userTrust	M	SHALL be set to 0.
keyMaterialDeleteResponse.entry. id	O	Includes the SHIP ID of the corresponding key material.
keyMaterialDeleteResponse.entry. note	O	SHALL NOT exceed 256 characters in length.

3053 *Table 29: Structure of SmeConnectionKeyMaterialDeleteResponseValue of SME "key material delete response" message.*

3054

3055 **13.4.7.9 SME "key material state" message**

3056 The following SHIP nodes SHALL support receiving and sending SME "key material state":

3057 - SHIP nodes with SHIP version $\geq$ SHIP 1.1

3058 The SME "key material state" message is defined as follows:

3059 `MessageType = %x01 ; control`

3060 `MessageValue = SmeConnectionKeyMaterialStateValue`

3061 `SmeConnectionKeyMaterialStateValue = *OCTET`

3062 The content of SmeConnectionKeyMaterialStateValue is defined as follows: The structure is defined
 3063 by the SHIP root tag "keyMaterialState" (including the root tag "keyMaterialState") of the XSD
 3064 "SHIP_TS_TransferProtocol.xsd". The default structure extensibility applies to this structure. The
 3065 format of this structure MUST be the format agreed with the protocol handshake (see 13.4.4.2).

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
keyMaterialState	M	Trustworthy key material of the originator (with the respective SHIP ID stated in element keyMaterialState.entry.id).
keyMaterialState.updateCounter	M (0..65535)	Counter which starts at 1 (not 0) in a device factory new condition. Every time any certificate is changed or added during the lifetime of the device, this counter is increased by 1.
keyMaterialState.entry	1..256	Includes list of SKIs owned by the sender
keyMaterialState.entry.currentState	M [predecessor current successor]	Denotes whether this entry is the currently used key material, a predecessor which is now unused or a successor that will soon be used.
keyMaterialState.entry.curve	M	Contains number of enumeration defined by IANA TLS Supported Groups. 23 = secp256r1 26 = brainpoolP256r1 27 = brainpoolP384r1
keyMaterialState.entry.ski	M	Subject Key Identifier of a SHIP node SHALL be provided as <xs:hexBinary>.
keyMaterialState.entry.publicKey	O	Beside the SKI also the corresponding public key MAY be provided as <xs:hexBinary>. The public key SHALL match with the SKI.

3066 Table 30: Structure of SmeConnectionKeyMaterialStateValue of SME "key material state" message

3067

3068 **13.4.7.10 SME "key material state response" message**

3069 The following SHIP nodes SHALL support receiving and sending SME "key material state response":

- 3070 - SHIP nodes with SHIP version $\geq$ SHIP 1.1

3071 The SME "key material state response" message is defined as follows:

3072 MessageType = %x01 ; control

3073 MessageValue = SmeConnectionKeyMaterialStateResponseValue

3074 SmeConnectionKeyMaterialStateResponseValue = *OCTET

3075 The content of SmeConnectionKeyMaterialStateResponseValue is defined as follows: The structure is
 3076 defined by the SHIP root tag "keyMaterialStateResponse" (including the root tag
 3077 "keyMaterialStateResponse") of the XSD "SHIP_TS_TransferProtocol.xsd". The default structure
 3078 extensibility applies to this structure. The format of this structure MUST be the format agreed with
 3079 the protocol handshake (see 13.4.4.2).

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
keyMaterialStateResponse	M	The acknowledgement of a state of key material of a trusted SHIP node that previously sent a "keyMaterialState" message.
keyMaterialStateResponse.accept	M	SHALL be set to true. Note: This just denotes the receipt of the a corresponding received "key material state" message.

3080 *Table 31: Structure of SmeConnectionKeyMaterialStateResponseValue of SME "key material state response" message*

3081

3082 **13.4.7.11 SME "key material state request" message**

3083 The following SHIP nodes SHALL support receiving and sending SME "key material state request":

- 3084 - SHIP nodes with SHIP version $\geq$ SHIP 1.1

3085 The SME "key material state request" message is defined as follows:

3086 MessageType = %x01 ; control

3087 MessageValue = SmeConnectionKeyMaterialStateRequestValue

3088 SmeConnectionKeyMaterialStateRequestValue = *OCTET

3089 The content of SmeConnectionKeyMaterialStateRequestValue is defined as follows: The structure is
 3090 defined by the SHIP root tag "keyMaterialStateRequest" (including the root tag
 3091 "keyMaterialStateRequest") of the XSD "SHIP_TS_TransferProtocol.xsd". The default structure

3092 extensibility applies to this structure. The format of this structure MUST be the format agreed with
 3093 the protocol handshake (see 13.4.4.2).

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
keyMaterialStateRequest	M	Request for the keyMaterialState message of a trusted SHIP node
keyMaterialStateRequest.knownUpdateCounter	M	We assume SHIP node "A" sends SME "key material state request" to SHIP node "B". Then, SHIP node "A" sets "knownUpdateCounter" to the latest known value of "updateCounter" previously received from SHIP node "B". "updateCounter" can be received via SME "hello" message" (see section 13.4.4.1) or via SME "key material state" message (see section 13.4.7.9), e.g. If no "updateCounter" of SHIP node "B" is known yet, "knownUpdateCounter" SHALL be set to 0.

3094 Table 32: Structure of SmeConnectionKeyMaterialStateRequestValue of SME "key material state request" message

3095

3096 13.4.8 Connection Termination

3097 13.4.8.1 Basic Definitions

3098 In this state, the SME Users announce or negotiate the termination of a connection. This denotes the
 3099 regular end of a connection in contrast to a sudden connection interrupt or failure. However, the
 3100 methods described here do NOT apply in general, i.e. they apply only for the states and situations
 3101 described below.

3102 This state can run in parallel to connection data exchange (in order to finish a pending "data"
 3103 message before the connection is finally closed, e.g.). In fact, this state MUST NOT be entered before
 3104 connection data exchange is entered (i.e. the termination process does NOT apply for earlier states
 3105 like "hello" or "CMI", e.g.).

3106 This state uses an SME "close" message which is defined as follows:

3107 MessageType = %x03 ; end

3108 MessageValue = SmeCloseValue

3109 SmeCloseValue = *OCTET

3110 The content of SmeCloseValue is defined as follows: The structure is defined by the SHIP root tag
 3111 "connectionClose" (including the root tag "connectionClose") of the XSD
 3112 "SHIP_TS_TransferProtocol.xsd". The default structure extensibility applies to this structure. The
 3113 format of this structure MUST be JSON-UTF8.

Element name	Mandatory/ Optional/ Not Valid (NV)	Brief explanation
connectionClose.phase	M	The sender's phase during the "close" process (enumeration: announce, confirm).
connectionClose.maxTime	O	Remaining time (in milliseconds) granted by the sender.
connectionClose.reason	O	Reason for the termination. See 13.4.8.1.1.

3114 Table 33: Structure of SmeCloseValue of SME "close" Message.

3115 The SME "close" process does not require knowledge of the connection role (server or client).
 3116 Instead, each of the SME Users SHALL execute the process as described subsequently.

3117 13.4.8.1.1 Process Overview

3118 Either side can initiate a connection termination. The respective other side SHALL confirm the
 3119 termination request accordingly. If – for any reason – a confirmation is not sent or not received in
 3120 time, the requesting side SHALL close the connection and the respective other side SHALL expect the
 3121 connection to be closed.

3122 Subsequently the following symbols are used:

3123 1. **DEV-A**

3124 This symbol is used for a device that initiates a connection termination.

3125 2. **DEV-B**

3126 This symbol is used for the communication partner of DEV-A.

3127 The reason to close a connection SHALL also be part of the message (sub-element "reason"). The
 3128 following reasons are defined:

3129 1. **unspecific**

3130 This value SHALL be used if no other value fits better.

3131 Remark (informative): This value typically denotes a rather temporary disconnection (e.g.
 3132 because a device has limited connection capabilities – it may just support one active connection
 3133 but needs to exchange data with multiple devices; e.g. the device needs to reboot for a firmware
 3134 update). This means it is likely that it is possible to re-establish a connection later on in order to
 3135 continue with next/remaining data exchange.

3136 2. **removedConnection**

3137 This value denotes the removal of the respective node from the list of "known" or "accepted"
 3138 devices.

3139 Remark (informative): This value does not mean a reconnection will not be possible at a later
3140 time. However, assuming a reconnection is executed, the device will be treated like a new or
3141 unknown device.

3142 13.4.8.1.2 Process Details

3143 Rules for DEV-A:

- 3144 1. If an SME User wants to initiate a connection termination, it SHALL send an SME "close" message
3145 to the communication partner with the following content:
 - 3146 a. Sub-element "phase" set to "announce".
 - 3147 b. Sub-element "maxTime" set to a value when DEV-A will close the connection at the latest
3148 (i.e. even if no confirmation from DEV-B was received). The value denotes the duration in ms
3149 (milliseconds), starting from the time the message is sent.
 - 3150 c. Sub-element "reason" set to a proper value (see 13.4.8.1.1).
- 3151 2. If an SME User initiated a connection termination, it SHALL close the connection latest after a
3152 duration of its announced "maxTime". I.e. the connection SHALL then be closed even if no
3153 confirmation from DEV-B was received.
- 3154 3. If an SME User initiated a connection termination and receives a confirmation from DEV-B in time
3155 (i.e. before the announced duration "maxTime" elapsed) it SHALL close the connection
3156 immediately.

3157 Rules for DEV-B:

- 3158 1. If an SME User receives a connection termination, it SHALL prepare stopping its state
3159 "connection data exchange" before the received duration of "maxTime" expires. If this was
3160 achieved in time and the connection is still not closed, it SHALL send an SME "close" message to
3161 the communication partner, with sub-element "phase" set to "confirm" and no other sub-
3162 element set. Afterwards, it SHALL close the connection.
- 3163 2. If an SME User receives a connection termination but does not manage to submit the
3164 confirmation in time, it SHALL consider the connection as closed after the received duration of
3165 "maxTime" expired.

3166 General rules:

- 3167 1. It can happen that both sides initiate a connection termination at almost the same time. In this
3168 case, each side is both a DEV-A as well as a DEV-B (with different parameters, esp. different
3169 "maxTime"). In this case, the confirmation that is sent first closes the connection (i.e. there is no
3170 need for a confirmation from both sides). However, the respectively received "reason" values
3171 need to be considered with regards to the importance. I.e. a received "removedConnection" is
3172 more important than the value "unspecific".

3173 The execution of reconnection attempts is application specific in general. However, in case of a
3174 regular termination process, it SHOULD be avoided to attempt a reconnection immediately.

3175

14 Well-known protocolId

protocolId	Definition
ee1.0	EEBus SPINE specifications that are compatible to the SPINE data model specification base version 1.0.

3176

3177